

Wireless Communication

Why mobile systems need communication—
and why it must be wireless

Yaxiong Xie
Fall 2026



A mobile system needs information beyond the device

- **Other devices**
coordinate motion · share state · avoid conflicts
- **Nearby infrastructure**
maps · localization · sensing · access
- **Remote computation**
data · services · edge / cloud compute



Without communication, a mobile device is isolated: it can use only local sensing and compute.

A moving system cannot depend on a fixed cable

- **The endpoint moves**
position · path · orientation keep changing
- **A cable has a boundary**
finite length · fixed attachment · constrained route
- **The link must persist**
communication continues while the system moves



Wireless lets the information move while the device moves.

01

Turn bits into a radio waveform

Start with the simplest question: which property of a physical signal can represent 0 and 1?

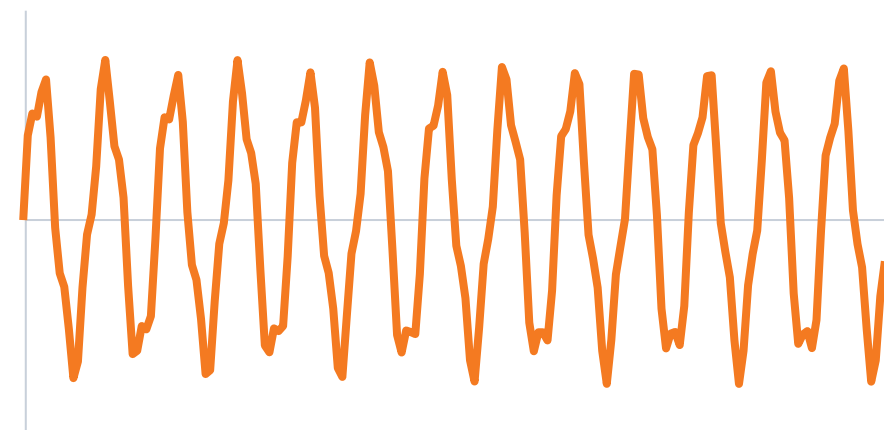
Bits cannot travel through the air by themselves

10110100

abstract values



signal



time

voltage / current / electromagnetic field

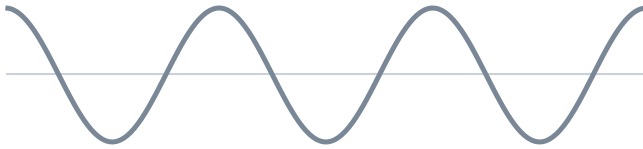
Modulation is the rule that maps information to physically distinguishable signal states.

First knob: amplitude changes signal height

$$s(t) = A \cos(2\pi f t + \varphi)$$

Change amplitude A

reference: $A = 1 \cdot f = 3 \cdot \varphi = 0^\circ$



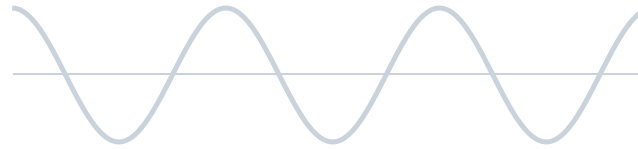
changed: $A = 0.45$



f and φ stay fixed

Change frequency f

reference: $A = 1 \cdot f = 3 \cdot \varphi = 0^\circ$



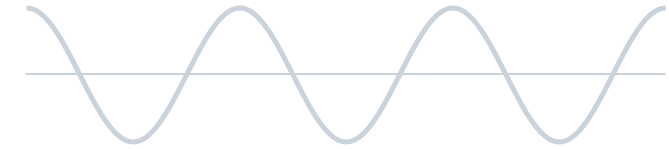
changed: $f = 5$ cycles



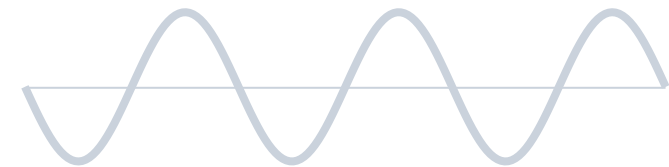
A and φ stay fixed

Change phase φ

reference: $A = 1 \cdot f = 3 \cdot \varphi = 0^\circ$



changed: $\varphi = 90^\circ$



A and f stay fixed

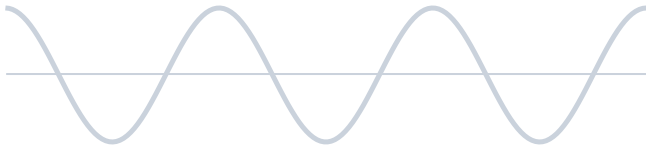
Change A ; keep frequency f and phase φ fixed.

Second knob: frequency changes cycles per second

$$s(t) = A \cos(2\pi f t + \varphi)$$

Change amplitude A

reference: $A = 1 \cdot f = 3 \cdot \varphi = 0^\circ$



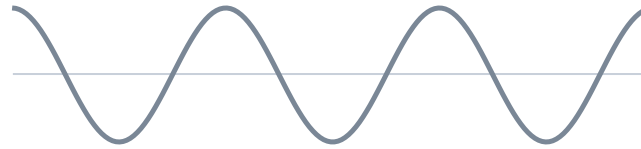
changed: $A = 0.45$



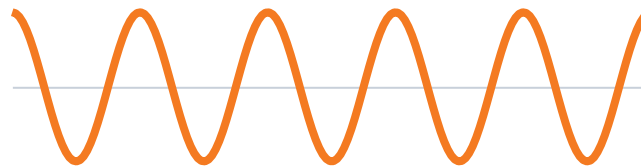
f and φ stay fixed

Change frequency f

reference: $A = 1 \cdot f = 3 \cdot \varphi = 0^\circ$



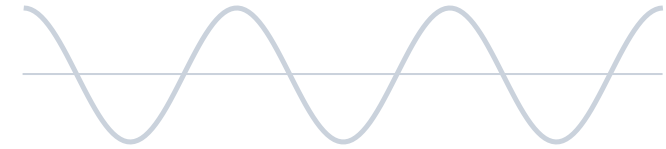
changed: $f = 5$ cycles



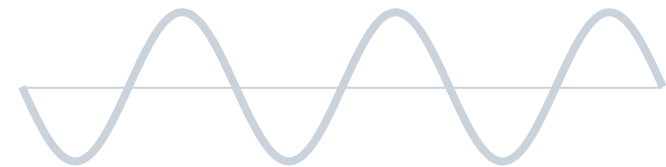
A and φ stay fixed

Change phase φ

reference: $A = 1 \cdot f = 3 \cdot \varphi = 0^\circ$



changed: $\varphi = 90^\circ$



A and f stay fixed

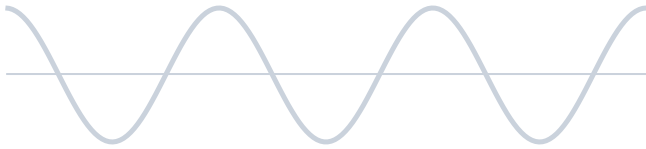
Change f ; keep amplitude A and phase φ fixed.

Third knob: phase shifts the waveform in time

$$s(t) = A \cos(2\pi f t + \varphi)$$

Change amplitude A

reference: $A = 1 \cdot f = 3 \cdot \varphi = 0^\circ$



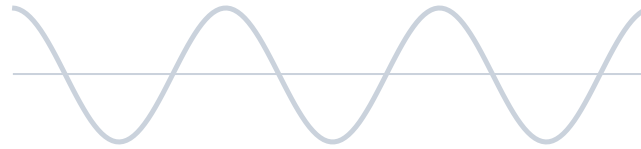
changed: $A = 0.45$



f and φ stay fixed

Change frequency f

reference: $A = 1 \cdot f = 3 \cdot \varphi = 0^\circ$



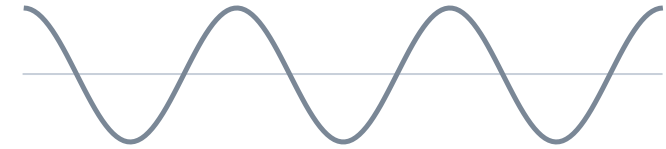
changed: $f = 5$ cycles



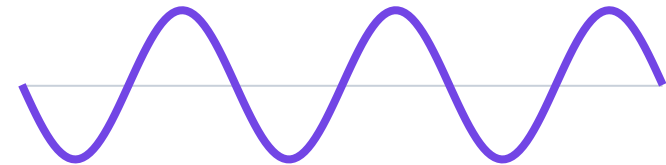
A and φ stay fixed

Change phase φ

reference: $A = 1 \cdot f = 3 \cdot \varphi = 0^\circ$



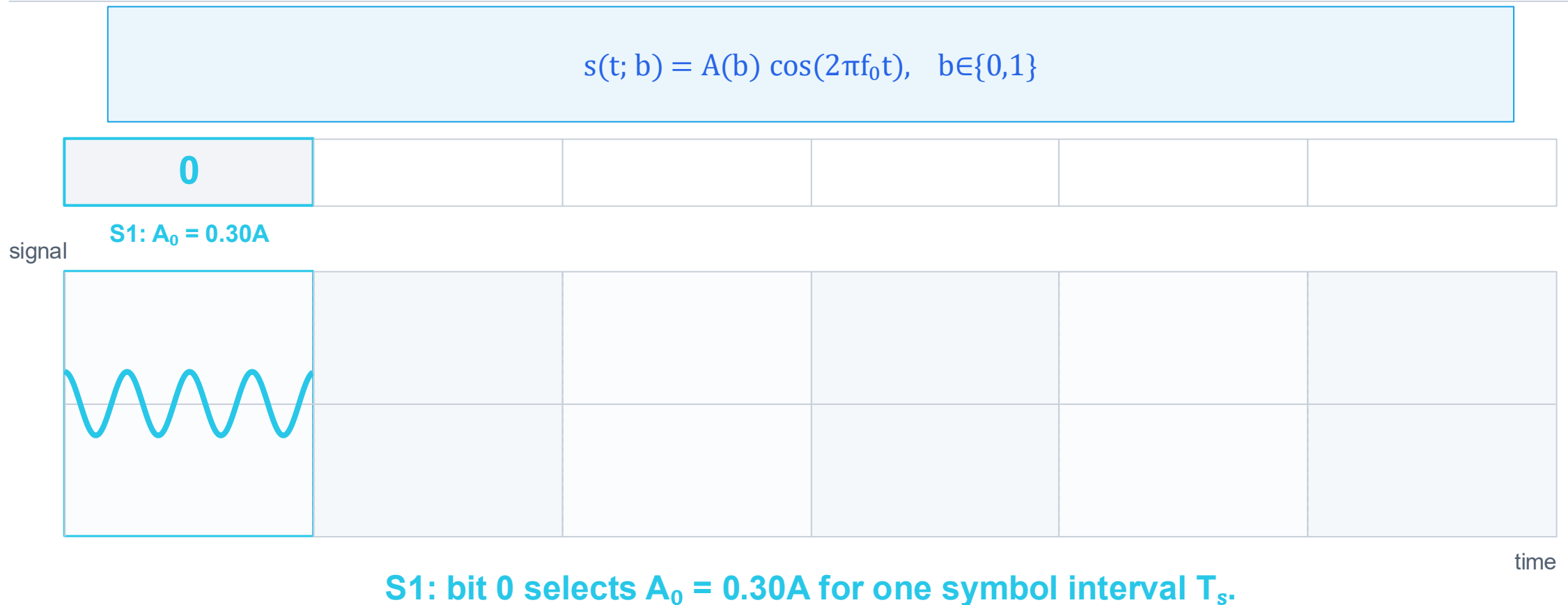
changed: $\varphi = 90^\circ$



A and f stay fixed

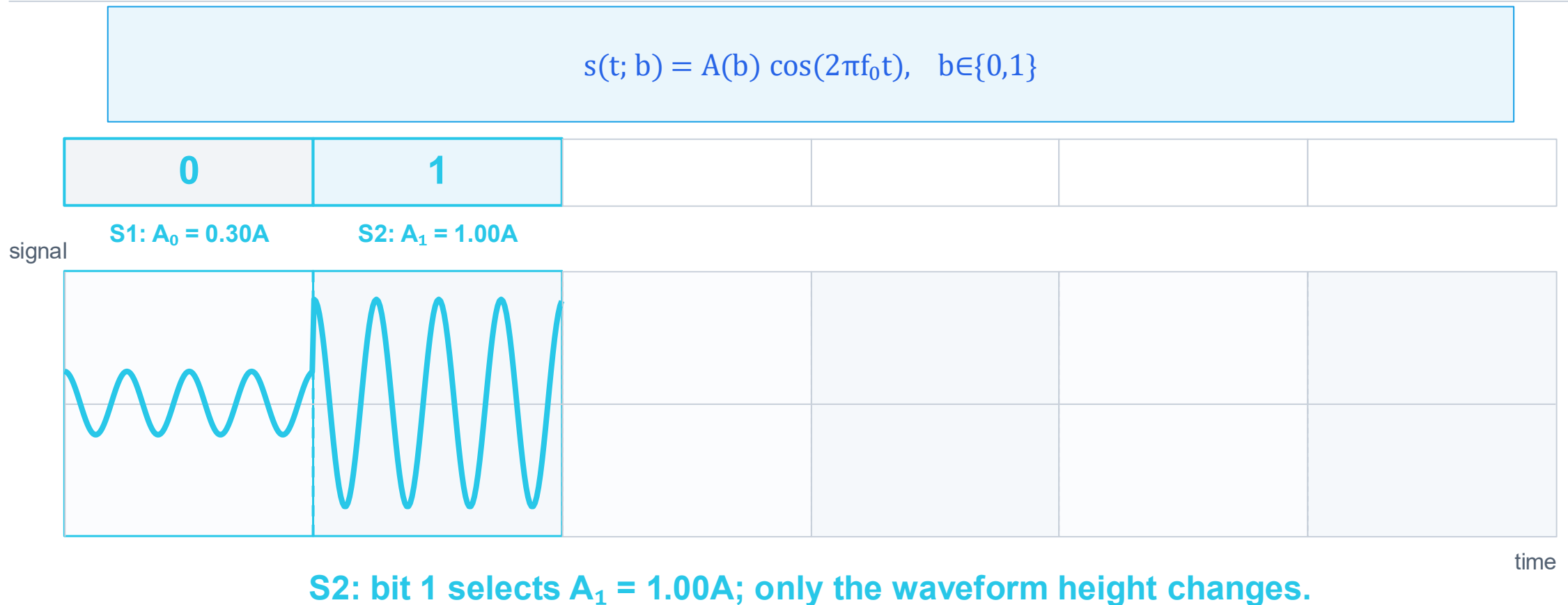
Change φ ; keep amplitude A and frequency f fixed.

This is amplitude modulation: bits select different amplitudes



For digital bits, this binary form of amplitude modulation is amplitude-shift keying (ASK).

This is amplitude modulation: bits select different amplitudes



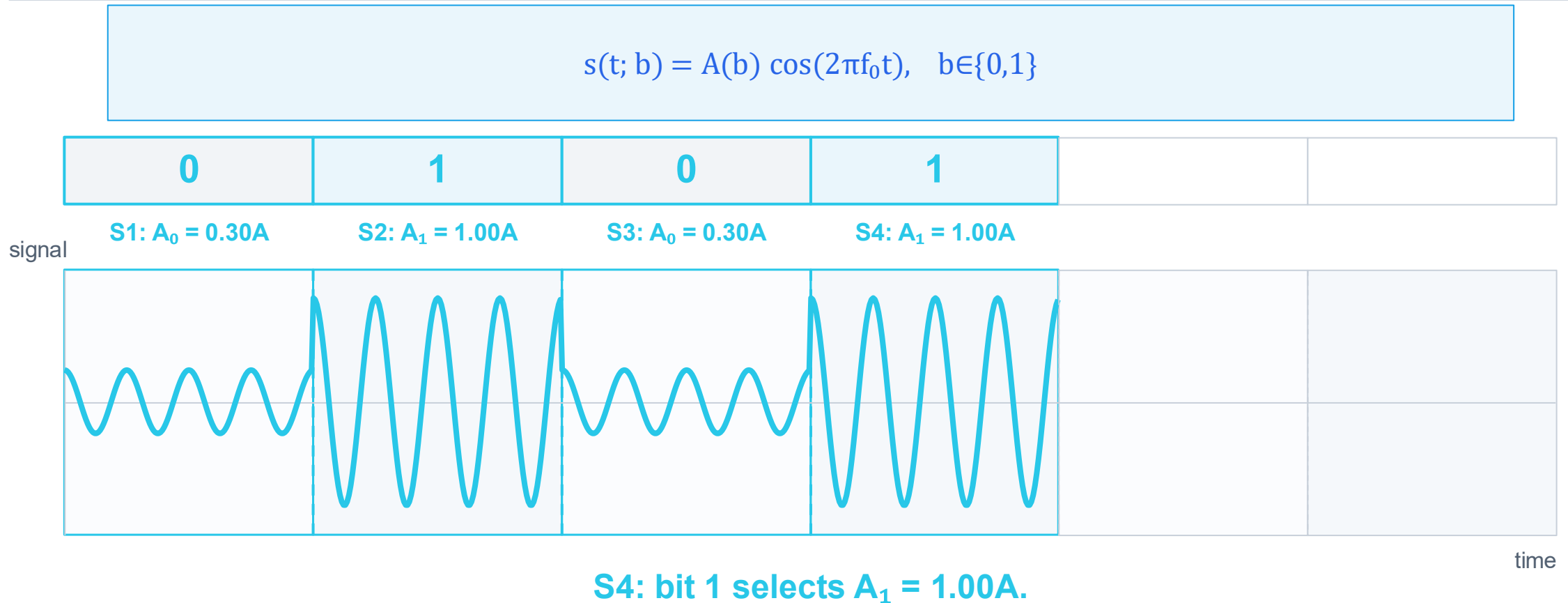
For digital bits, this binary form of amplitude modulation is amplitude-shift keying (ASK).

This is amplitude modulation: bits select different amplitudes



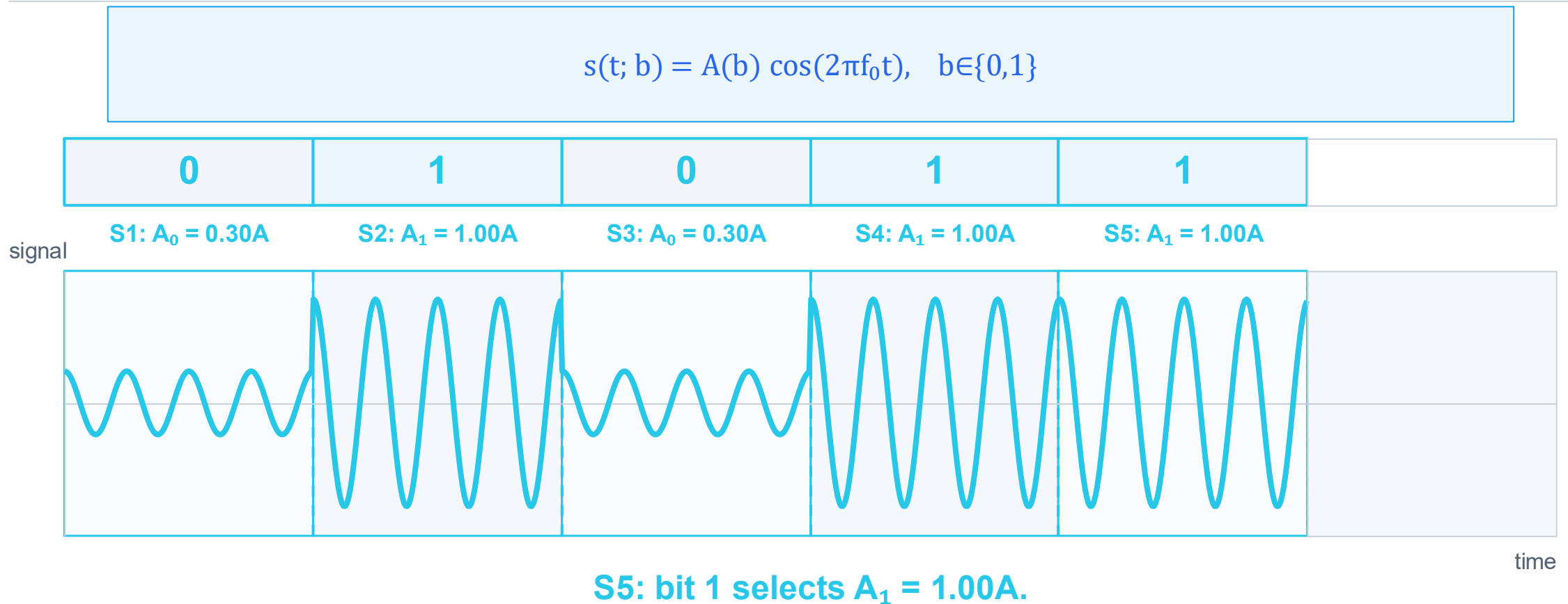
For digital bits, this binary form of amplitude modulation is amplitude-shift keying (ASK).

This is amplitude modulation: bits select different amplitudes



For digital bits, this binary form of amplitude modulation is amplitude-shift keying (ASK).

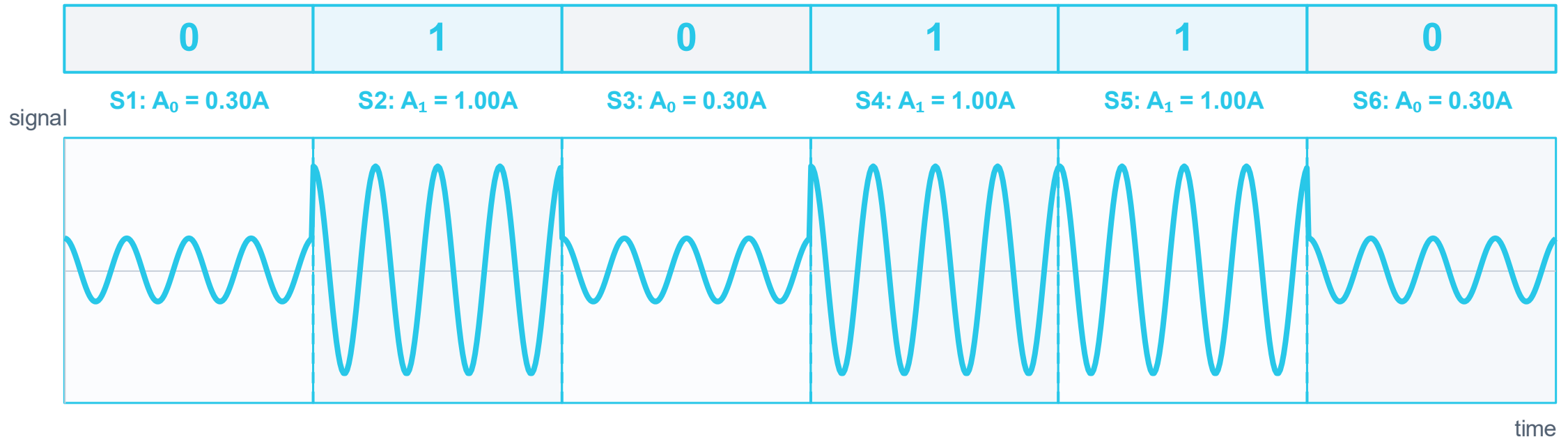
This is amplitude modulation: bits select different amplitudes



For digital bits, this binary form of amplitude modulation is amplitude-shift keying (ASK).

This is amplitude modulation: bits select different amplitudes

$$s(t; b) = A(b) \cos(2\pi f_0 t), \quad b \in \{0, 1\}$$

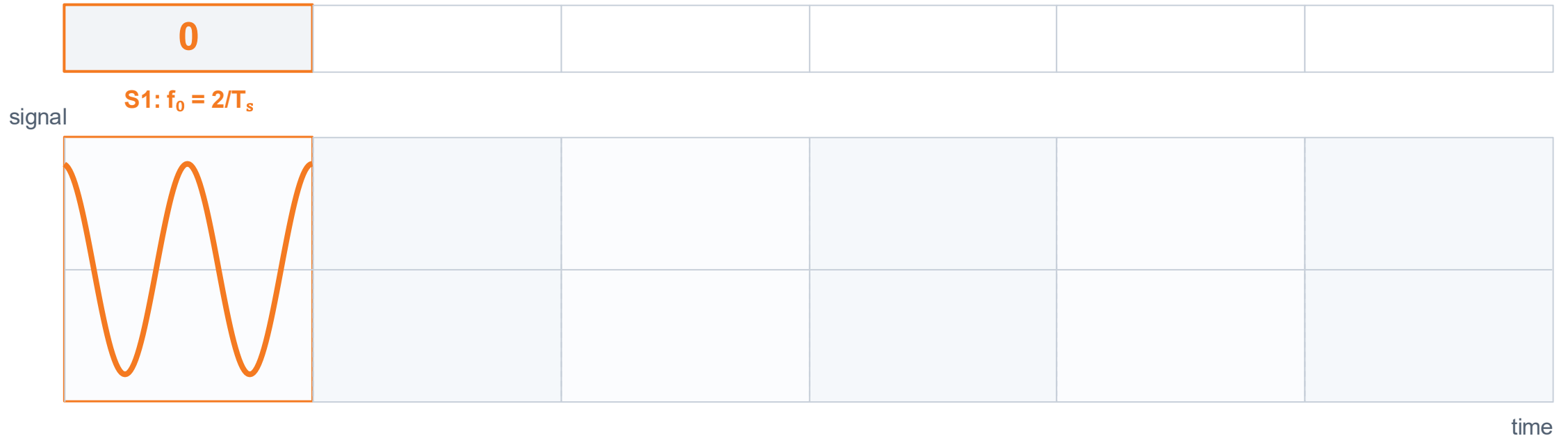


S6 completes the sequence: every bit independently selects A_0 or A_1 .

For digital bits, this binary form of amplitude modulation is amplitude-shift keying (ASK).

This is frequency modulation: bits select different frequencies

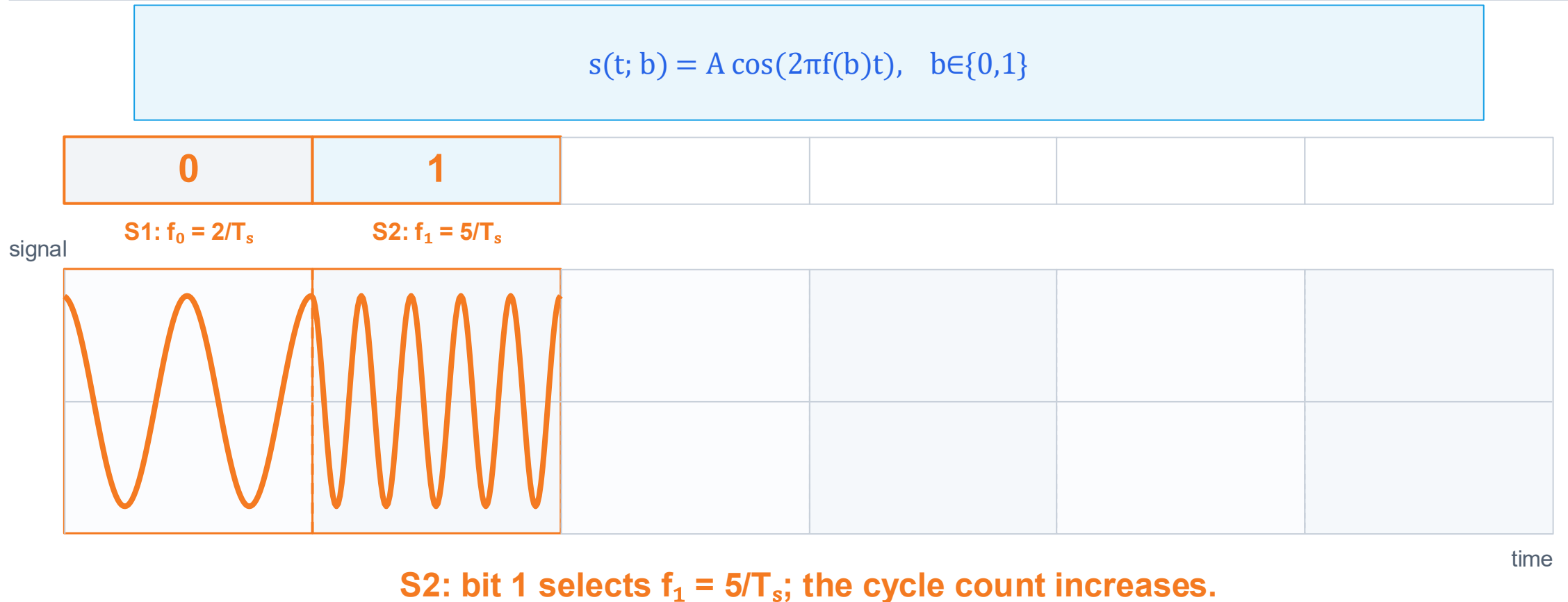
$$s(t; b) = A \cos(2\pi f(b)t), \quad b \in \{0, 1\}$$



S1: bit 0 selects $f_0 = 2/T_s$ for one symbol interval T_s .

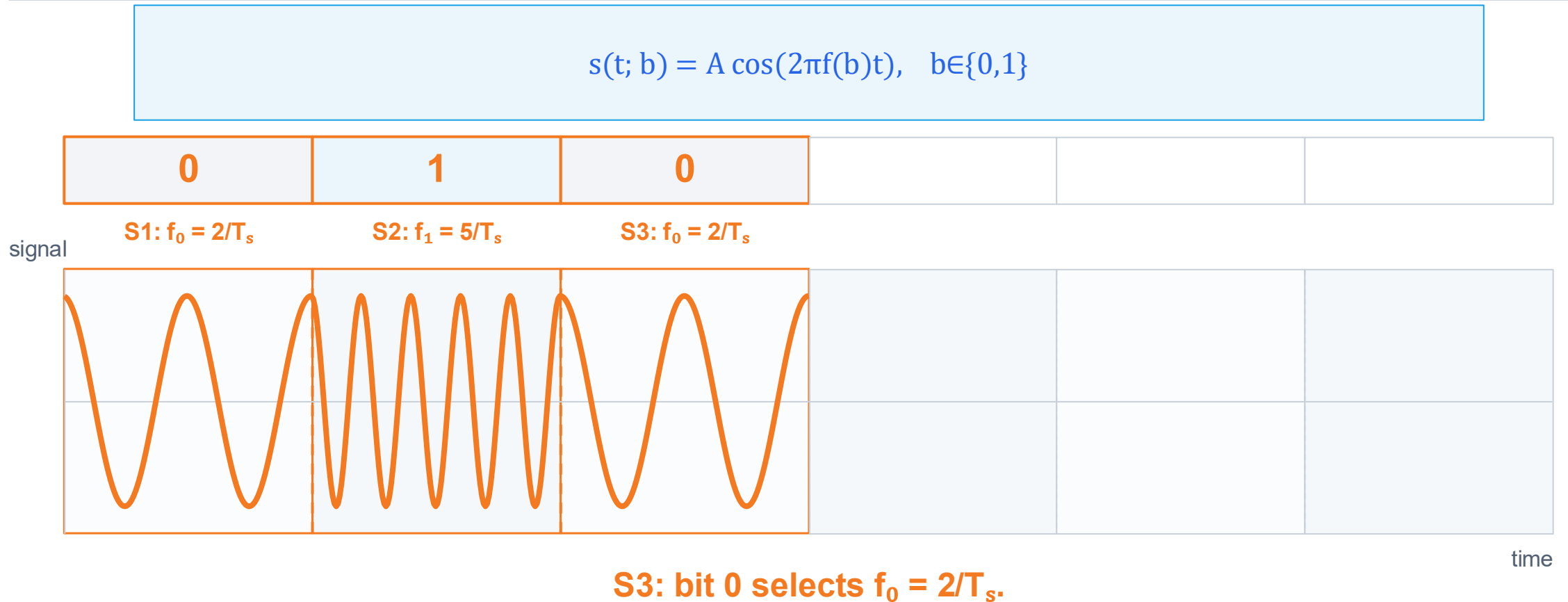
For digital bits, this binary form of frequency modulation is frequency-shift keying (FSK).

This is frequency modulation: bits select different frequencies



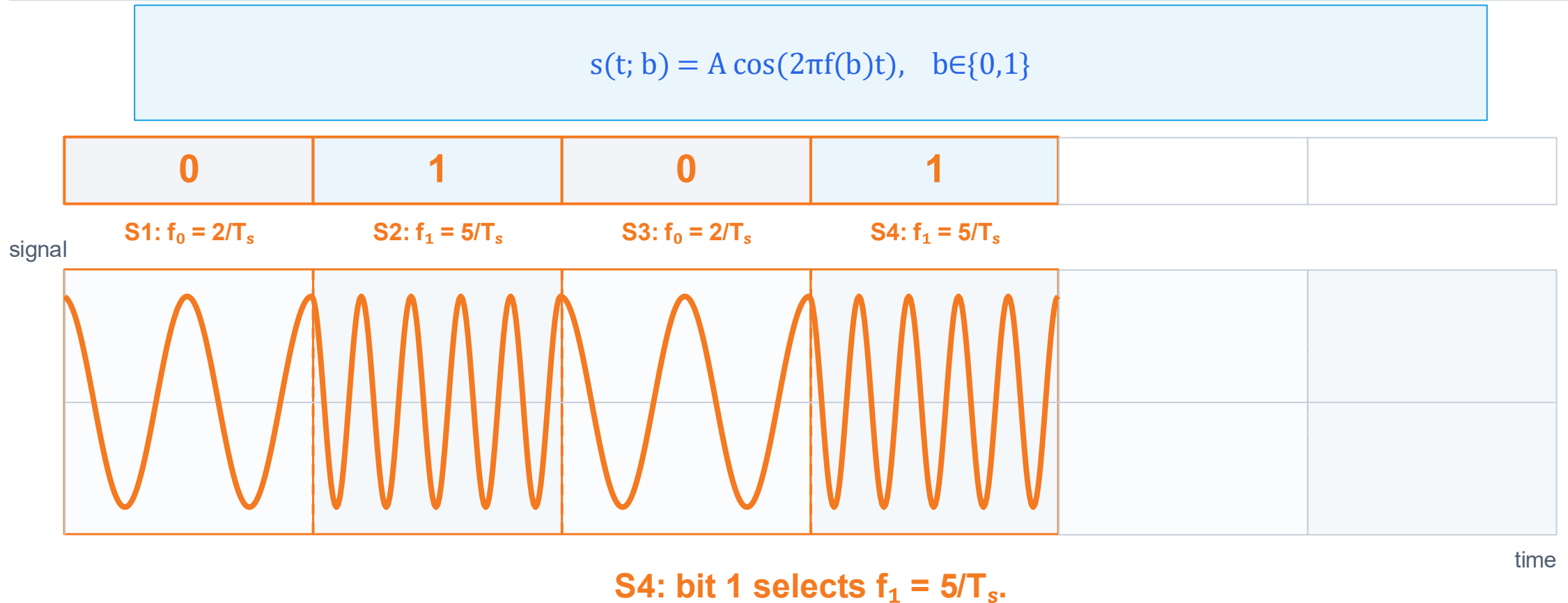
For digital bits, this binary form of frequency modulation is frequency-shift keying (FSK).

This is frequency modulation: bits select different frequencies



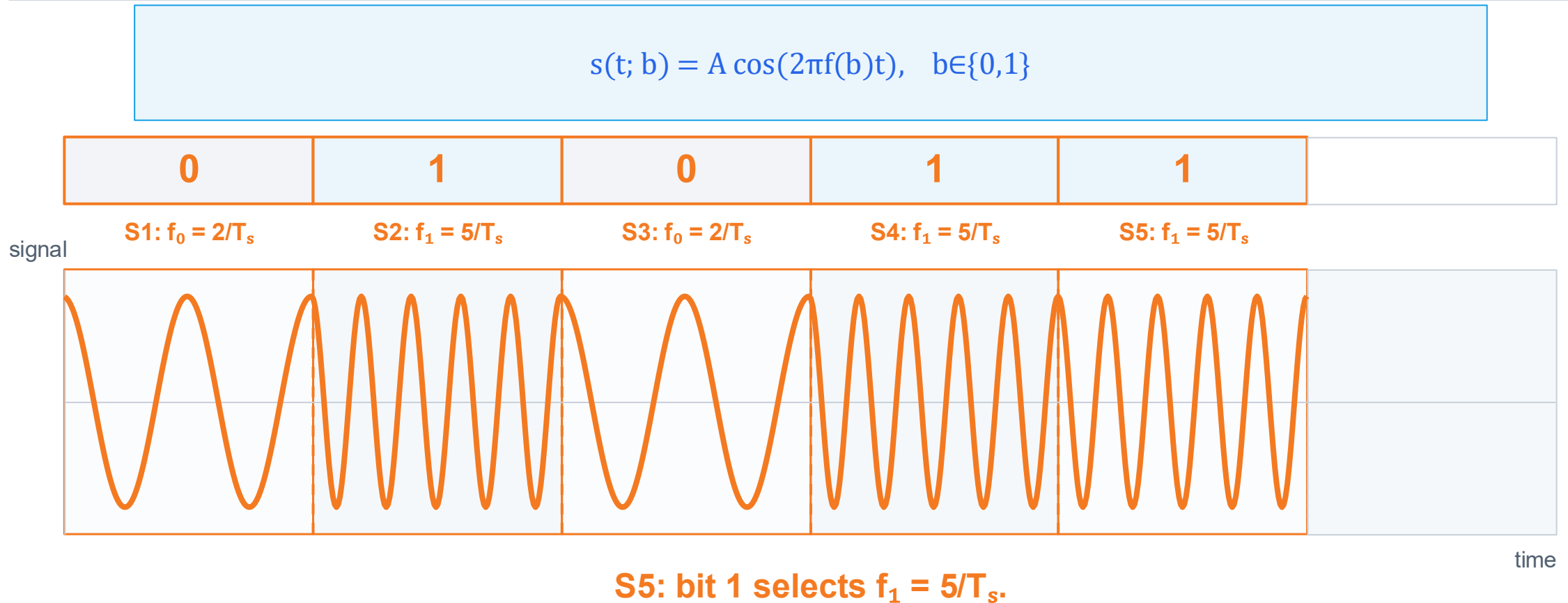
For digital bits, this binary form of frequency modulation is frequency-shift keying (FSK).

This is frequency modulation: bits select different frequencies



For digital bits, this binary form of frequency modulation is frequency-shift keying (FSK).

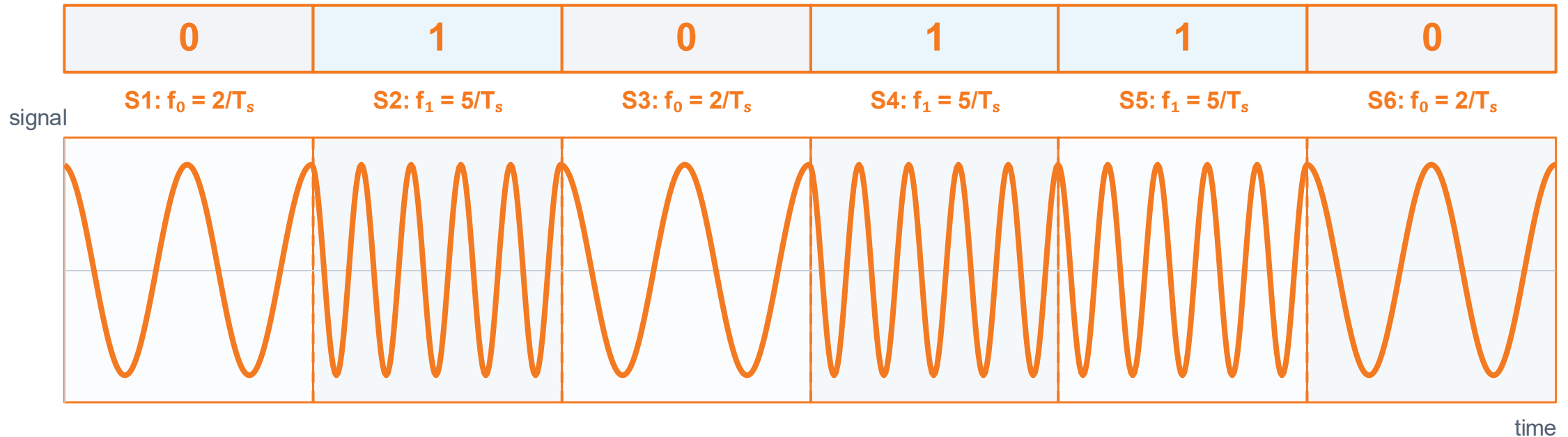
This is frequency modulation: bits select different frequencies



For digital bits, this binary form of frequency modulation is frequency-shift keying (FSK).

This is frequency modulation: bits select different frequencies

$$s(t; b) = A \cos(2\pi f(b)t), \quad b \in \{0, 1\}$$



S6 completes the sequence: every bit independently selects f_0 or f_1 .

For digital bits, this binary form of frequency modulation is frequency-shift keying (FSK).

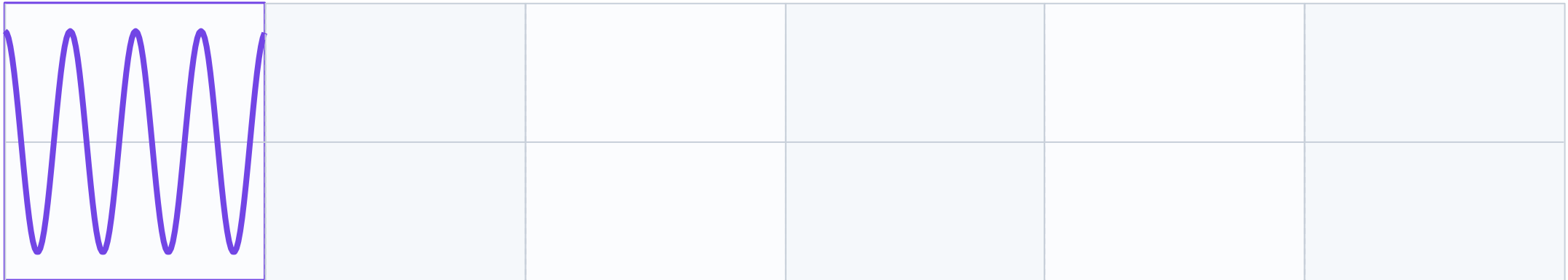
This is phase modulation: bits select different phases

$$s(t; b) = A \cos(2\pi f_0 t + \pi b), \quad b \in \{0, 1\}$$

0

S1: $\varphi_0 = 0^\circ$

signal

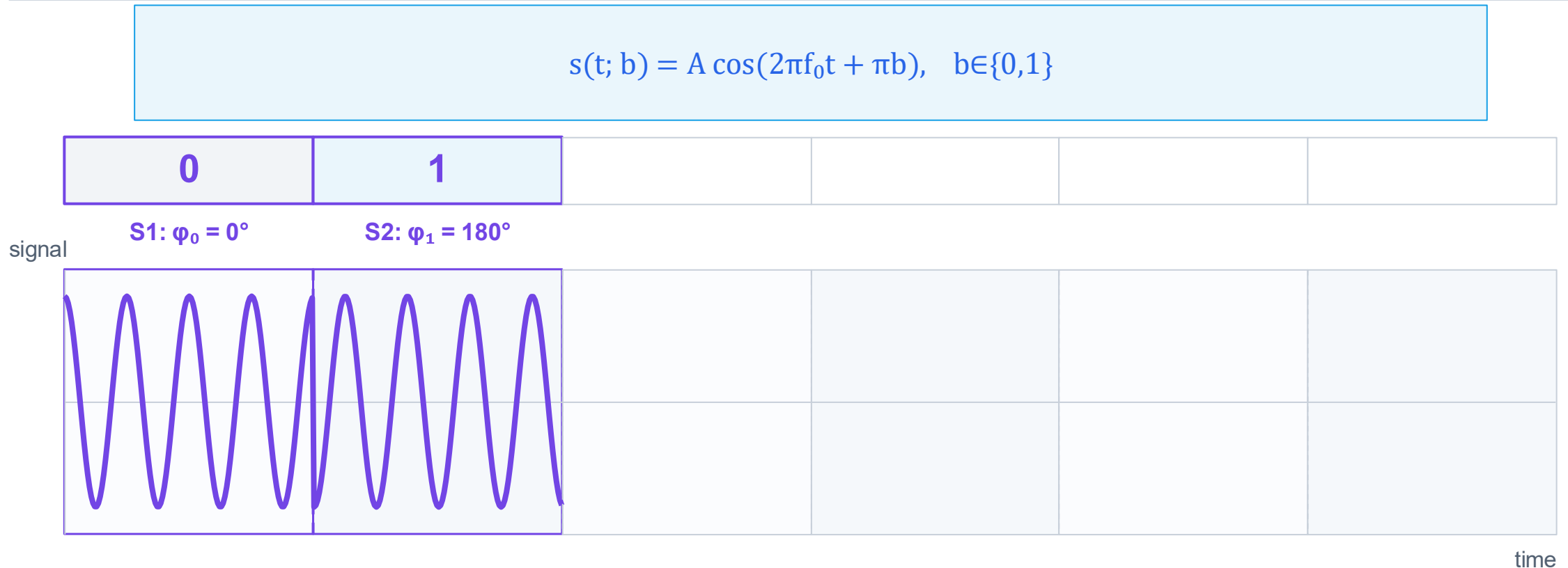


time

S1: bit 0 selects the reference phase $\varphi_0 = 0^\circ$.

Binary phase-shift keying (BPSK) uses two phases separated by 180° .

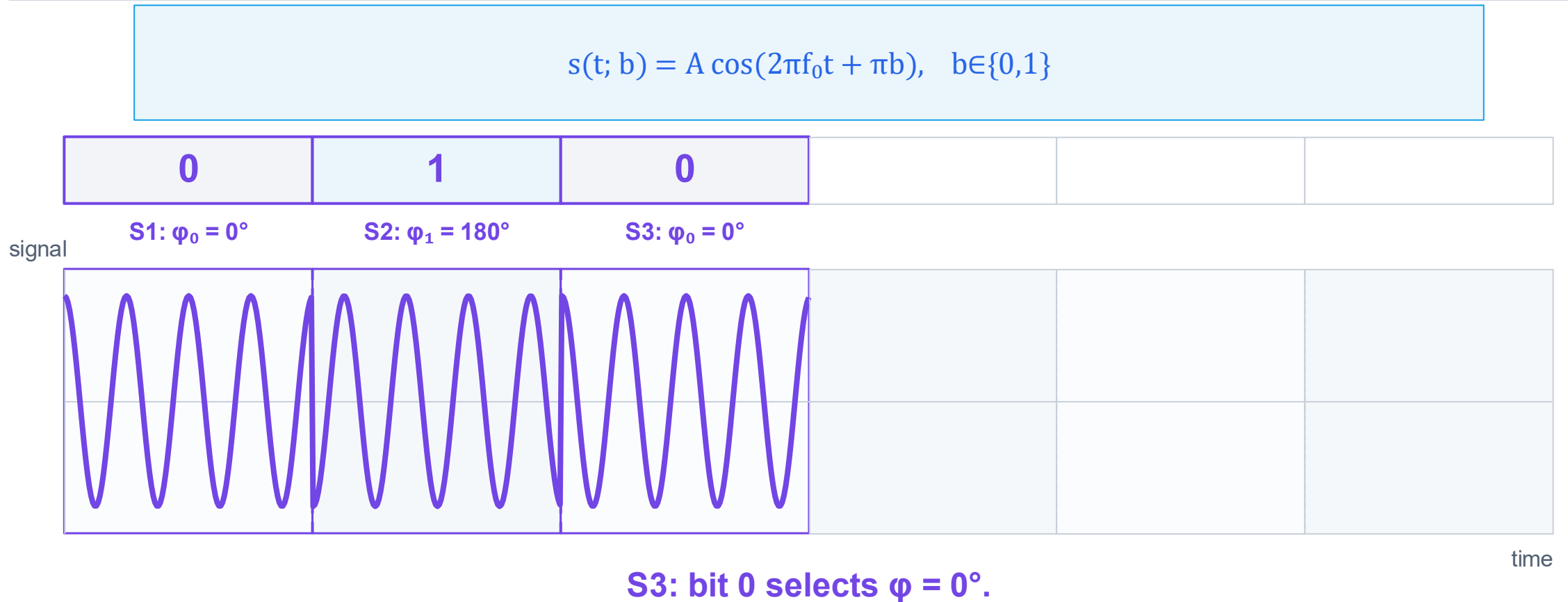
This is phase modulation: bits select different phases



S2: bit 1 selects $\varphi_1 = 180^\circ$ —the carrier is shifted by half a cycle.

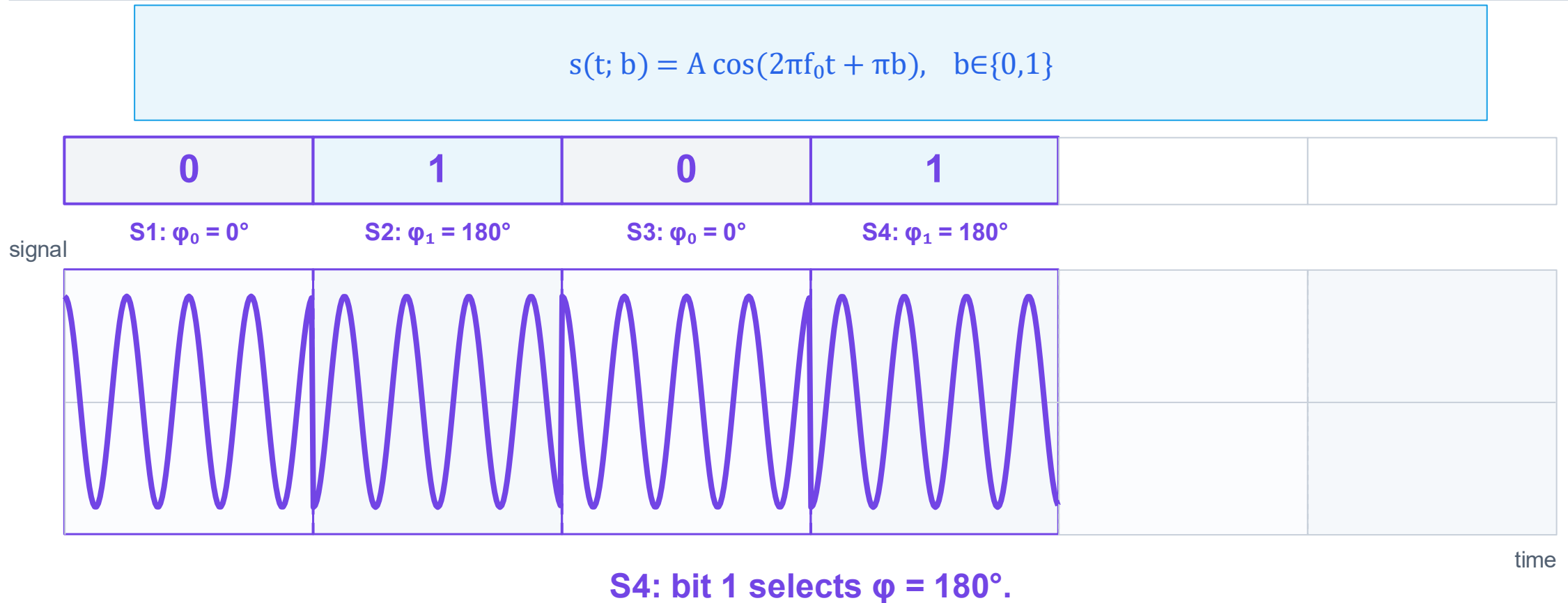
Binary phase-shift keying (BPSK) uses two phases separated by 180° .

This is phase modulation: bits select different phases



Binary phase-shift keying (BPSK) uses two phases separated by 180° .

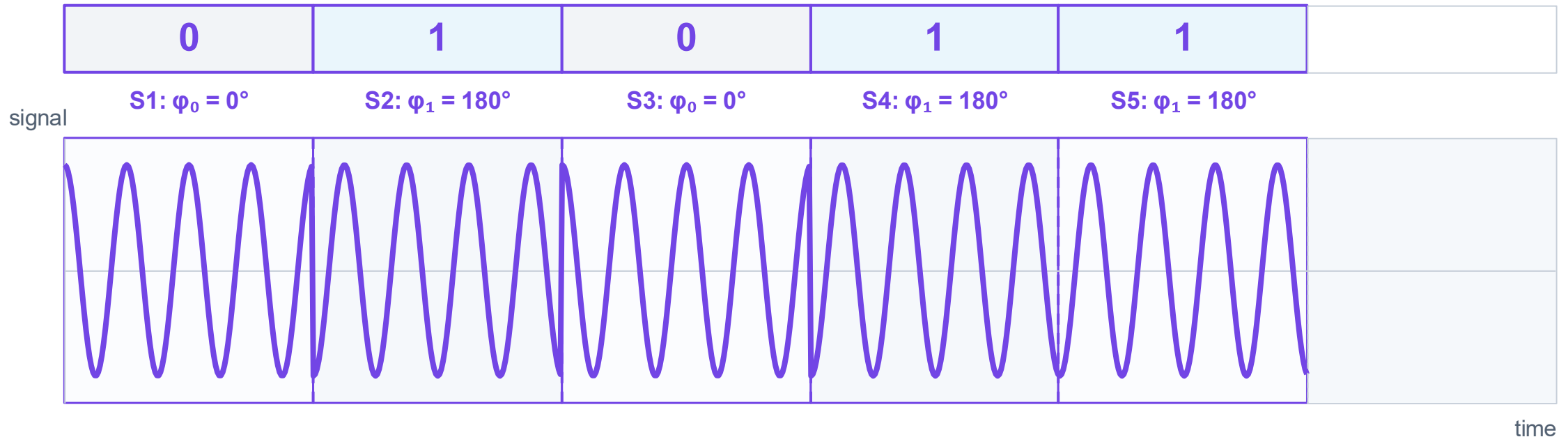
This is phase modulation: bits select different phases



Binary phase-shift keying (BPSK) uses two phases separated by 180° .

This is phase modulation: bits select different phases

$$s(t; b) = A \cos(2\pi f_0 t + \pi b), \quad b \in \{0, 1\}$$

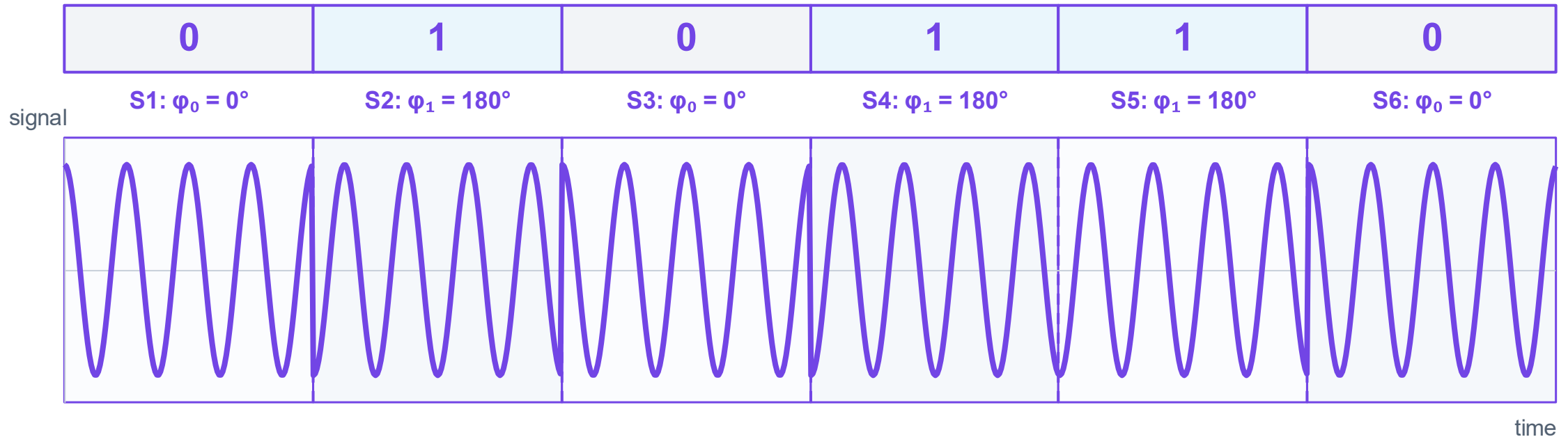


S5: another bit 1 keeps $\varphi = 180^\circ$ across the next symbol boundary.

Binary phase-shift keying (BPSK) uses two phases separated by 180° .

This is phase modulation: bits select different phases

$$s(t; b) = A \cos(2\pi f_0 t + \pi b), \quad b \in \{0, 1\}$$



S6: bit 0 returns to $\varphi = 0^\circ$; the full phase sequence is now visible.

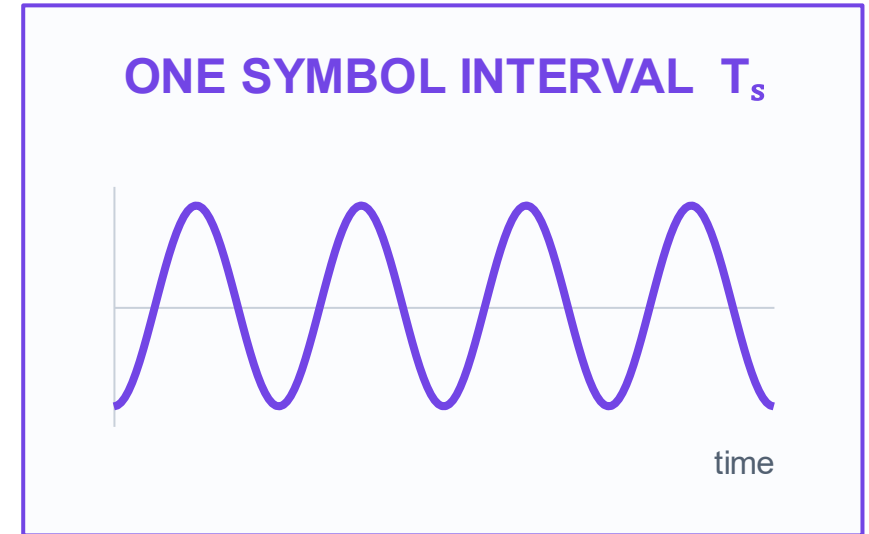
Binary phase-shift keying (BPSK) uses two phases separated by 180° .

A symbol is one signal choice during one time interval

input bit



choose one of
 $M = 2$ allowed
states



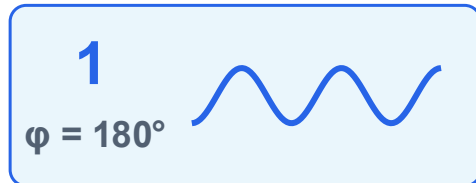
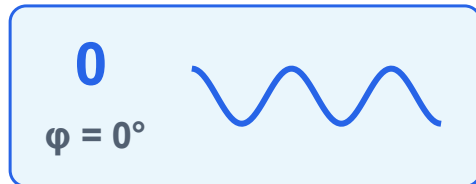
symbol = selected signal state + its interval T_s

Binary ASK, FSK, and BPSK have $M = 2$ states, so each symbol interval carries one bit.

Use phase for two states, then add amplitude for more states

$$\text{bits per symbol} = \log_2 M$$

PHASE ONLY: 0° vs 180°



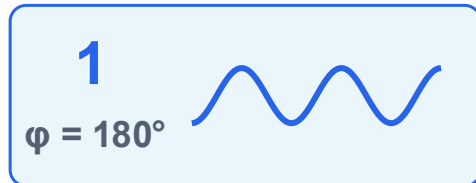
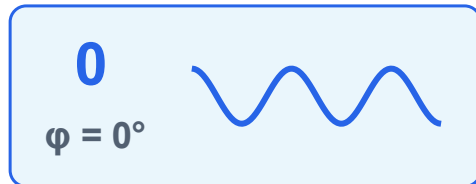
2 phases $\rightarrow$ 2 states $\rightarrow$ 1 bit/symbol

BPSK uses phase alone: $\varphi = 0^\circ$ represents one state and $\varphi = 180^\circ$ represents the other.

Use phase for two states, then add amplitude for more states

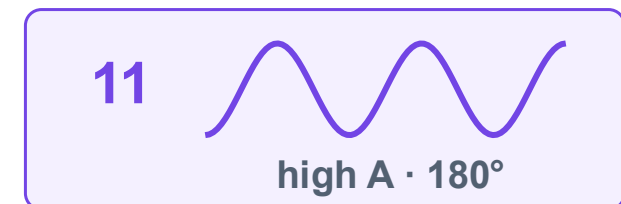
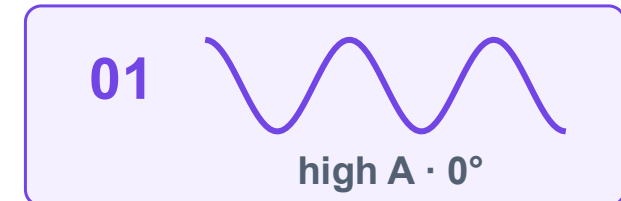
$$\text{bits per symbol} = \log_2 M$$

PHASE ONLY: 0° vs 180°



2 phases $\rightarrow$ 2 states $\rightarrow$ 1 bit/symbol

ADD AMPLITUDE: 2 amplitudes $\times$ 2 phases



4 states $\rightarrow$ 2 bits/symbol

Keep the two phases and add two amplitudes: $2 \times 2 = 4$ states, so one symbol carries two bits.

Keep the same symbol clock; carry more bits in each symbol

$$R_\beta = R_s \log_2 M$$

bit rate = symbol rate × bits per symbol

M = 2
1 bit/symbol



6 bits

With M = 2, six symbol intervals carry six bits.

Keep the same symbol clock; carry more bits in each symbol

$$R_\beta = R_s \log_2 M$$

bit rate = symbol rate × bits per symbol

M = 2
1 bit/symbol



6 bits

M = 4
2 bits/symbol



12 bits

The same six intervals carry twelve bits when each symbol selects one of four states.

Keep the same symbol clock; carry more bits in each symbol

$$R_\beta = R_s \log_2 M$$

bit rate = symbol rate × bits per symbol

M = 2
1 bit/symbol



6 bits

M = 4
2 bits/symbol



12 bits

M = 16
4 bits/symbol



24 bits

How do we create many states? Add independent modulation dimensions—next: I and Q.

One bit asks the transmitter to choose between two signal states

1

bit / symbol

$$M = 2^1 = 2 \text{ states}$$

A single bit only asks for one binary physical choice.

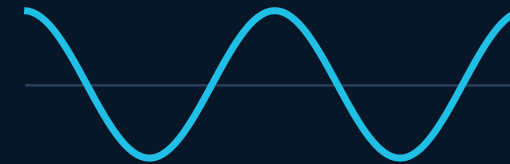
TWO WAYS TO BUILD A 2-STATE SIGNAL ALPHABET

AMPLITUDE · ASK

0 low A



1 high A

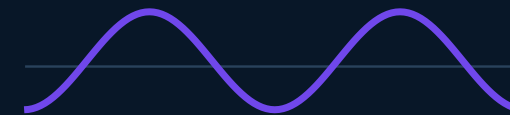


PHASE · BPSK

0 $\phi = 0^\circ$



1 $\phi = 180^\circ$



With one bit, changing either amplitude or phase is enough to create two distinguishable states.

Two bits need four choices—use amplitude and phase together

2

bits / symbol

$$M = 2^2 = 4 \text{ states}$$

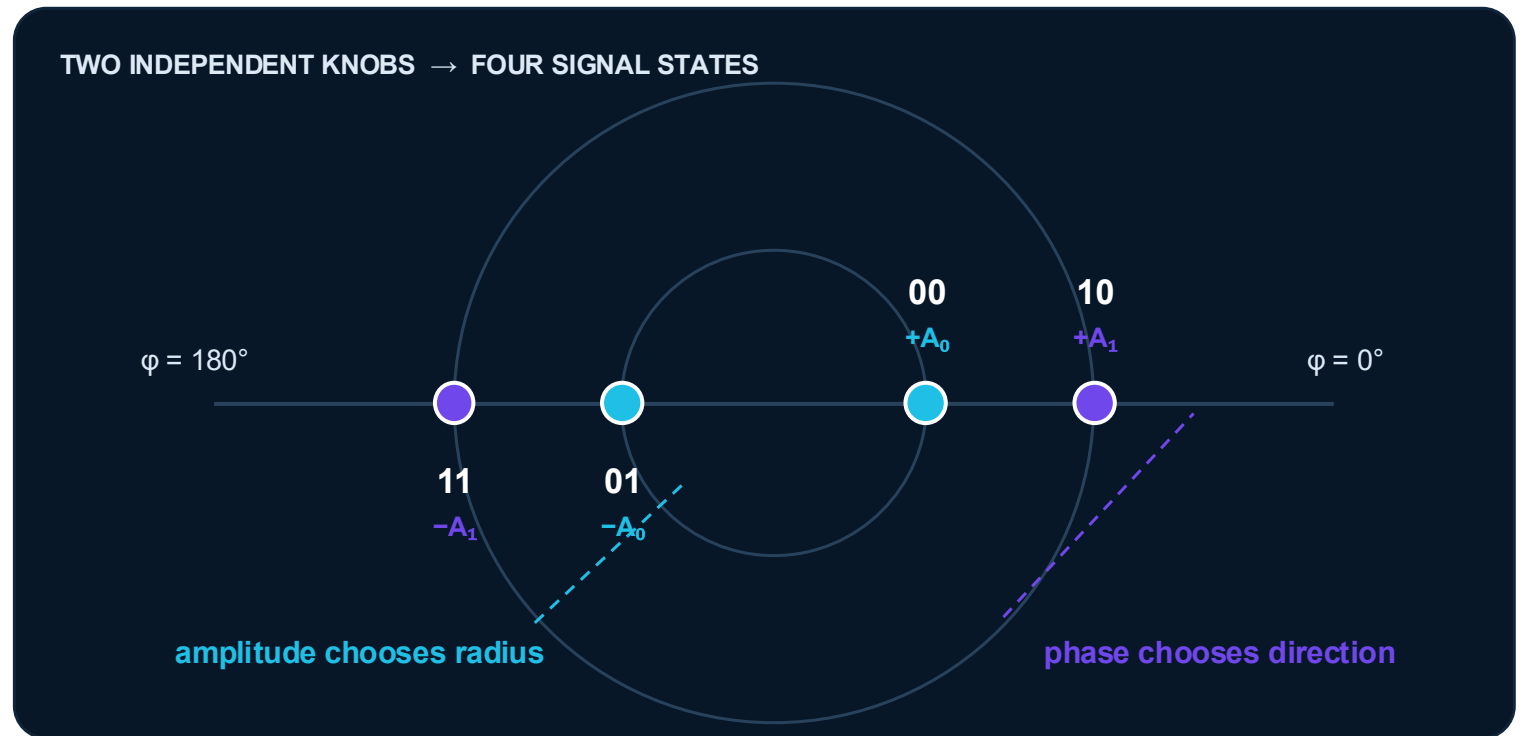
Let one bit choose radius and the other choose direction.

 b_1

amplitude: low / high

 b_0 phase: 0° / 180°

Two binary controls create $2 \text{ amplitudes} \times 2 \text{ phases} = 4 \text{ distinguishable states}$.



Four, six, and eight bits make the signal alphabet explode

k

bits / symbol

$$M = 2^k \text{ states}$$

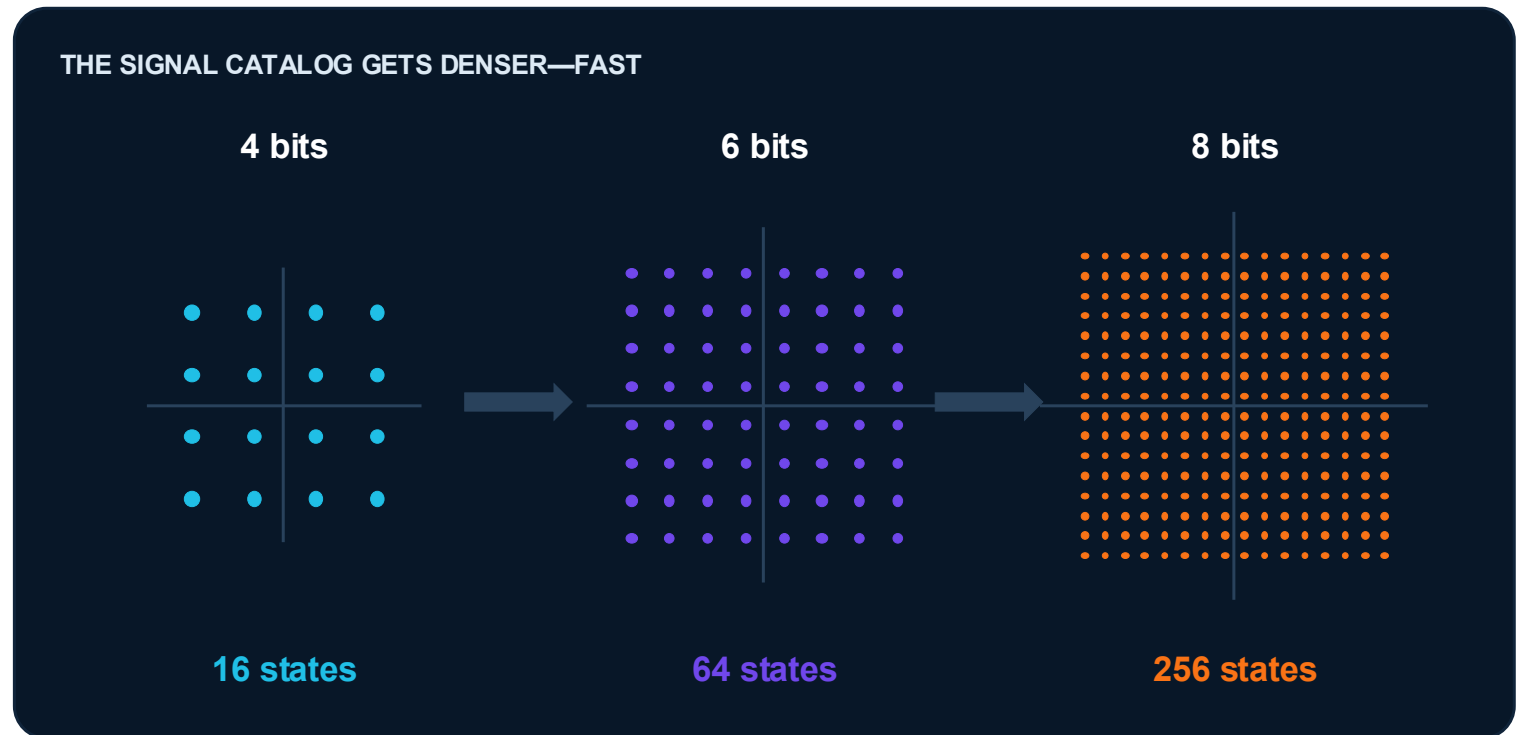
Each extra bit doubles the receiver's hypothesis set.

4 bits → 16

6 bits → 64

8 bits → 256

The count grows exponentially; we need a coordinate recipe that expands cleanly for any k.



Orthogonal directions have zero projection onto each other

$$\mathbf{v} = a \mathbf{e}_1 + b \mathbf{e}_2$$

A basis provides directions; the coordinates tell us how much of each direction is present.

$$\mathbf{e}_1 \cdot \mathbf{e}_2 = 0$$

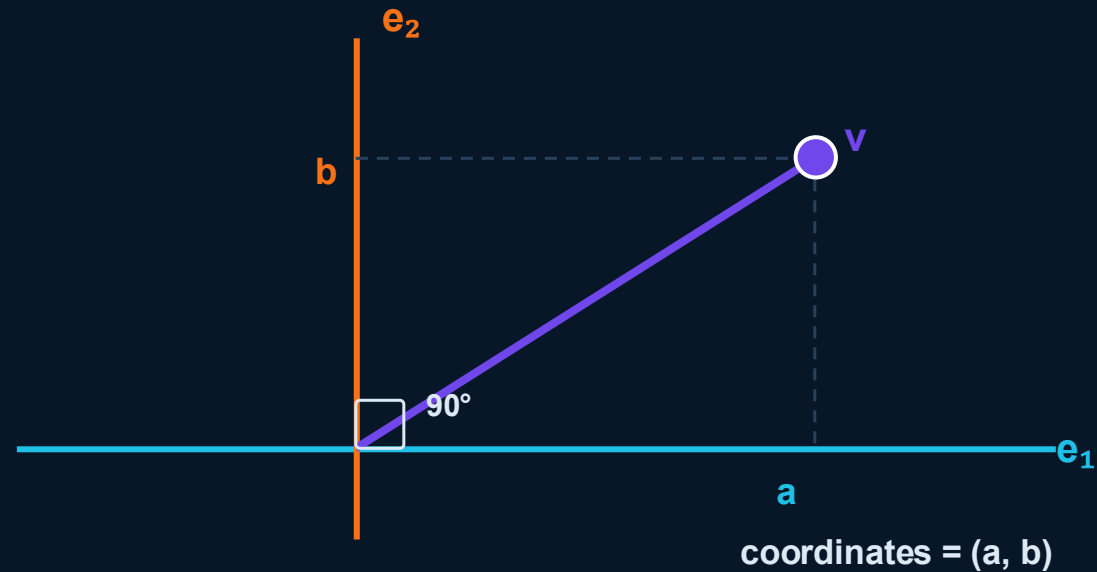
Zero dot product means neither axis projects onto the other.

For waveforms:

$$\langle \varphi_1, \varphi_2 \rangle = \int_0^T \varphi_1(t) \varphi_2(t) dt = 0$$

Orthogonality is the property we need: measuring one basis direction does not pick up the other.

ORTHOGONAL = ZERO CROSS-PROJECTION



Cosine and sine provide two waveform directions over one symbol

$$0 \leq t \leq T_s \quad \text{and} \quad f_0 T_s = N \in \mathbb{Z}$$

TWO NORMALIZED WAVEFORM DIRECTIONS OVER THE SAME SYMBOL INTERVAL

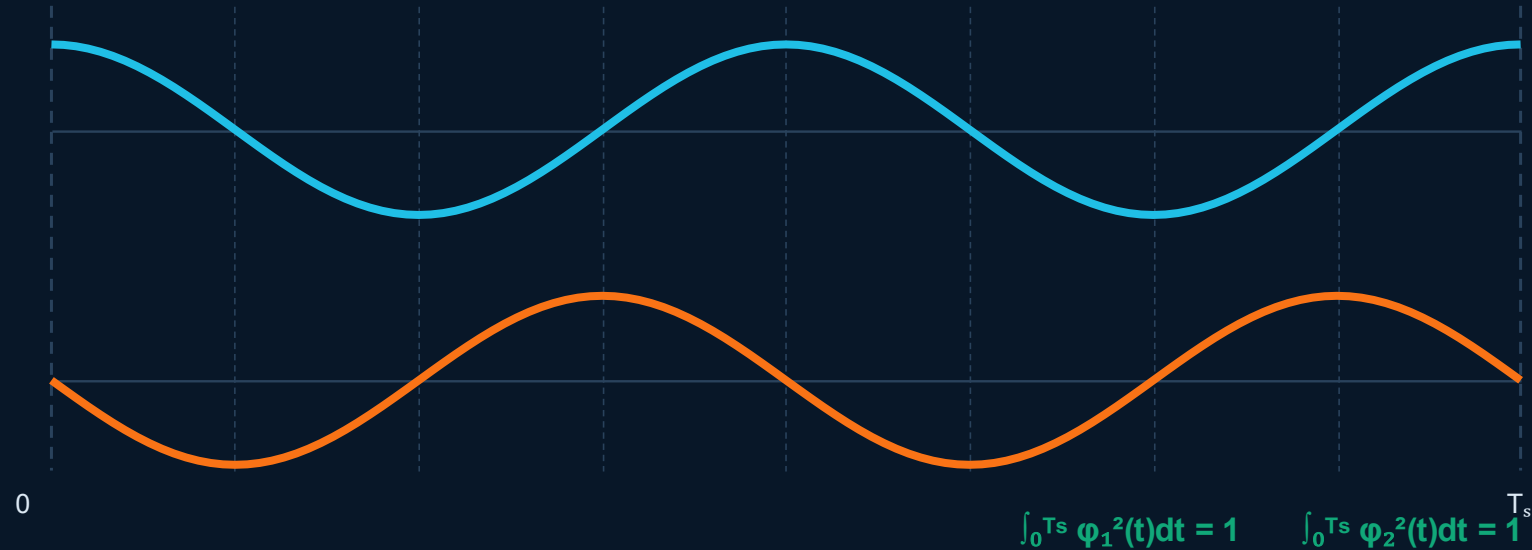
$$\phi_1(t) = \sqrt{2/T_s} \cos(2\pi f_0 t)$$

I basis

$$\phi_2(t) = -\sqrt{2/T_s} \sin(2\pi f_0 t)$$

Q basis

90° phase shift

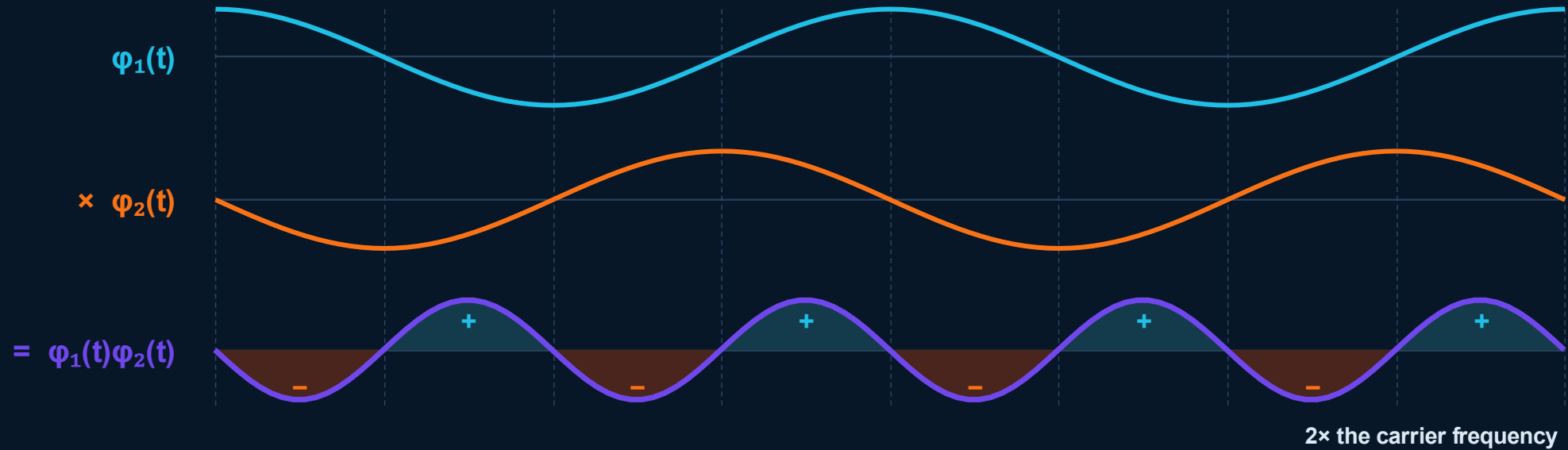


The $\sqrt{2/T_s}$ factor gives unit energy; over complete cycles, the 90° shift makes cosine and sine orthogonal.

Their cross-product balances positive and negative lobes

$$\varphi_1(t)\varphi_2(t) = -(1/T_s) \sin(4\pi f_0 t)$$

POINTWISE MULTIPLICATION REVEALS THE CROSS-TERM

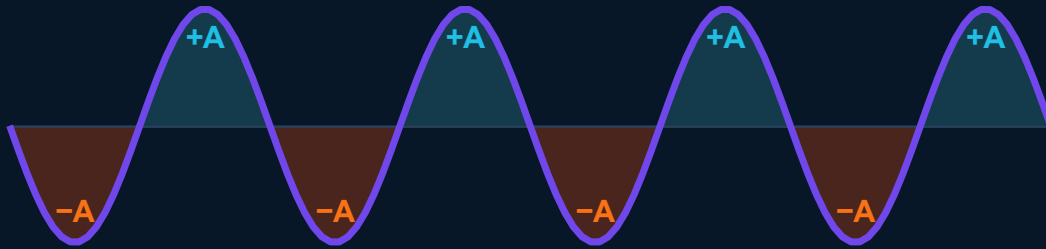


The cross-product alternates positive and negative area twice as fast as the original carrier.

Complete cycles make the cross-inner-product exactly zero

$f_0 T_s = N \in \mathbb{Z} \rightarrow$ the observation contains complete carrier cycles

GEOMETRIC VIEW · SIGNED AREA OVER T_s



$$(-A + A) + (-A + A) + (-A + A) + (-A + A) = 0$$

ALGEBRAIC VIEW

- 1 $\langle \phi_1, \phi_2 \rangle = -(1/T_s) \int_0^{T_s} \sin(4\pi f_0 t) dt$
- 2 $= [\cos(4\pi f_0 t) / (4\pi f_0 T_s)]_0^{T_s}$
- 3 $= [\cos(4\pi N) - 1] / (4\pi N)$

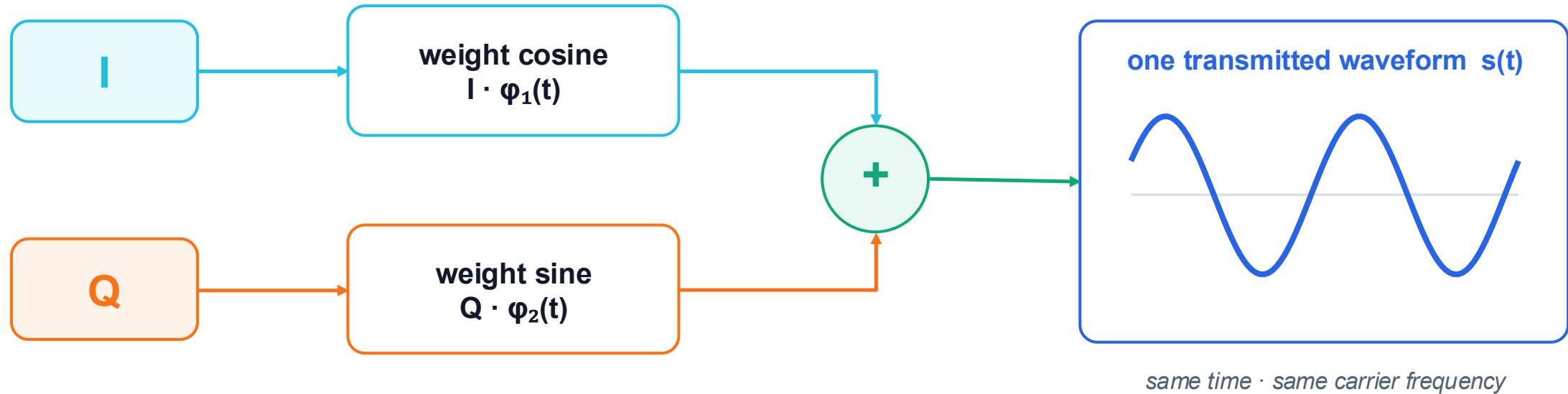
$$= 0$$

and $\langle \phi_1, \phi_1 \rangle = \langle \phi_2, \phi_2 \rangle = 1$

Complete cycles make the cross-inner-product zero: cosine and sine are orthonormal over the symbol interval.

The transmitter adds the I and Q waveforms into one signal

$$s(t) = I \cdot \varphi_1(t) + Q \cdot \varphi_2(t), \quad 0 \leq t \leq T_s$$

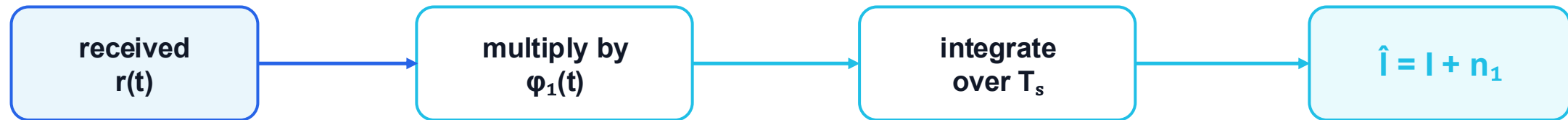


The transmitter sends two coordinates at once by weighting two orthogonal waveforms and adding them.

Projecting onto cosine recovers I because the Q term vanishes

$$r(t) = I \cdot \phi_1(t) + Q \cdot \phi_2(t) + n(t)$$

PROJECT $r(t)$ ONTO THE COSINE BASIS

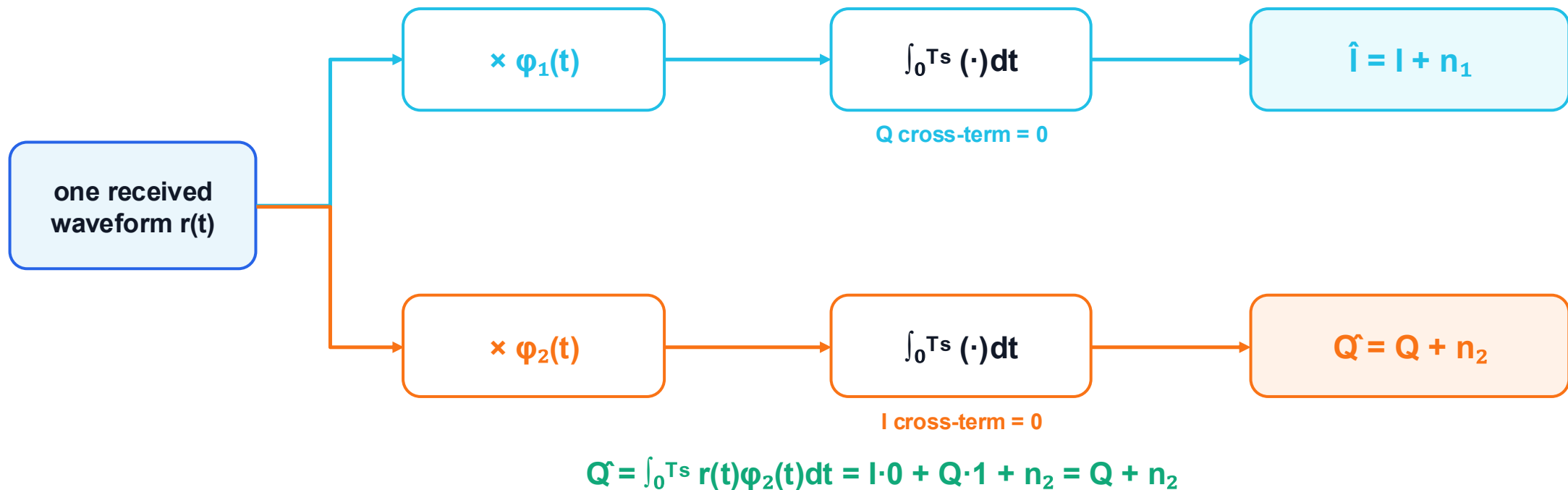


$$\begin{aligned} \hat{I} &= \int_0^{T_s} r(t) \phi_1(t) dt = I \underbrace{\int \phi_1^2 dt}_{\text{unit energy}} + Q \underbrace{\int \phi_2 \phi_1 dt}_{\text{orthogonality}} + \underbrace{\int n \phi_1 dt}_{\text{noise projection}} \\ &= I \cdot 1 + Q \cdot 0 + n_1 = I + n_1 \end{aligned}$$

The cosine projection keeps I, removes Q through the zero cross-term, and leaves only projected noise.

Projecting onto sine recovers Q—two measurements undo the sum

RUN THE TWO PROJECTIONS IN PARALLEL



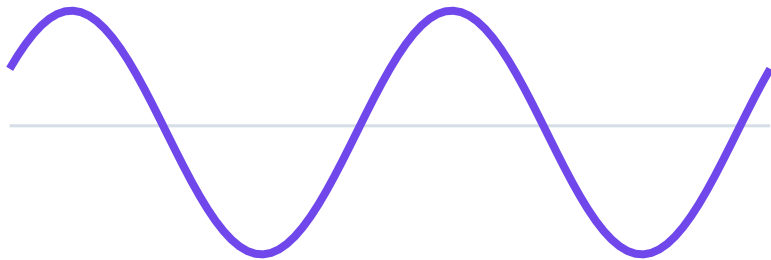
Two correlations undo the transmitted sum: one returns I and the other returns Q.

Every transmitted waveform is a point with coordinates (I, Q)

$$s_m(t) = I_m \phi_1(t) + Q_m \phi_2(t) \leftrightarrow \text{coordinate point } (I_m, Q_m)$$

WAVEFORM VIEW

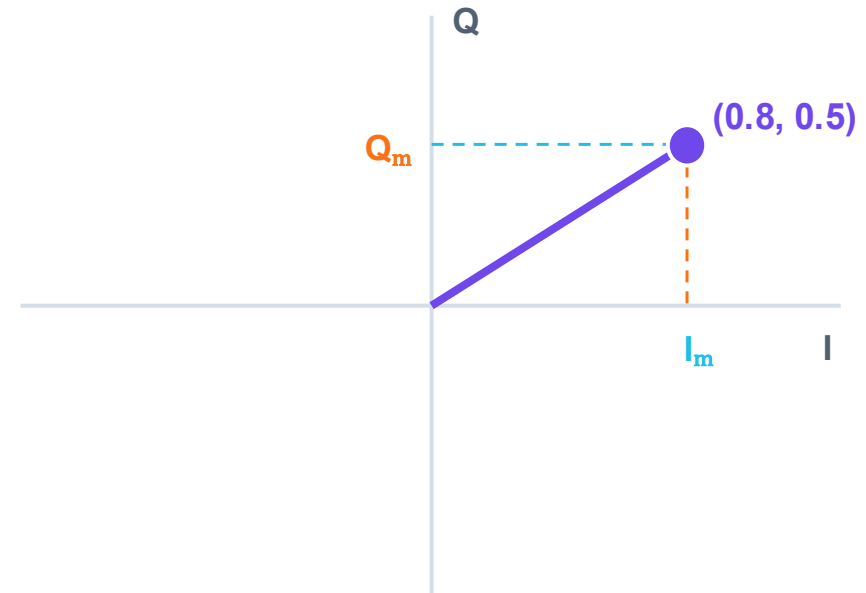
Example: $I_m = +0.8$, $Q_m = +0.5$



one physical waveform over T_s



SIGNAL-SPACE VIEW



After projection, a whole waveform can be represented by just two numbers: its I and Q coordinates.

Choose allowed I and Q values; every pair becomes a constellation point

Choose allowed coordinate levels

$$I \in \{-3, -1, +1, +3\}$$

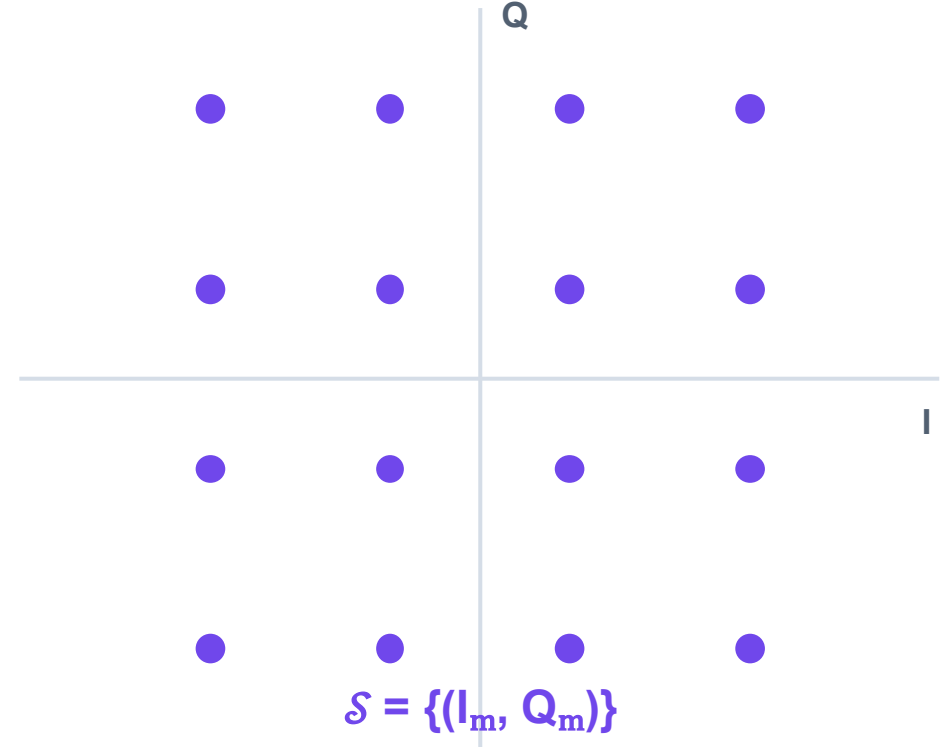
$$Q \in \{-3, -1, +1, +3\}$$

4 I levels $\times$ 4 Q levels = 16 allowed pairs

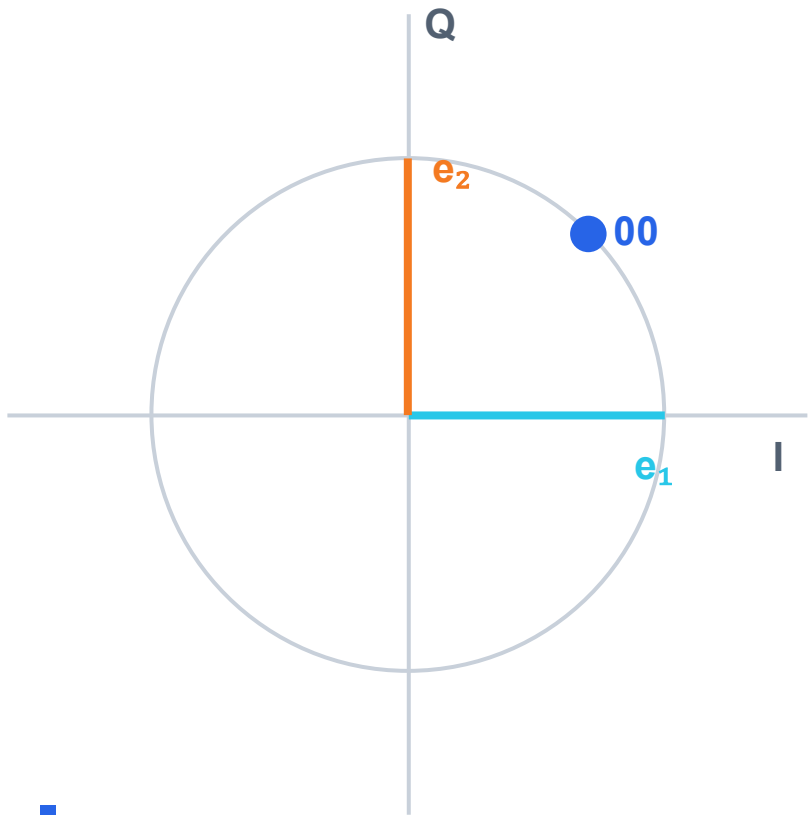
16 points $\rightarrow \log_2 16 = 4$ bits / symbol
one point $\leftrightarrow$ one 4-bit word

A constellation is a codebook of allowed (I,Q) pairs; its geometry and labeling determine robustness and bit mapping.

THE ALLOWED-POINT CATALOG IS THE CONSTELLATION



QPSK chooses four coordinate pairs on one constant-radius circle



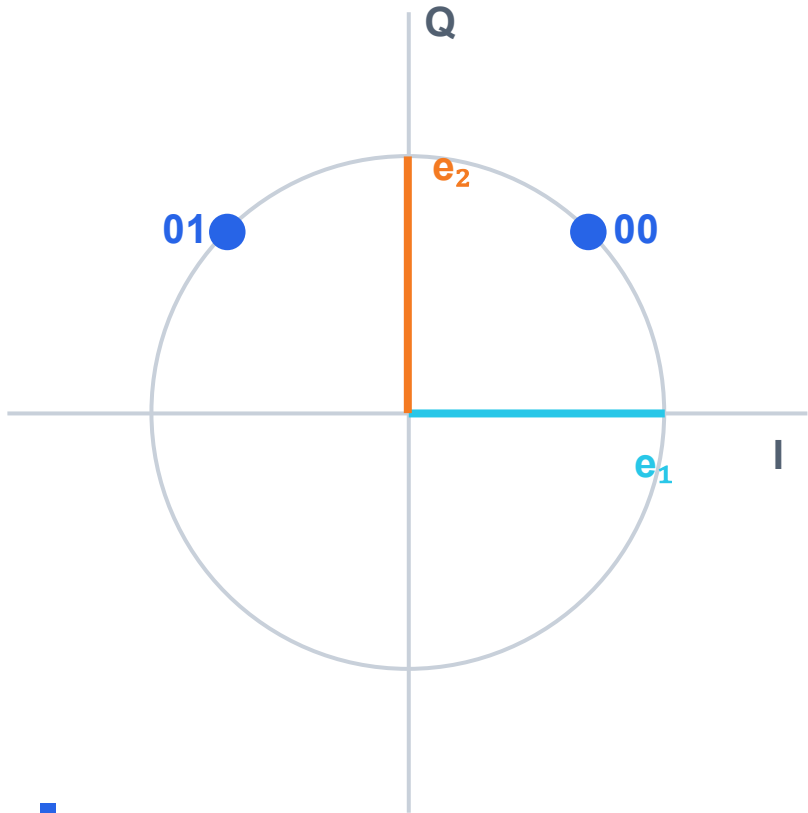
$$s_{00} = \frac{1}{\sqrt{2}} e_1 + \frac{1}{\sqrt{2}} e_2$$

normalized symbol energy: $\|s\|^2 = 1$

00 $\rightarrow$ **(+1/ $\sqrt{2}$, +1/ $\sqrt{2}$)**

Add the 00 hypothesis at coordinates (+1/ $\sqrt{2}$, +1/ $\sqrt{2}$).

QPSK chooses four coordinate pairs on one constant-radius circle



$$s_{00} = \frac{1}{\sqrt{2}} e_1 + \frac{1}{\sqrt{2}} e_2$$

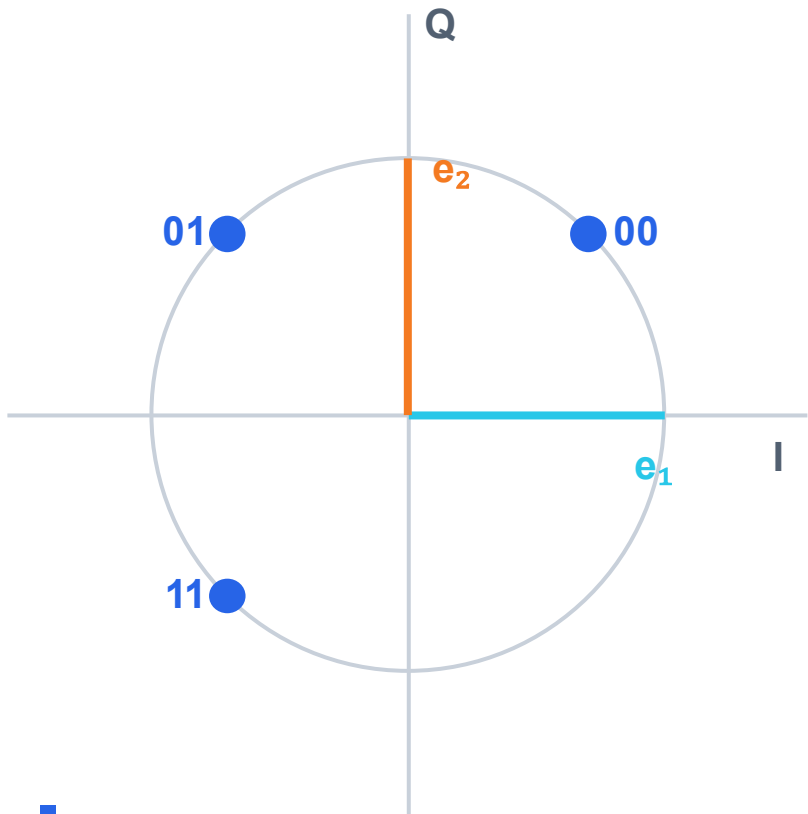
normalized symbol energy: $\|s\|^2 = 1$

00 $\rightarrow$ **$(+1/\sqrt{2}, +1/\sqrt{2})$**

01 $\rightarrow$ **$(-1/\sqrt{2}, +1/\sqrt{2})$**

Add the 01 hypothesis at coordinates $(-1/\sqrt{2}, +1/\sqrt{2})$.

QPSK chooses four coordinate pairs on one constant-radius circle



$$s_{00} = \frac{1}{\sqrt{2}} e_1 + \frac{1}{\sqrt{2}} e_2$$

normalized symbol energy: $\|s\|^2 = 1$

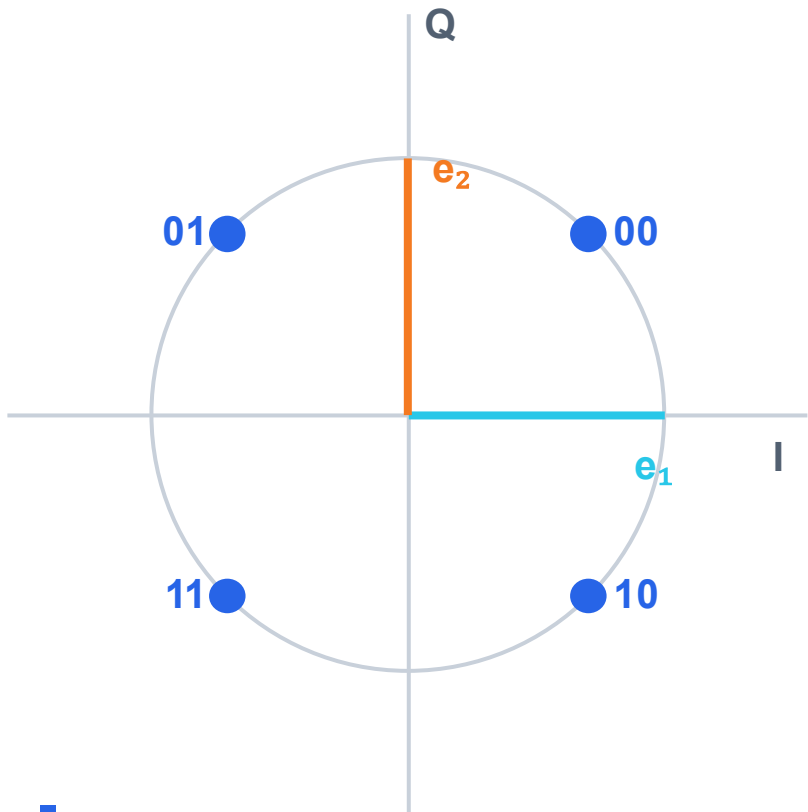
00 $\rightarrow$ **(+1/√2, +1/√2)**

01 $\rightarrow$ **(-1/√2, +1/√2)**

11 $\rightarrow$ **(-1/√2, -1/√2)**

Add the 11 hypothesis at coordinates (-1/√2, -1/√2).

QPSK chooses four coordinate pairs on one constant-radius circle



$$s_{00} = \frac{1}{\sqrt{2}} e_1 + \frac{1}{\sqrt{2}} e_2$$

normalized symbol energy: $\|s\|^2 = 1$

00 $\rightarrow$ **(+1/√2, +1/√2)**

01 $\rightarrow$ **(-1/√2, +1/√2)**

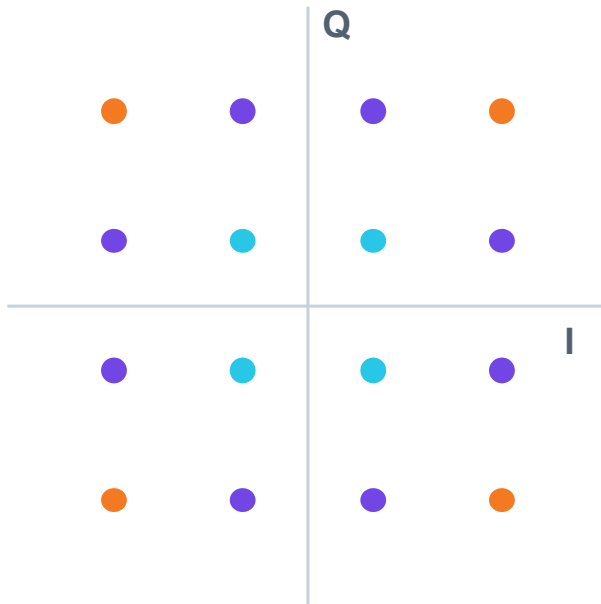
11 $\rightarrow$ **(-1/√2, -1/√2)**

10 $\rightarrow$ **(+1/√2, -1/√2)**

QPSK lies on one circle because every symbol has the same energy; QAM keeps the axes but adds radii.

16-QAM has three energy shells; normalization sets the average energy to 1

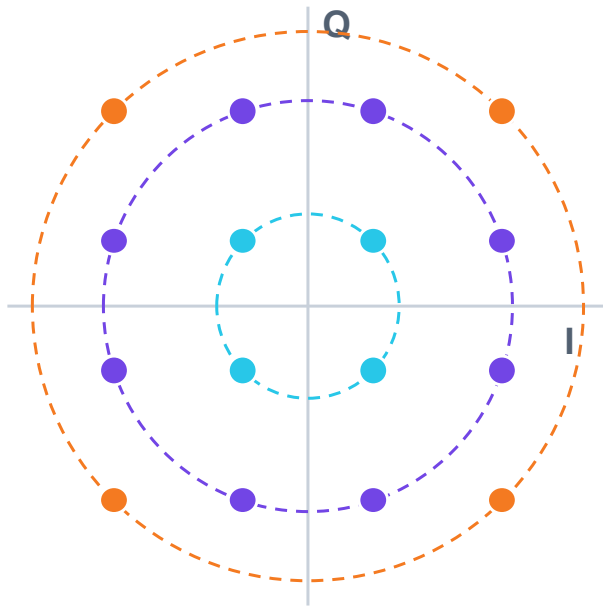
raw coordinates $I, Q \in \{\pm 1, \pm 3\}$



Sixteen coordinate pairs are sixteen symbol states, so one symbol can carry four bits.

16-QAM has three energy shells; normalization sets the average energy to 1

raw coordinates $I, Q \in \{\pm 1, \pm 3\}$



● 4 inner points $E = 2$

● 8 edge points $E = 10$

● 4 corner points $E = 18$

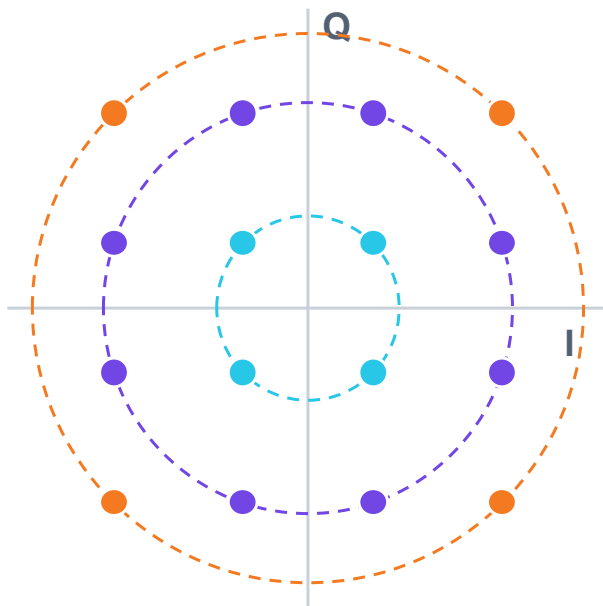
$$E = ||s||^2 = I^2 + Q^2$$

one circle = one instantaneous energy

The circles are not decision boundaries: they group symbols with the same instantaneous energy $I^2 + Q^2$.

16-QAM has three energy shells; normalization sets the average energy to 1

raw coordinates $I, Q \in \{\pm 1, \pm 3\}$



● 4 inner points $E = 2$

● 8 edge points $E = 10$

● 4 corner points $E = 18$

$$E = ||s||^2 = I^2 + Q^2$$

one circle = one instantaneous energy

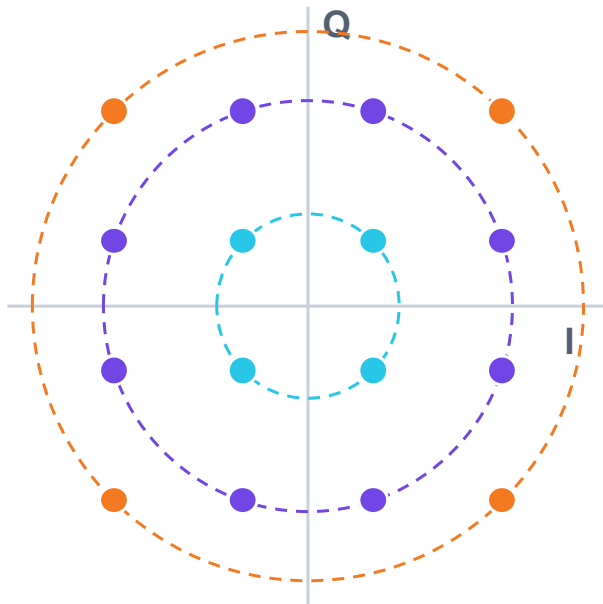
average over all 16 equally likely symbols

$$E_s = (4 \cdot 2 + 8 \cdot 10 + 4 \cdot 18) / 16 = 10$$

The raw 16-QAM grid has average symbol energy 10—not 1.

16-QAM has three energy shells; normalization sets the average energy to 1

raw coordinates $I, Q \in \{\pm 1, \pm 3\}$



- 4 inner points $E = 2$
- 8 edge points $E = 10$
- 4 corner points $E = 18$

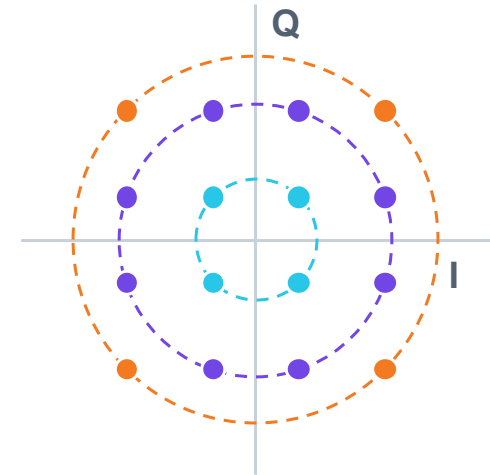
$$E = ||s||^2 = I^2 + Q^2$$

one circle = one instantaneous energy

average over all 16 equally likely symbols

$$E_s = (4 \cdot 2 + 8 \cdot 10 + 4 \cdot 18) / 16 = 10$$

normalized constellation



$$E[||s_{\text{norm}}||^2] = 1$$

$$s_{\text{norm}} = s / \sqrt{10}$$

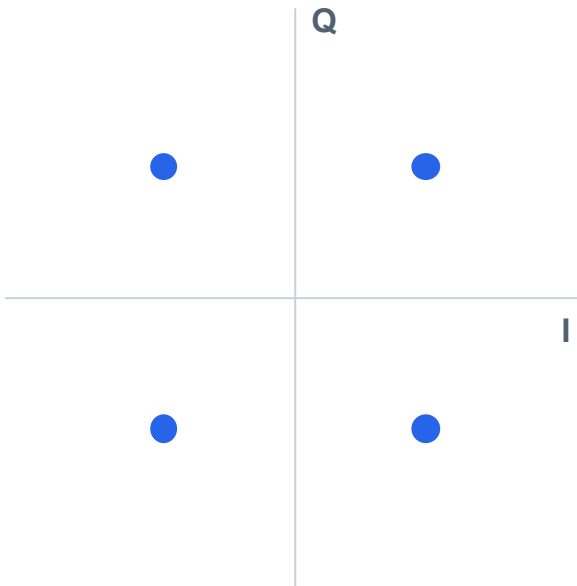
Normalize the whole constellation by $\sqrt{10}$. The average energy becomes 1, while individual symbols still lie on three different radii.

Normalize average symbol energy before comparing constellation distance

$$\bar{E}_s = E[||s||^2] = 1$$

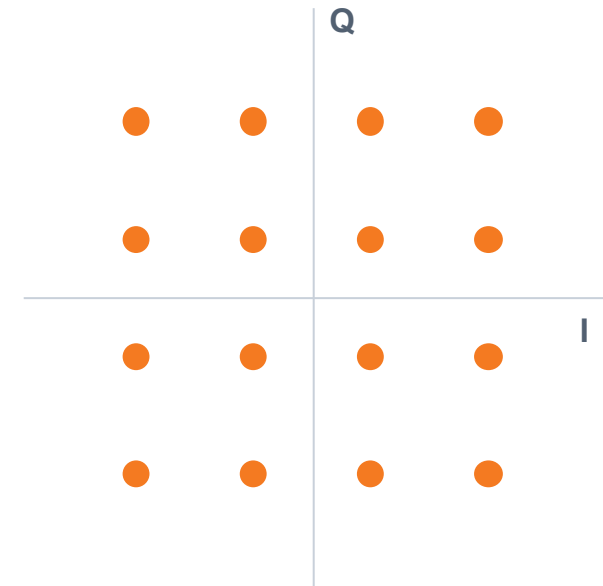
QPSK · 2 bits/symbol

$I, Q = \pm 1/\sqrt{2}$



16-QAM · 4 bits/symbol

$I, Q = \pm 1/\sqrt{10}, \pm 3/\sqrt{10}$



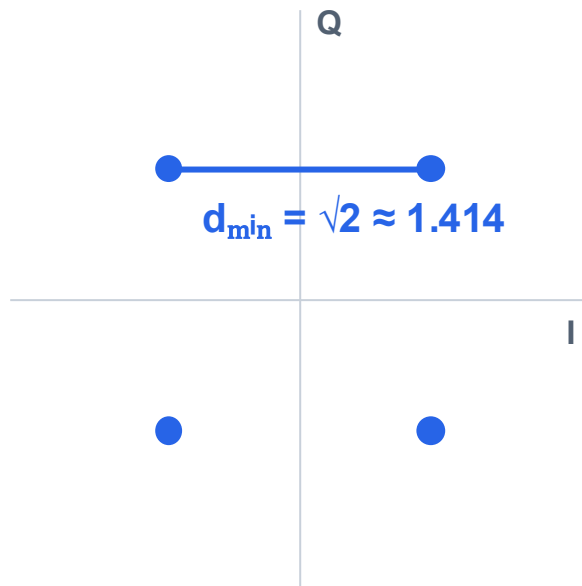
The unit circle is a constant-energy property of QPSK—not a requirement for every modulation.

Normalize average symbol energy before comparing constellation distance

$$\bar{E}_s = E[|s|^2] = 1$$

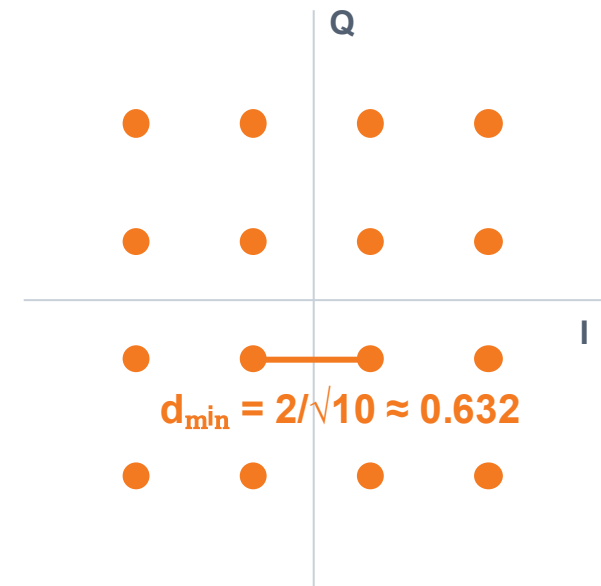
QPSK · 2 bits/symbol

$$I, Q = \pm 1/\sqrt{2}$$



16-QAM · 4 bits/symbol

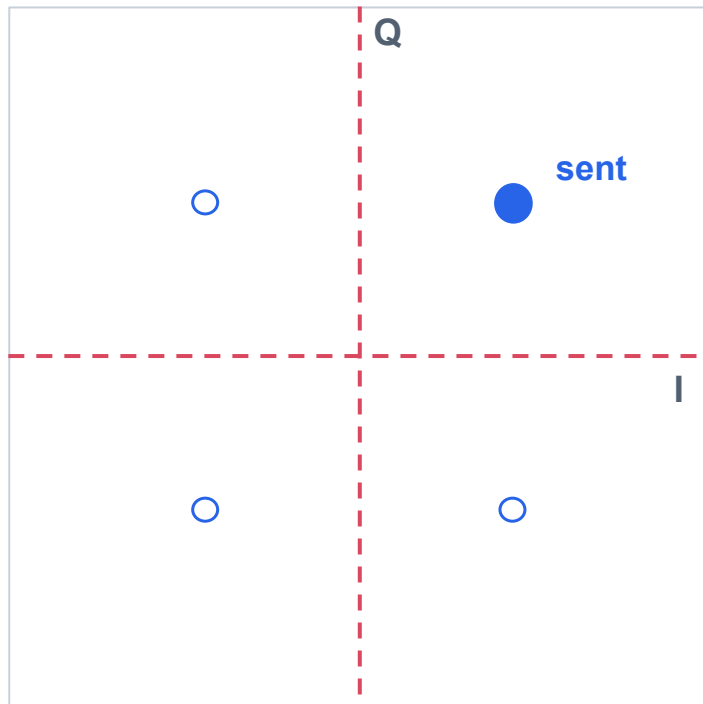
$$I, Q = \pm 1/\sqrt{10}, \pm 3/\sqrt{10}$$



After the same-energy normalization, compare nearest-neighbor Euclidean distance: $\sqrt{2}$ versus $2/\sqrt{10}$.

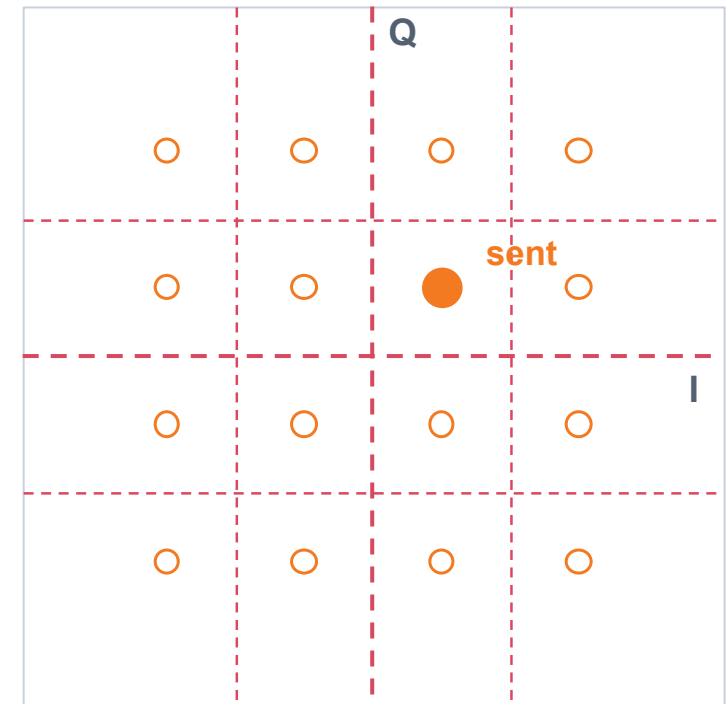
Increase noise around one transmitted symbol

QPSK



$\sigma = 0$ · same fixed normalized perturbations

16-QAM



dashed red lines = nearest-neighbor decision boundaries

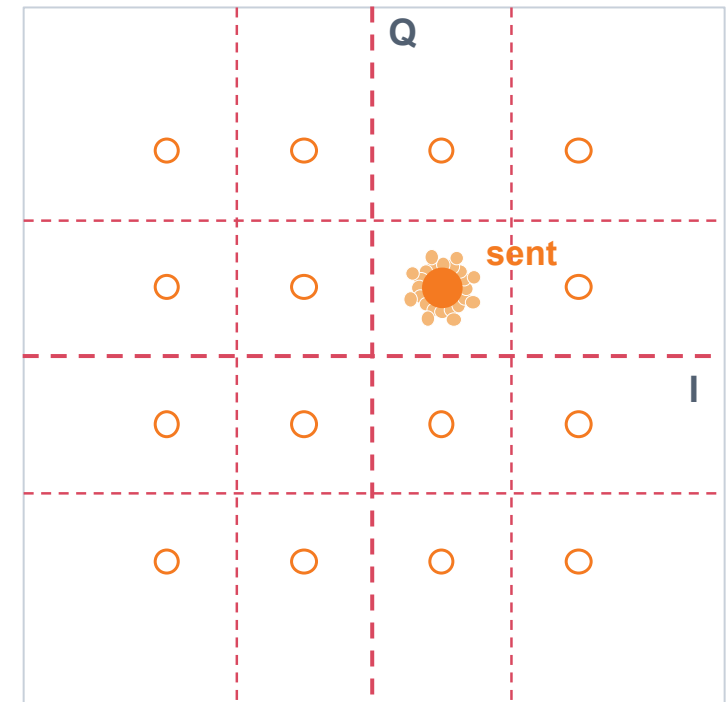
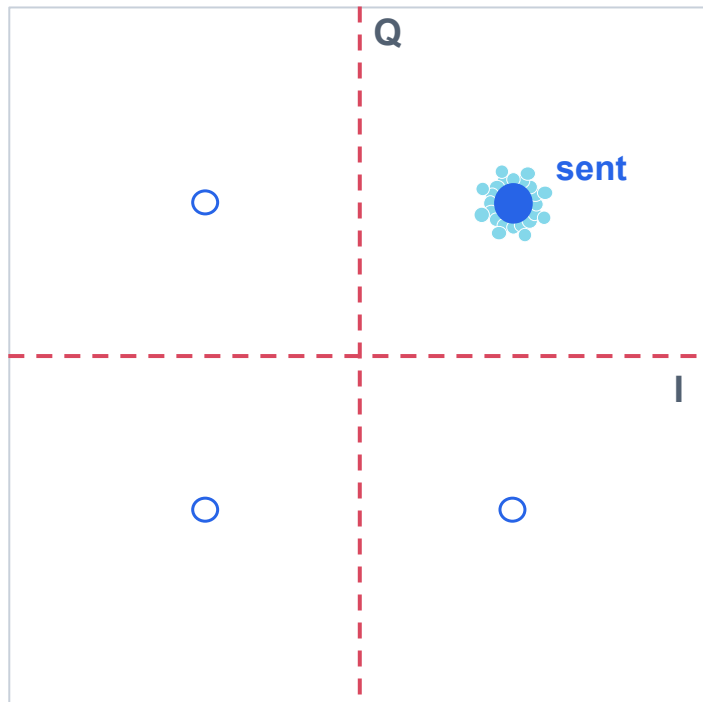
Choose one transmitted symbol in each normalized constellation.

Increase noise around one transmitted symbol

QPSK

small noise · $\sigma = 0.07$ · same fixed normalized perturbations

16-QAM



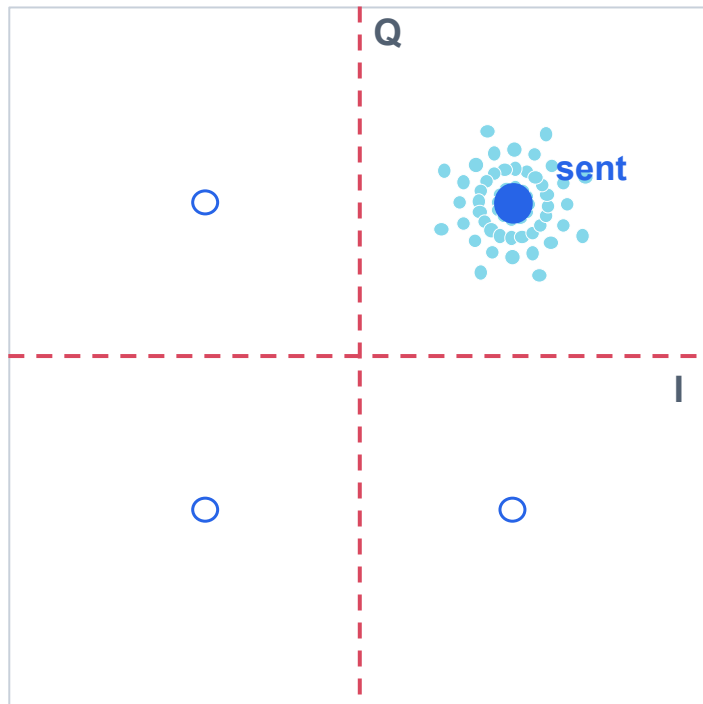
red observations = this transmitted symbol would be decoded as a neighbor

Small noise: both observation clouds remain inside the intended decision region.

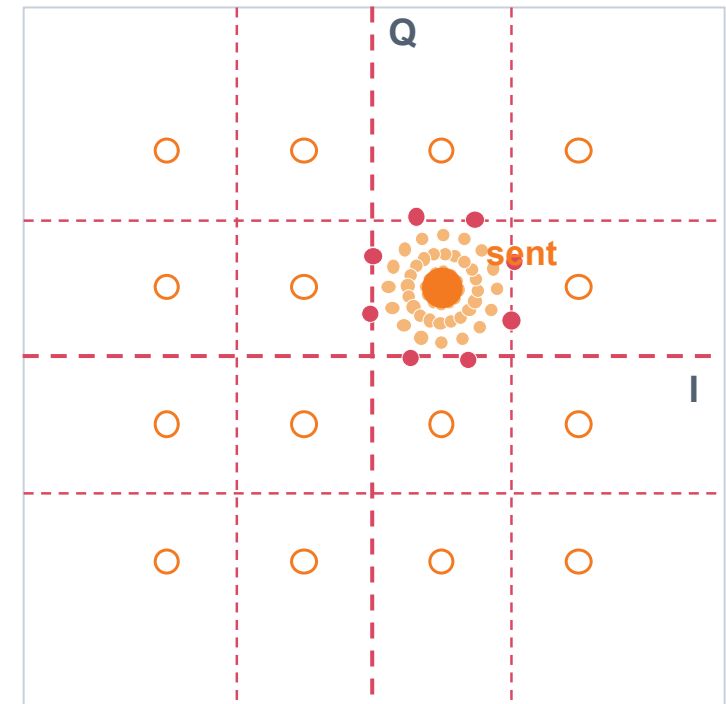
Increase noise around one transmitted symbol

QPSK

medium noise · $\sigma = 0.16$ · same fixed normalized perturbations



16-QAM



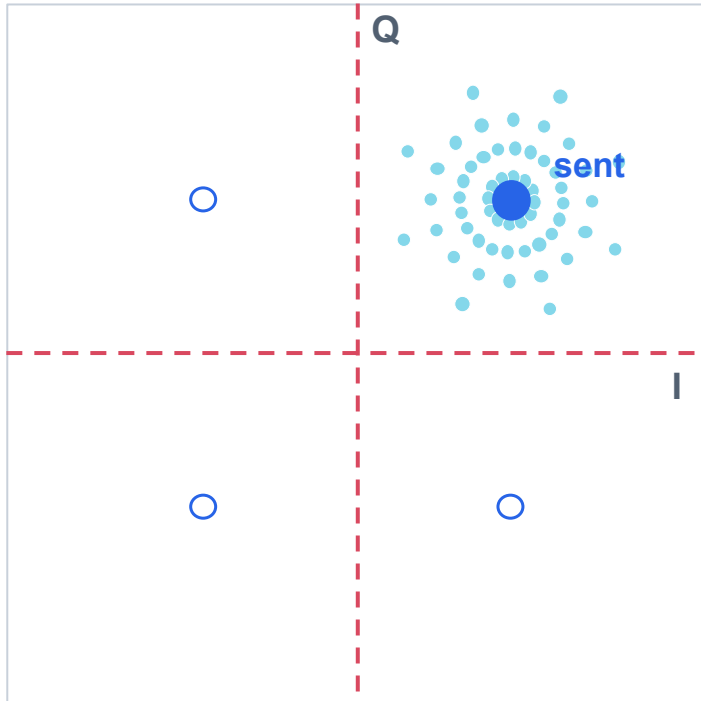
red observations = this transmitted symbol would be decoded as a neighbor

The QPSK cloud is still separated; 16-QAM observations begin crossing into neighbors.

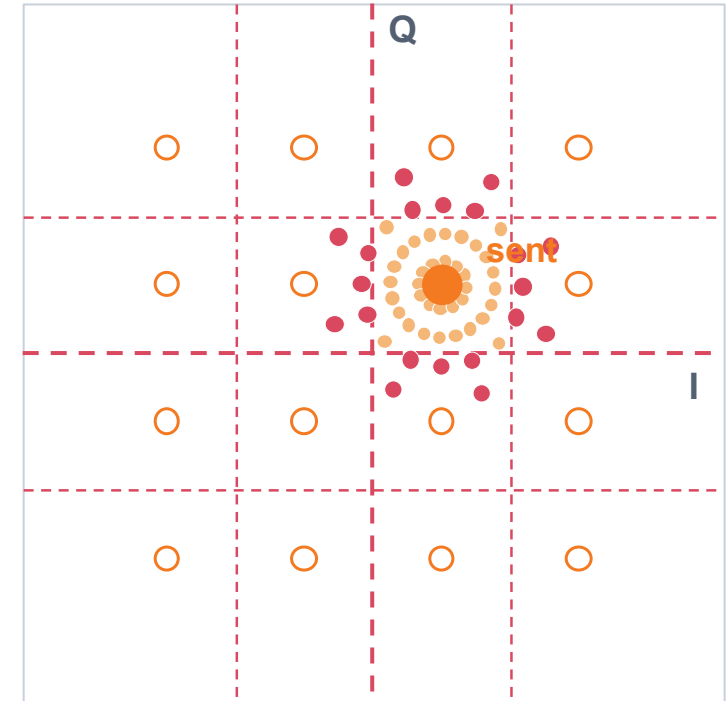
Increase noise around one transmitted symbol

QPSK

larger noise · $\sigma = 0.24$ · same fixed normalized perturbations



16-QAM



red observations = this transmitted symbol would be decoded as a neighbor

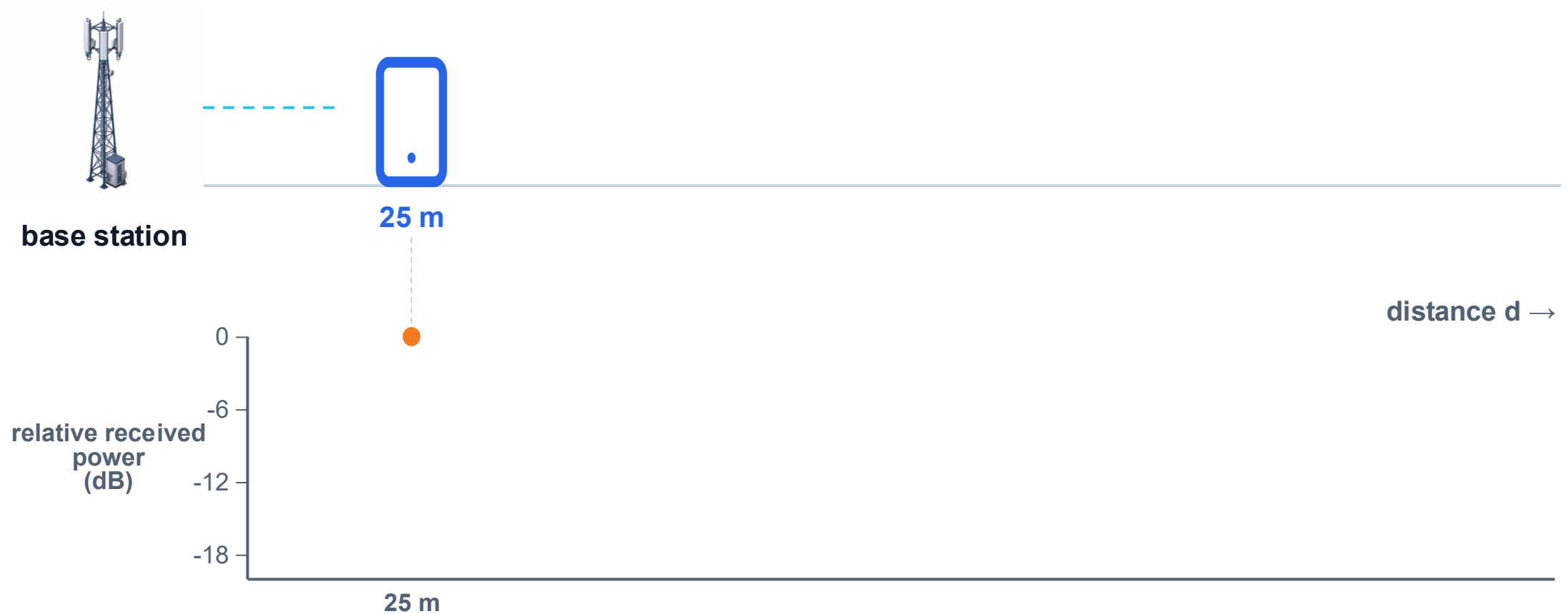
QPSK remains distinguishable, while 16-QAM observations occupy several symbol regions.

02

Understand what the air does

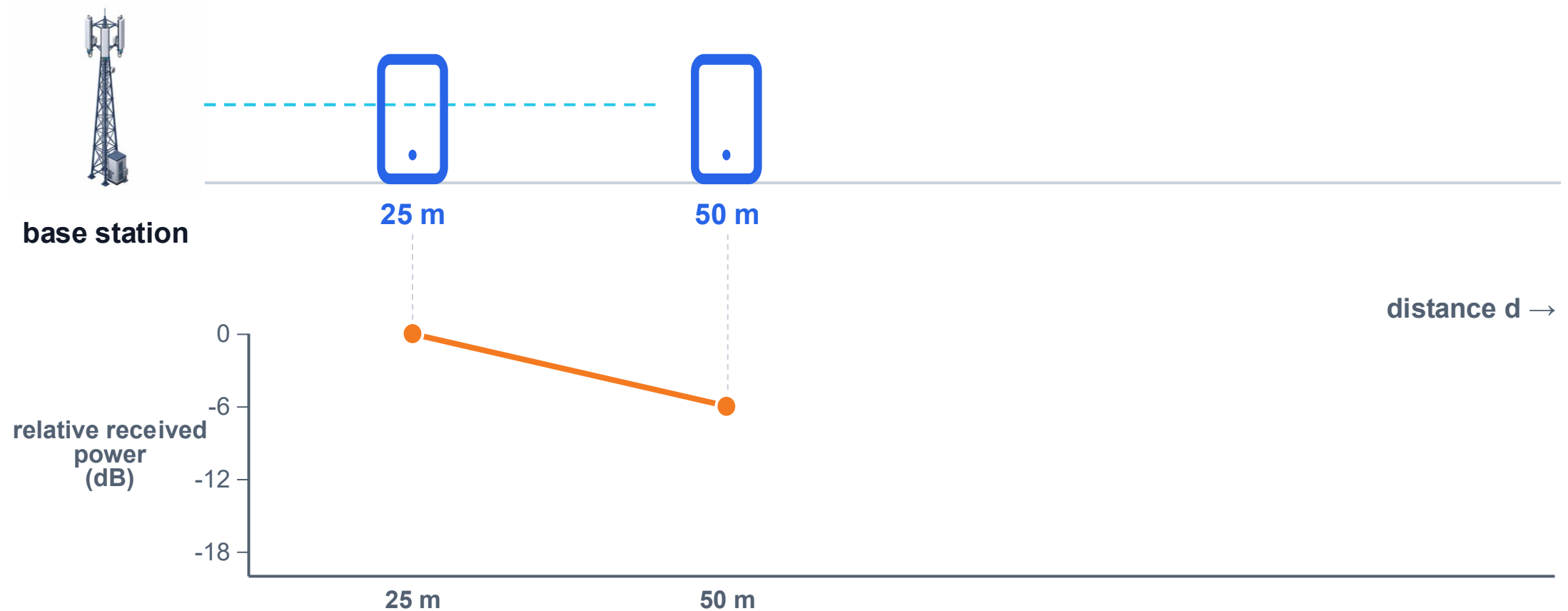
First distance attenuates power. Then the real environment creates delayed copies that add at the antenna.

As the phone moves away, received free-space power falls



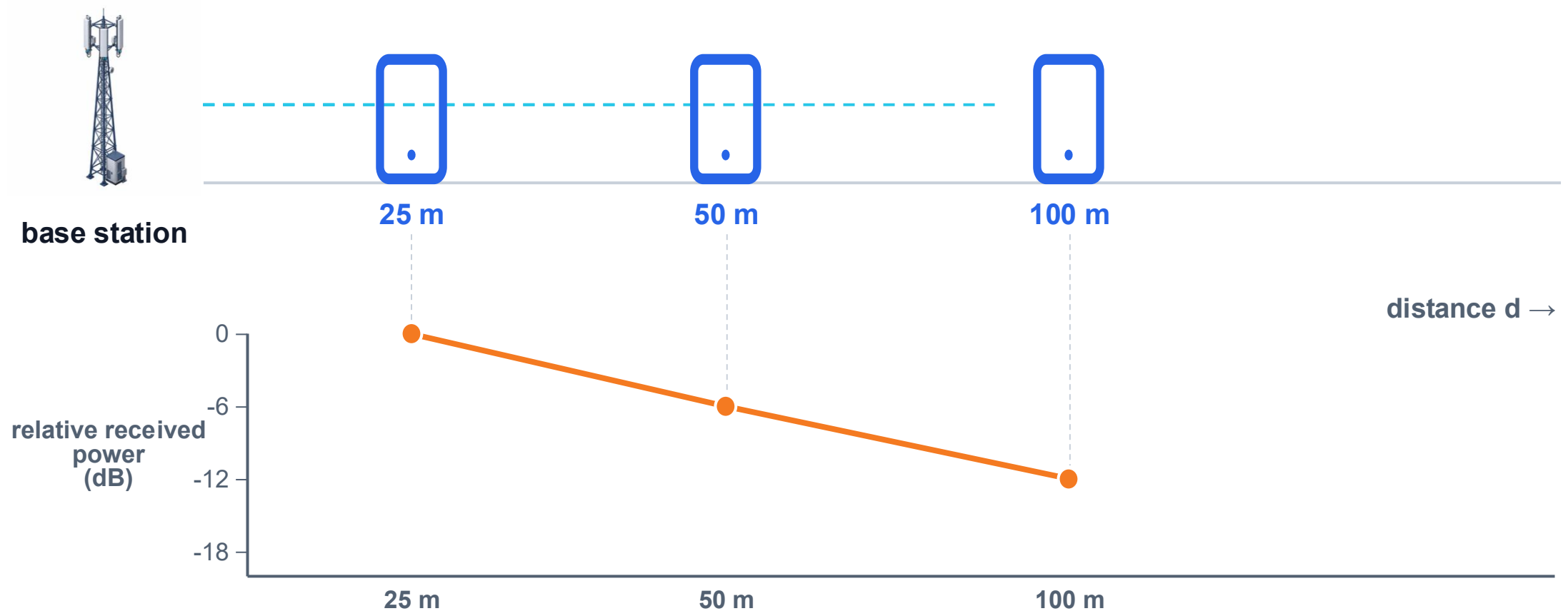
Use the nearest phone as the 0 dB reference.

As the phone moves away, received free-space power falls



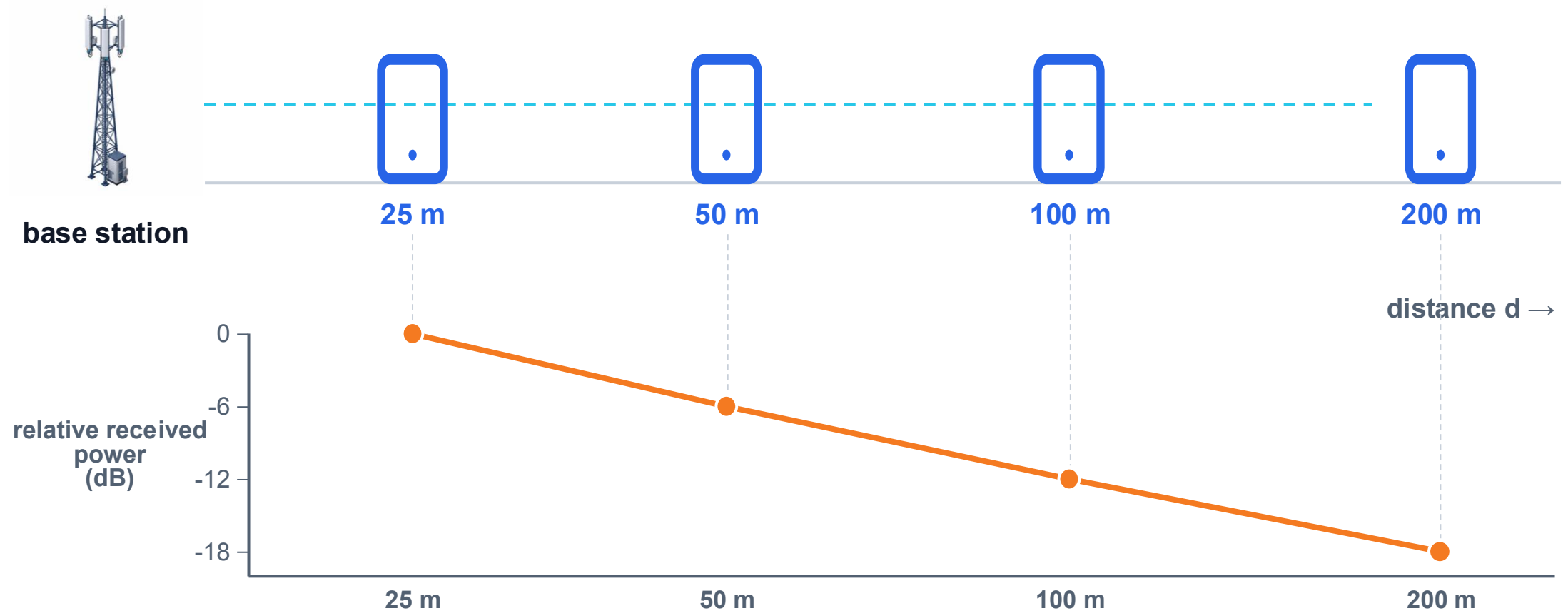
Doubling distance 25 → 50 m reduces ideal received power by about 6 dB.

As the phone moves away, received free-space power falls



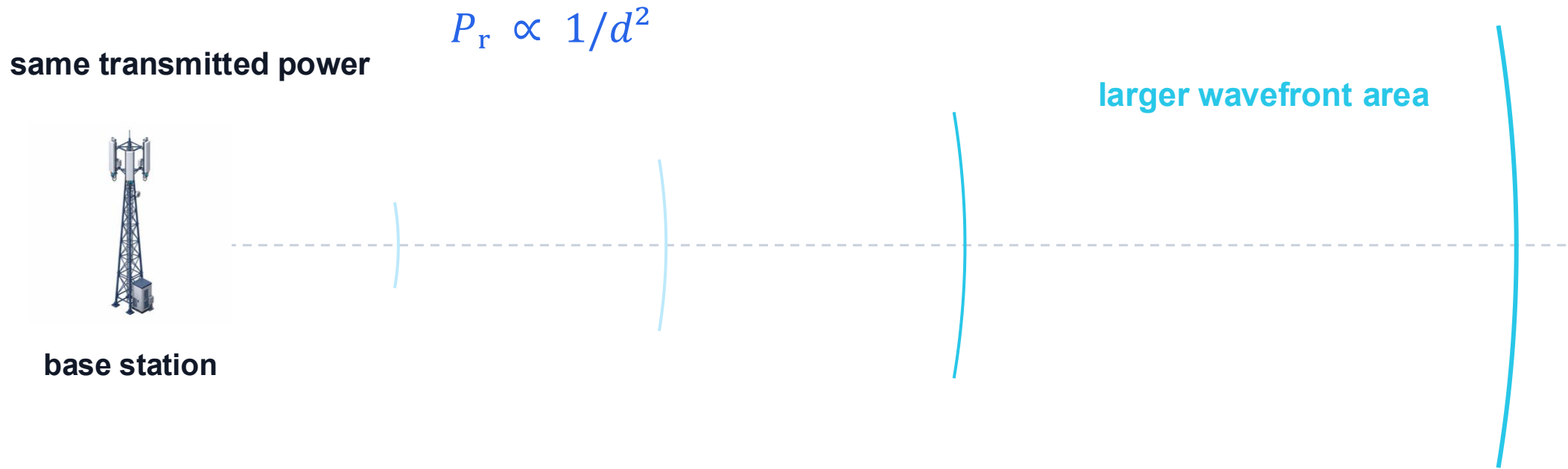
Doubling distance 50 → 100 m reduces ideal received power by about 6 dB.

As the phone moves away, received free-space power falls



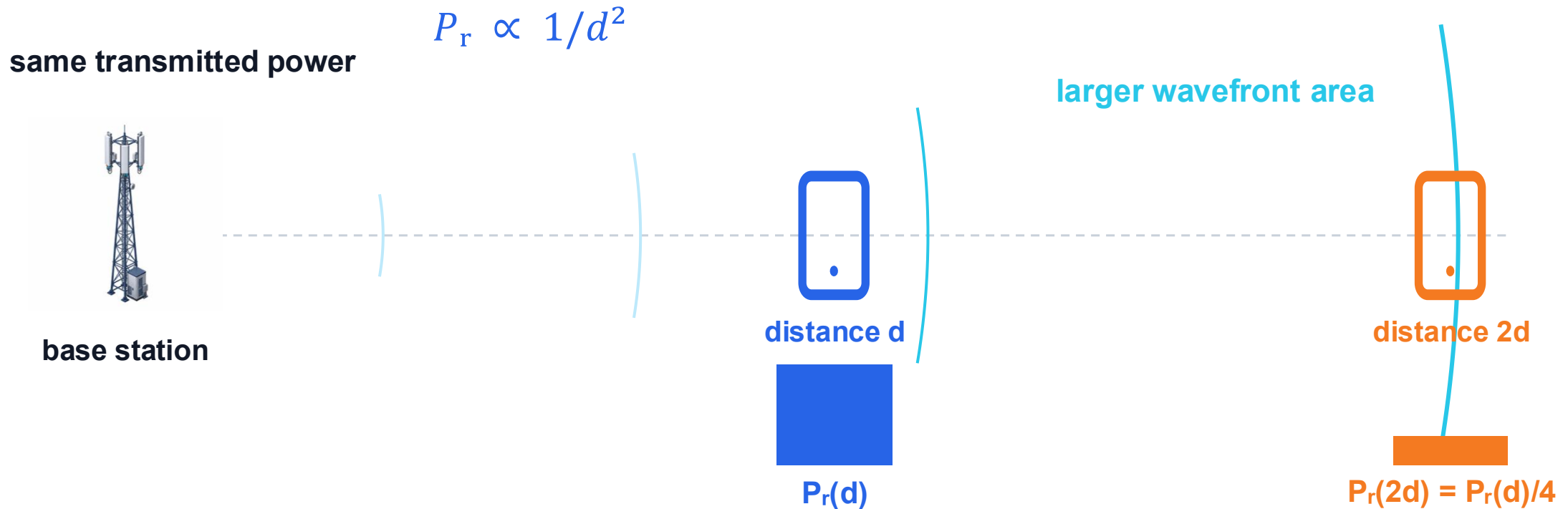
Doubling distance 100 → 200 m reduces ideal received power by about 6 dB.

Free-space loss comes from spreading the same power over a larger wavefront



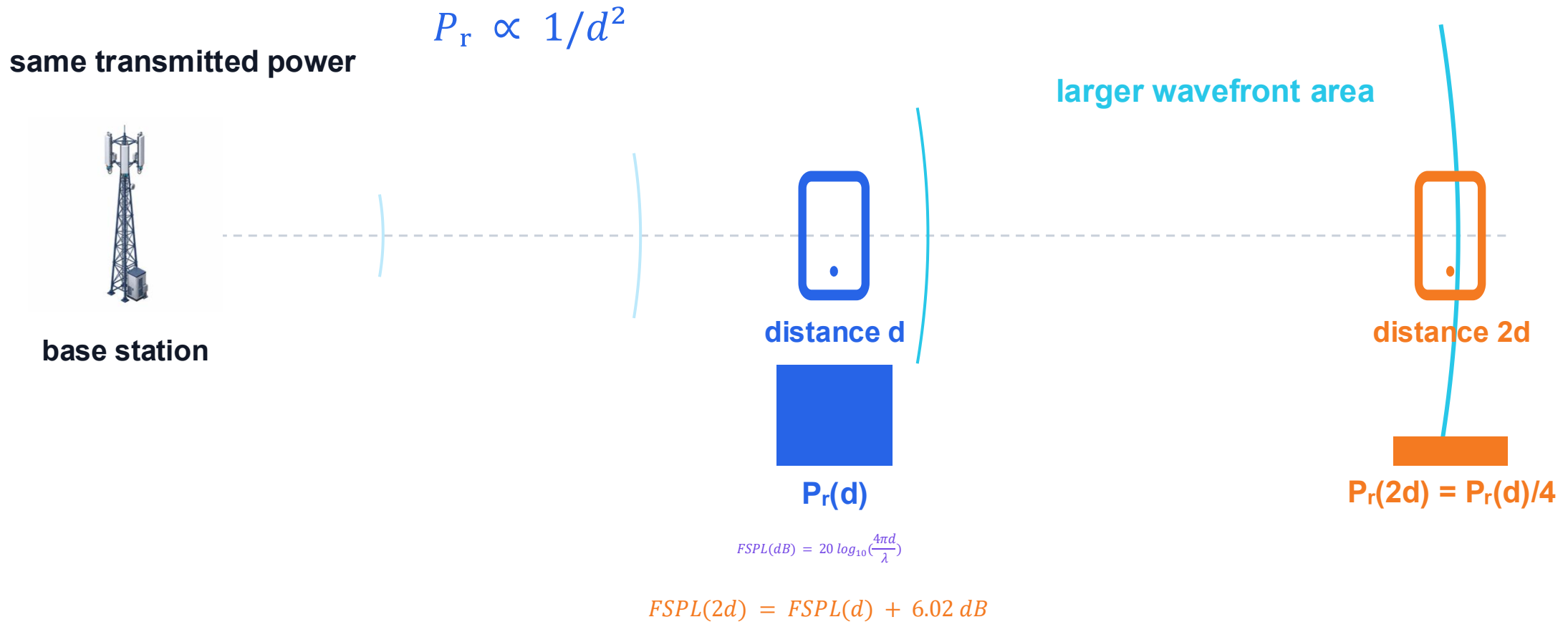
As the wavefront expands, the same transmit power is distributed over a larger area.

Free-space loss comes from spreading the same power over a larger wavefront



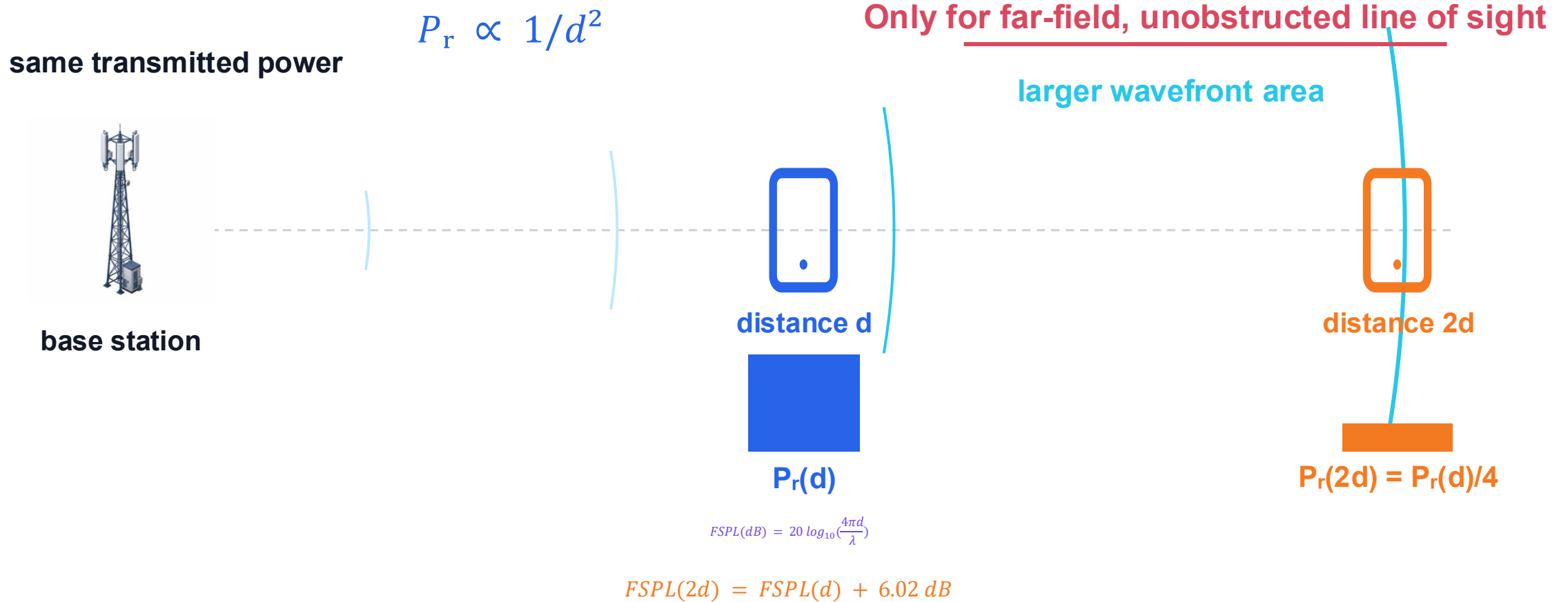
Doubling distance makes the ideal received power one quarter as large.

Free-space loss comes from spreading the same power over a larger wavefront



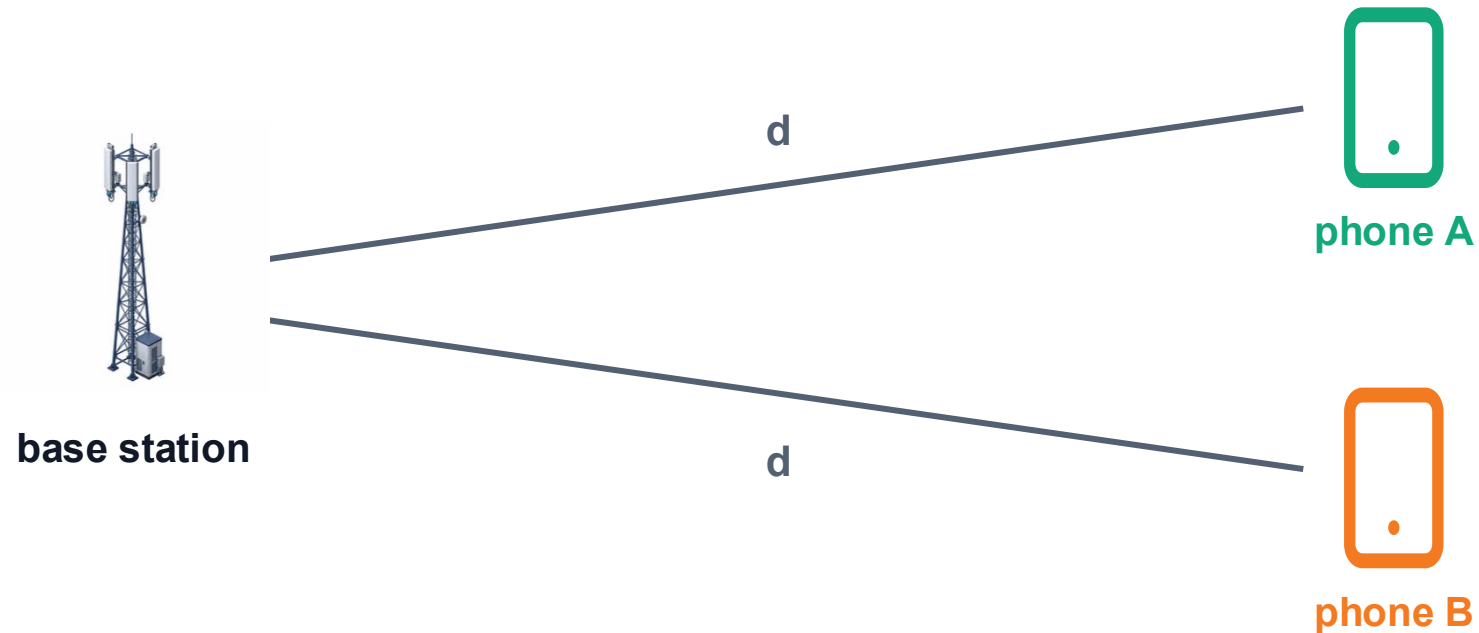
FSPL means Free-Space Path Loss. Doubling distance adds 6.02 dB of loss.

Free-space loss comes from spreading the same power over a larger wavefront



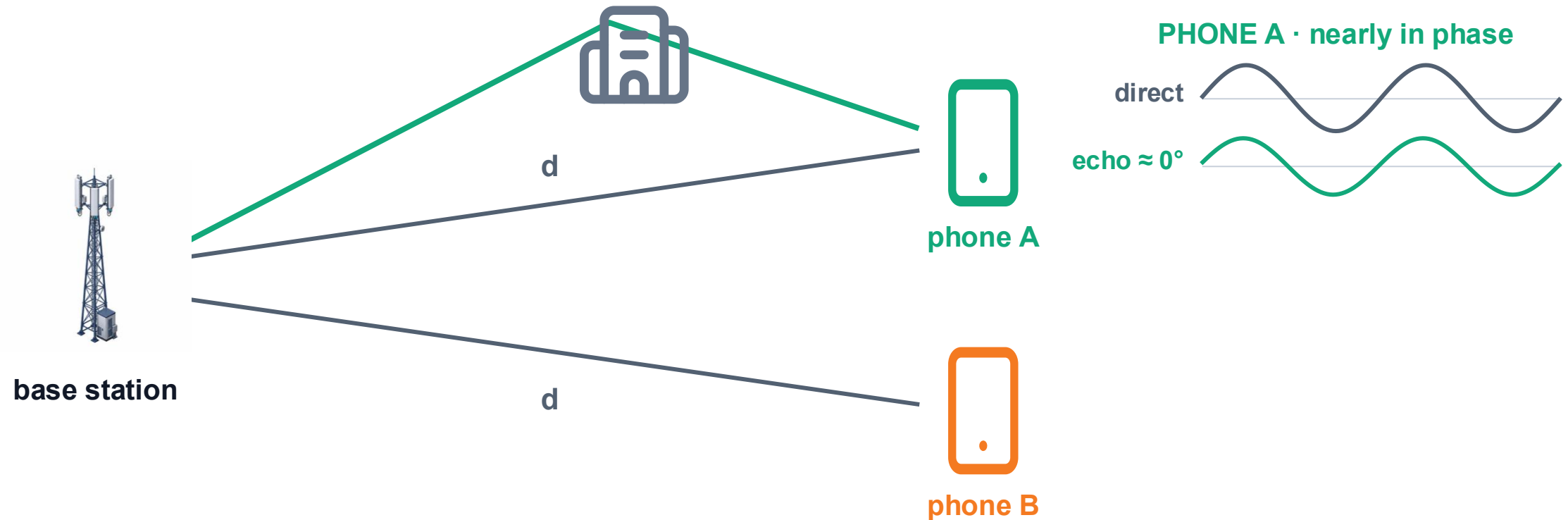
FSPL is a clean baseline. Buildings, reflections, diffraction, and motion create the real channel.

Same distance, different path phases: one signal is strong and the other is weak



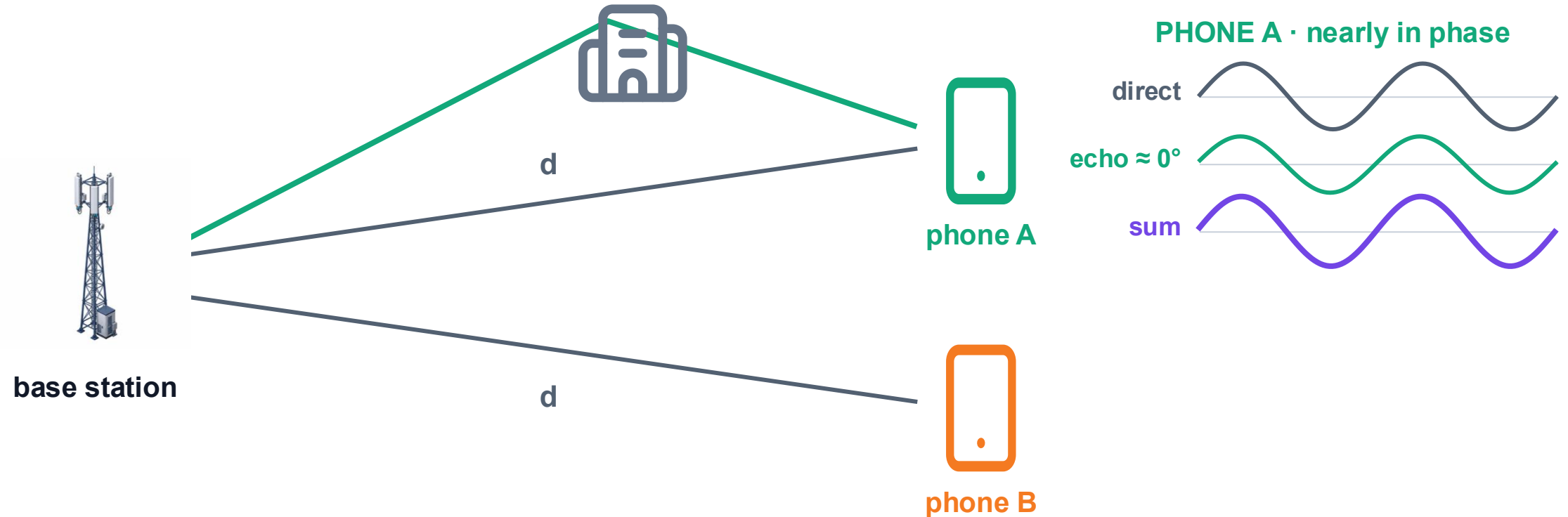
Free-space loss predicts the same power because the two direct paths have the same length d .

Same distance, different path phases: one signal is strong and the other is weak



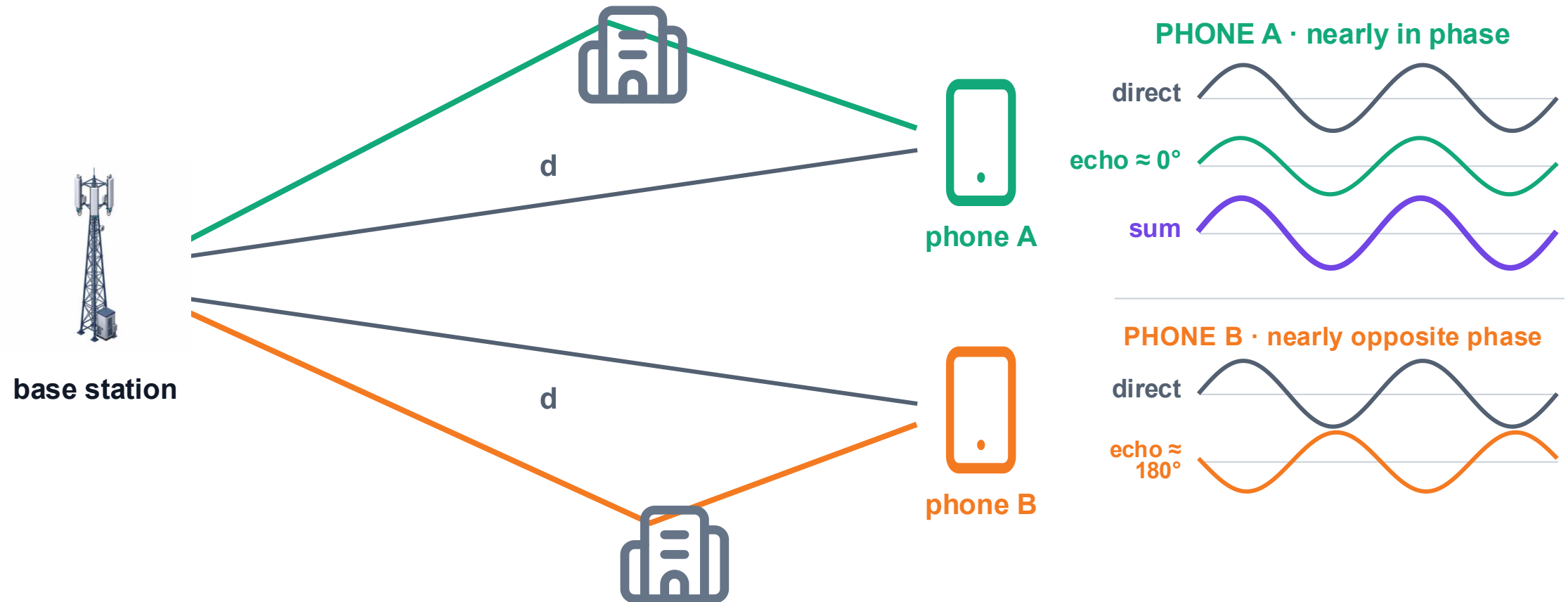
Phone A: the direct and reflected sine waves arrive nearly in phase.

Same distance, different path phases: one signal is strong and the other is weak



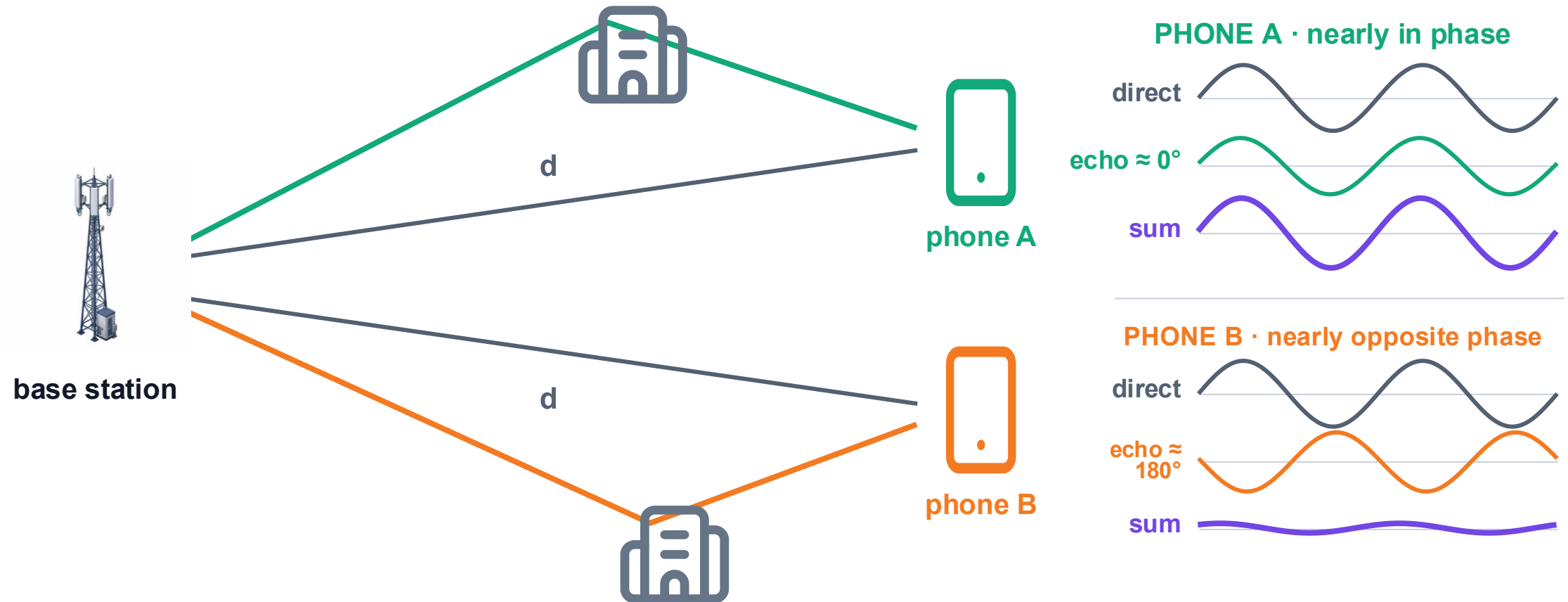
Their peaks and troughs reinforce sample by sample, so the summed waveform has a larger amplitude.

Same distance, different path phases: one signal is strong and the other is weak



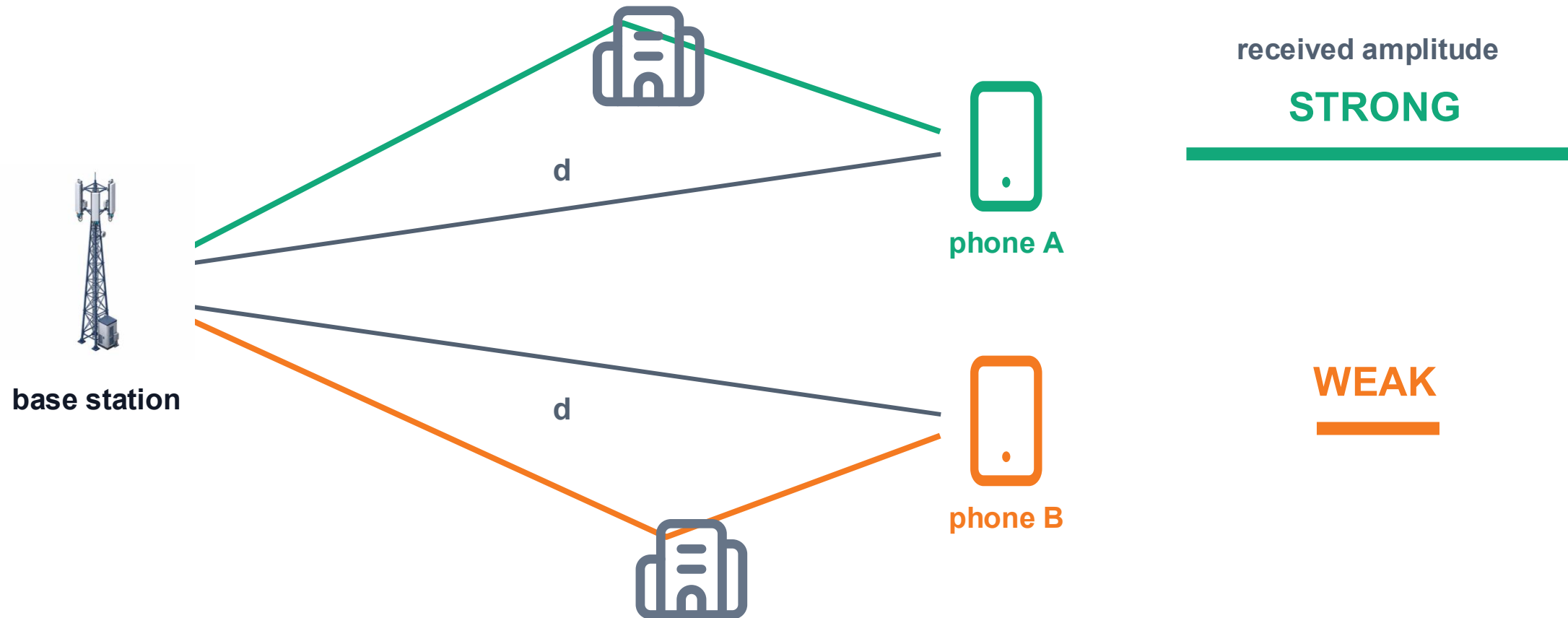
Phone B: the reflected sine wave arrives nearly 180° out of phase with the direct wave.

Same distance, different path phases: one signal is strong and the other is weak



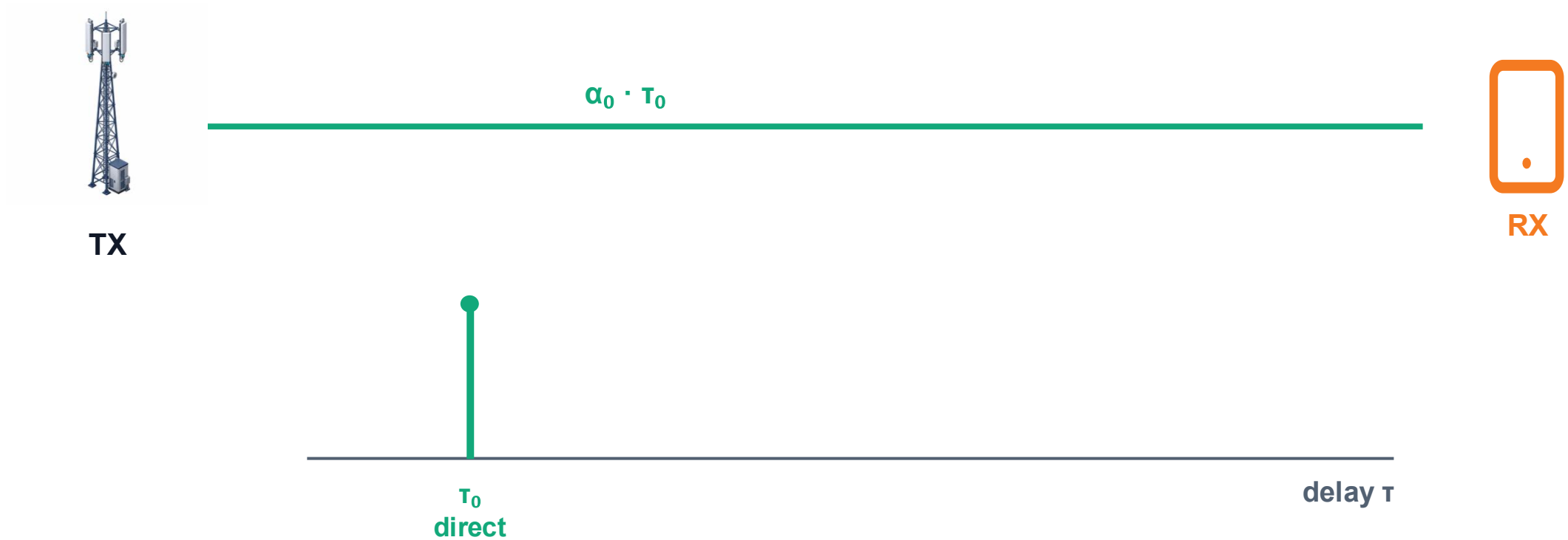
Peaks meet troughs, so most of the two waveforms cancel and only a weak sum remains.

Same distance, different path phases: one signal is strong and the other is weak



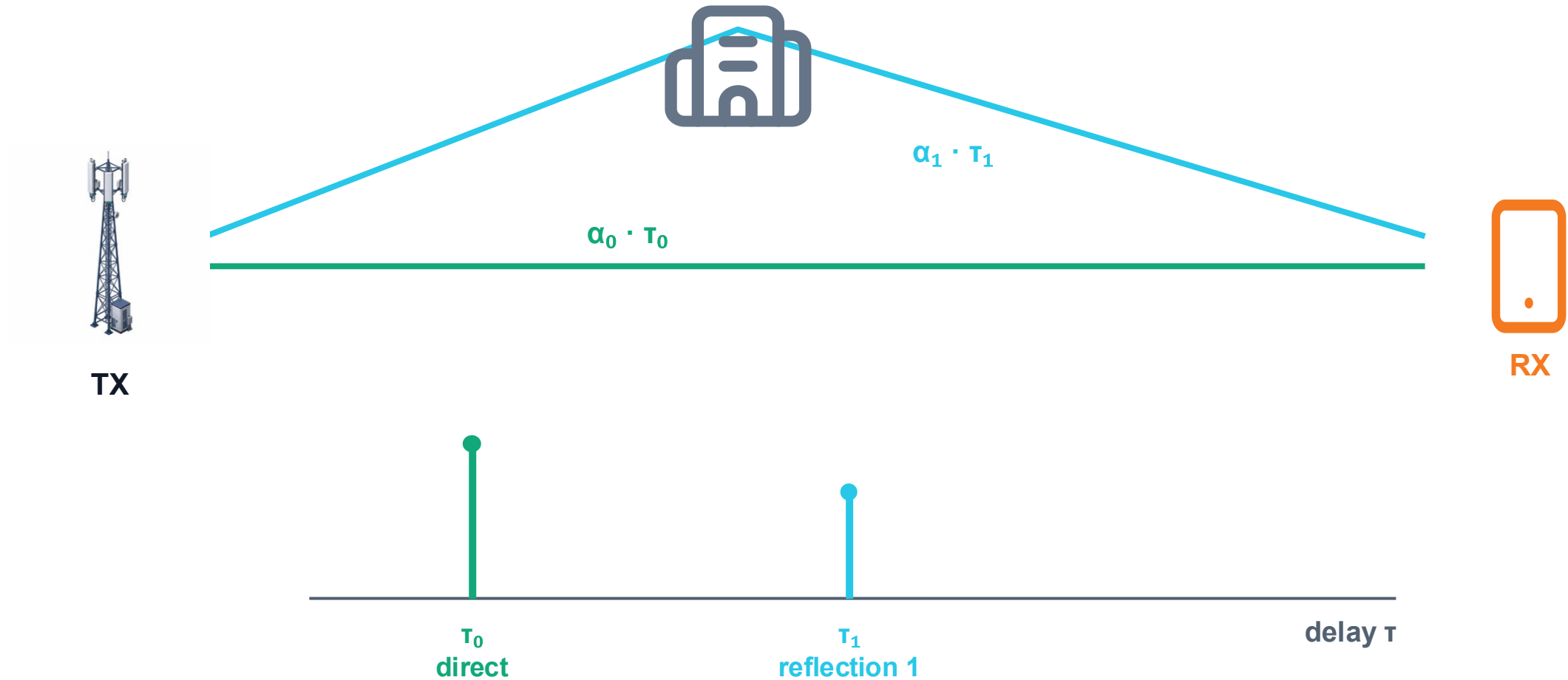
Same distance d does not imply the same received strength: phone A is strong; phone B is weak.

Different path lengths create delayed, scaled copies



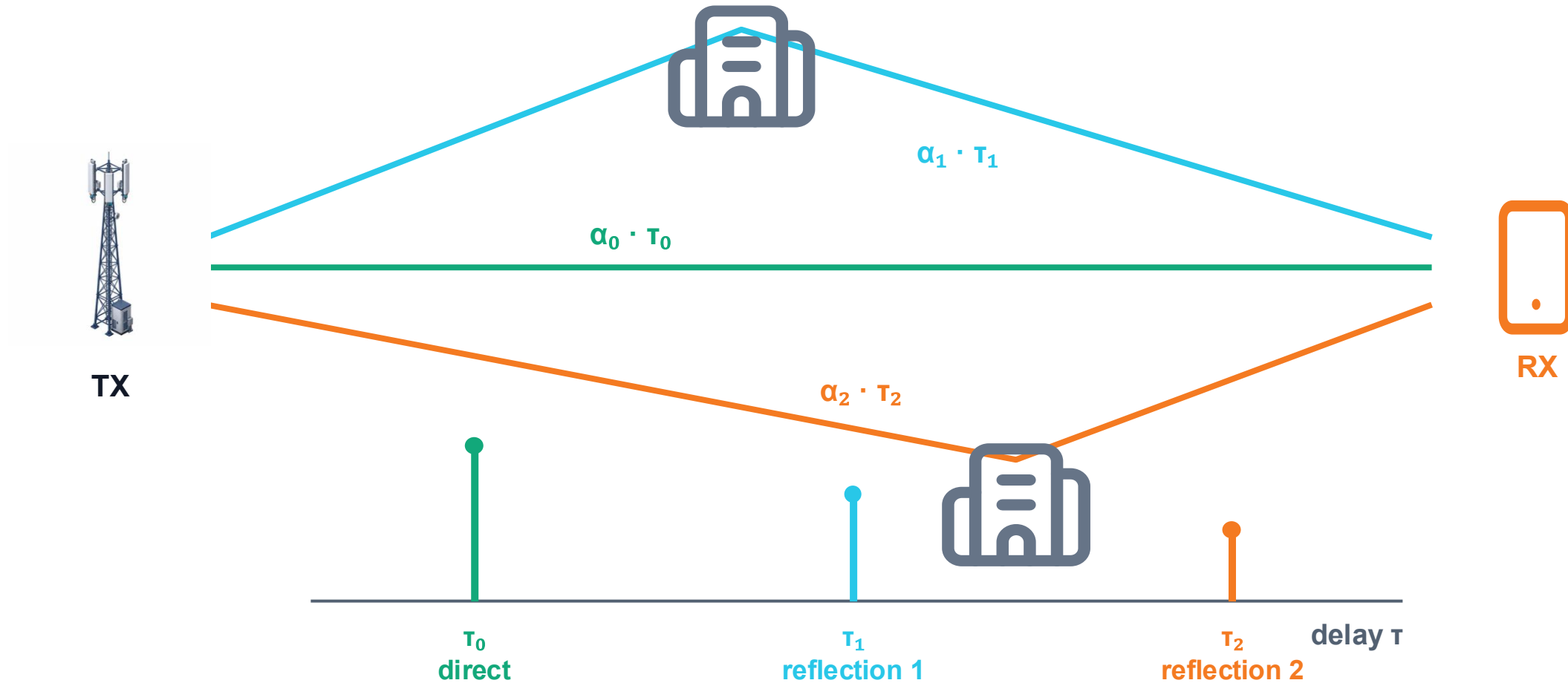
One path produces one copy with gain α_0 and delay τ_0 .

Different path lengths create delayed, scaled copies



Path 2 adds another copy with its own gain and delay.

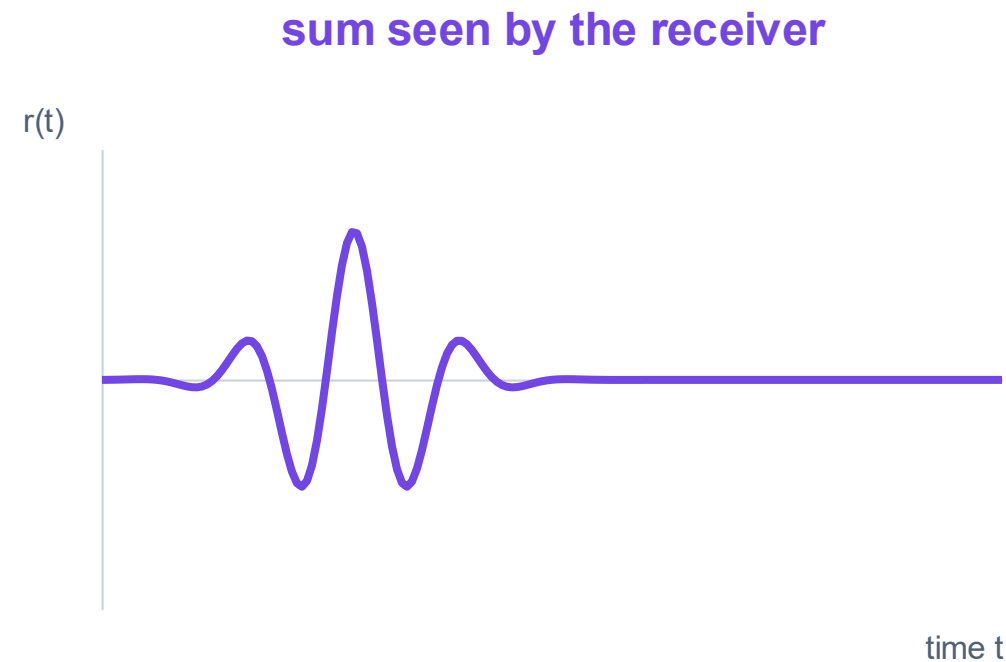
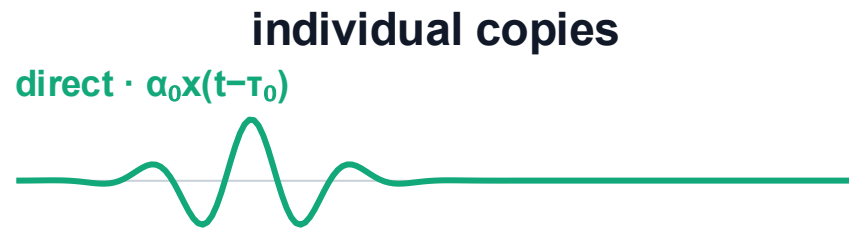
Different path lengths create delayed, scaled copies



Path 3 adds another copy with its own gain and delay.

At the antenna, delayed copies add sample by sample

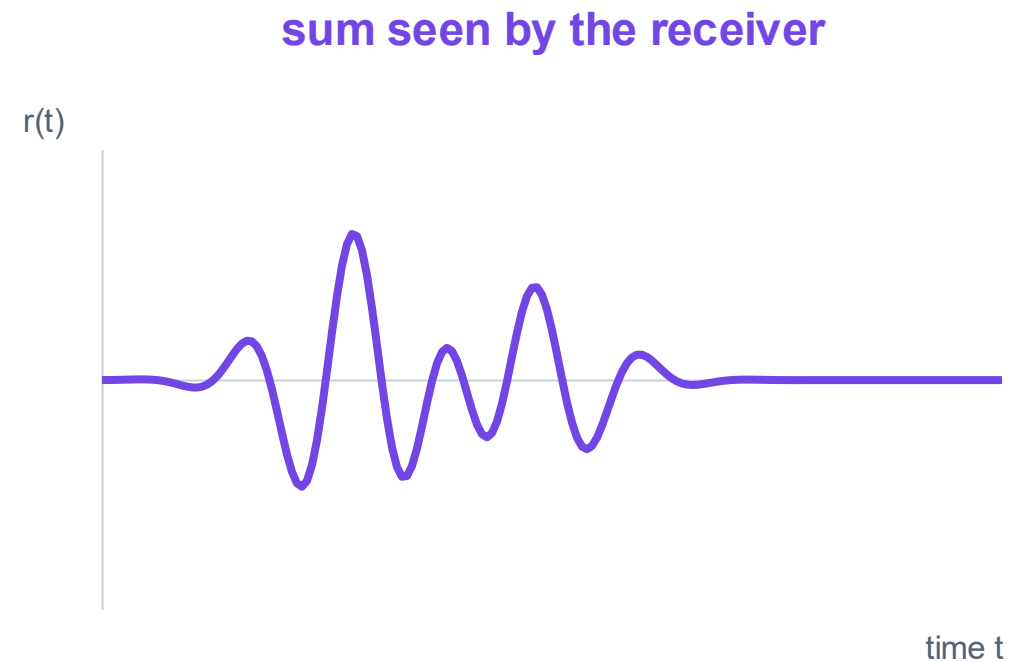
$$r(t) = \alpha_0 x(t - \tau_0)$$



With one path, the receiver sees one delayed and scaled waveform.

At the antenna, delayed copies add sample by sample

$$r(t) = \alpha_0 x(t - \tau_0) + \alpha_1 x(t - \tau_1)$$



Adding a path changes the received sum—even though the transmitter sent the same $x(t)$.

At the antenna, delayed copies add sample by sample

$$r(t) = \alpha_0 x(t - \tau_0) + \alpha_1 x(t - \tau_1) + \alpha_2 x(t - \tau_2)$$

individual copies

direct · $\alpha_0 x(t - \tau_0)$



echo 1 · $\alpha_1 x(t - \tau_1)$



echo 2 · $\alpha_2 x(t - \tau_2)$



sum seen by the receiver

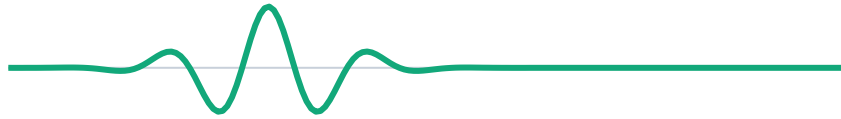


Adding a path changes the received sum—even though the transmitter sent the same $x(t)$.

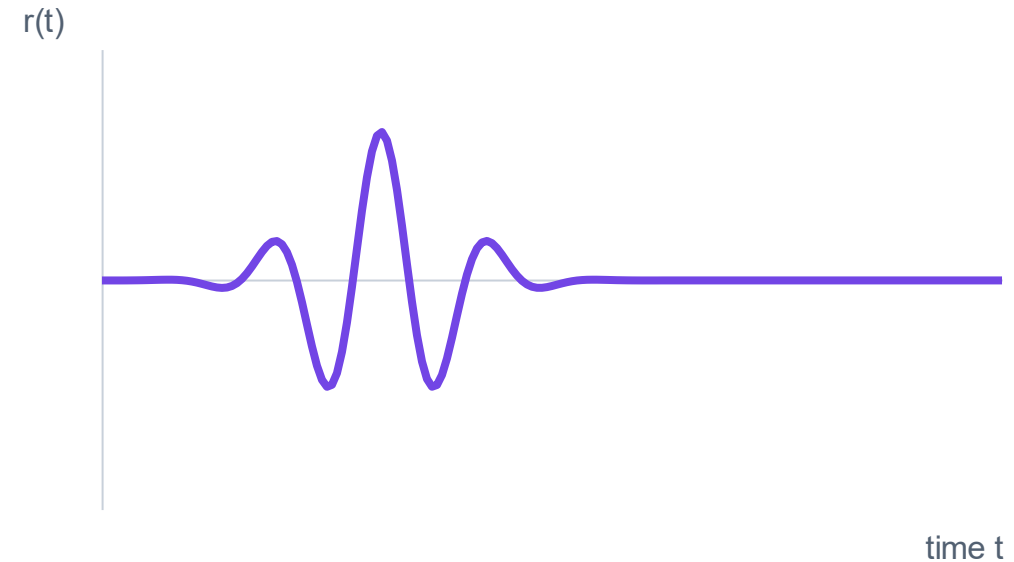
Close—but unequal—path delays make the copies overlap and interfere

$$r(t) = \alpha_0 x(t - \tau_0)$$

small but nonzero delay offsets
direct · τ_0



sum seen by the receiver



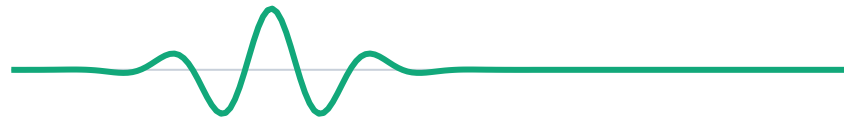
Start the small-delay-spread example with the direct copy only.

Close—but unequal—path delays make the copies overlap and interfere

$$r(t) = \alpha_0 x(t - \tau_0) + \alpha_1 x(t - \tau_1)$$

small but nonzero delay offsets

direct · τ_0

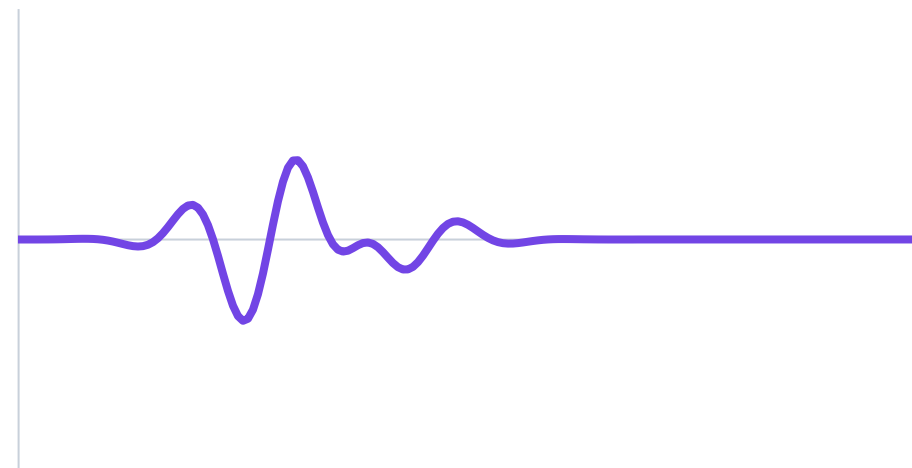


echo 1 · $\tau_1 = \tau_0 + \Delta\tau$



sum seen by the receiver

$r(t)$



time t

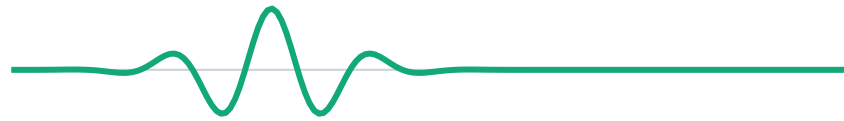
Add the first nearby echo. It is shifted—not identical—so the overlap reshapes the sum.

Close—but unequal—path delays make the copies overlap and interfere

$$r(t) = \alpha_0 x(t - \tau_0) + \alpha_1 x(t - \tau_1) + \alpha_2 x(t - \tau_2)$$

small but nonzero delay offsets

direct · τ_0



echo 1 · $\tau_1 = \tau_0 + \Delta\tau$

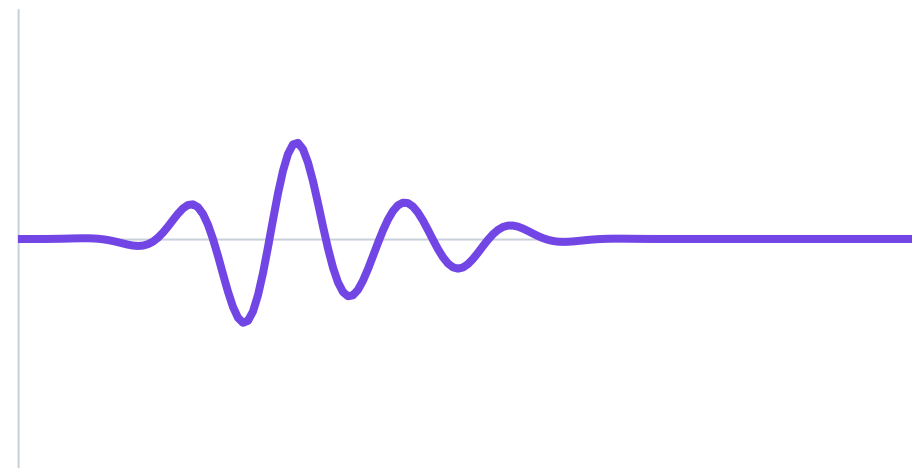


echo 2 · $\tau_2 = \tau_0 + 2\Delta\tau$



sum seen by the receiver

$r(t)$



time t

Add the second nearby echo. The receiver now sees one changed waveform, not three separate bursts.

The received waveform is written by the physical environment



**THE ENVIRONMENT
IS THE CHANNEL**

geometry

materials

motion

**each route has its own
gain α and delay τ**

Geometry, materials, motion, and device orientation all change the signal that reaches the antenna.

The received waveform is written by the physical environment



**THE ENVIRONMENT
IS THE CHANNEL**

geometry

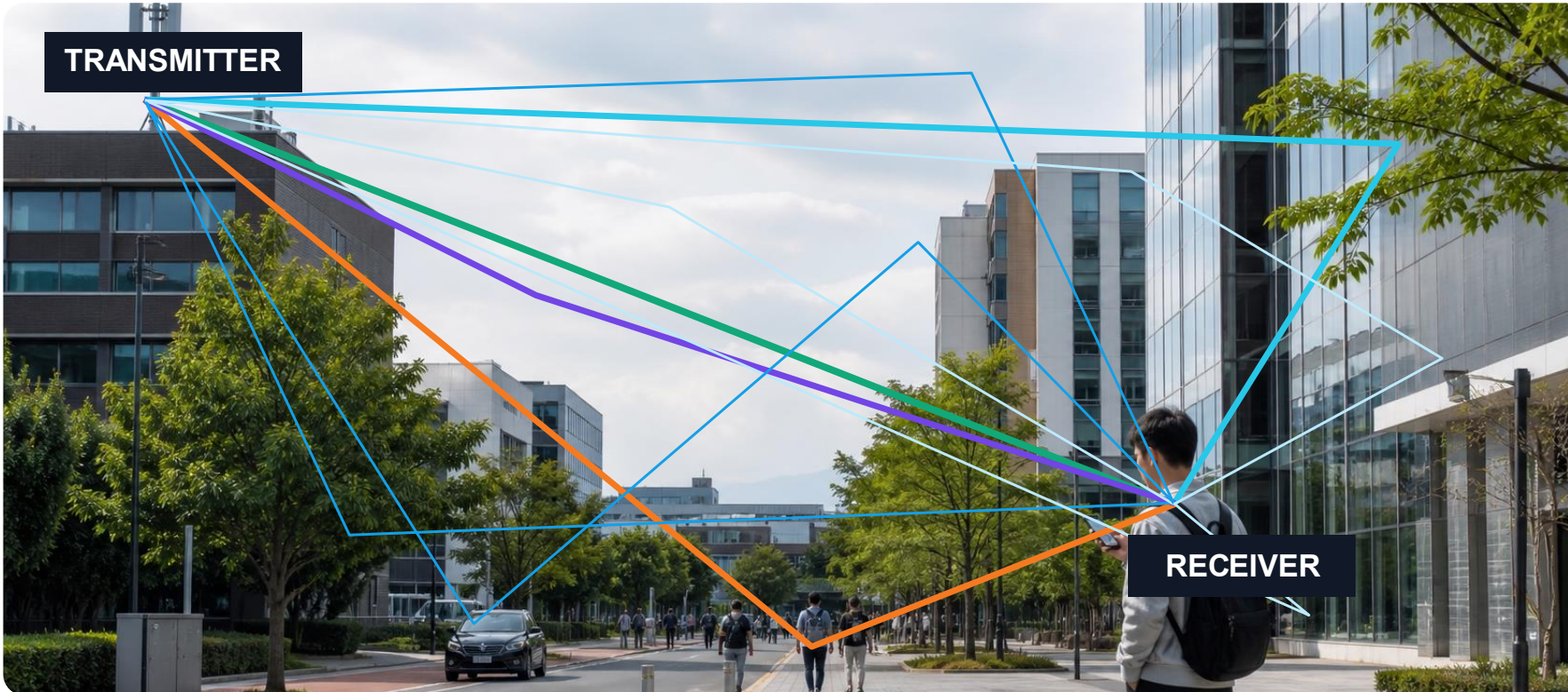
materials

motion

each route has its own
gain α and delay τ

The antenna collects direct, reflected, diffracted, and scattered copies—not one clean path.

The received waveform is written by the physical environment



**THE ENVIRONMENT
IS THE CHANNEL**

geometry

materials

motion

each route has its own
gain α and delay τ

**Path-by-path prediction
is impractical**

Small environmental changes alter many copies, so real-time path-by-path prediction is impractical.

Instead of predicting every path, measure the channel as a black box



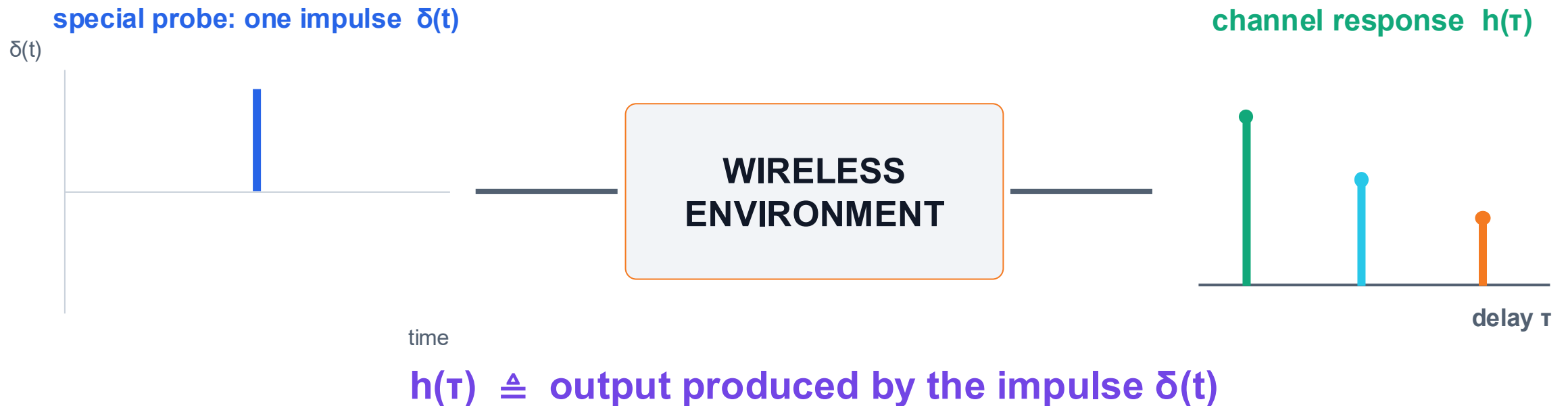
The receiver does not need a perfect 3D map; it needs the environment's effect on signals.

Instead of predicting every path, measure the channel as a black box



An impulse contains one sharply localized event. Every delayed arrival is therefore visible in the output.

Instead of predicting every path, measure the channel as a black box



That measured output is the channel impulse response: the channel's reusable input-output fingerprint.

The impulse response compresses the whole environment into gain versus delay

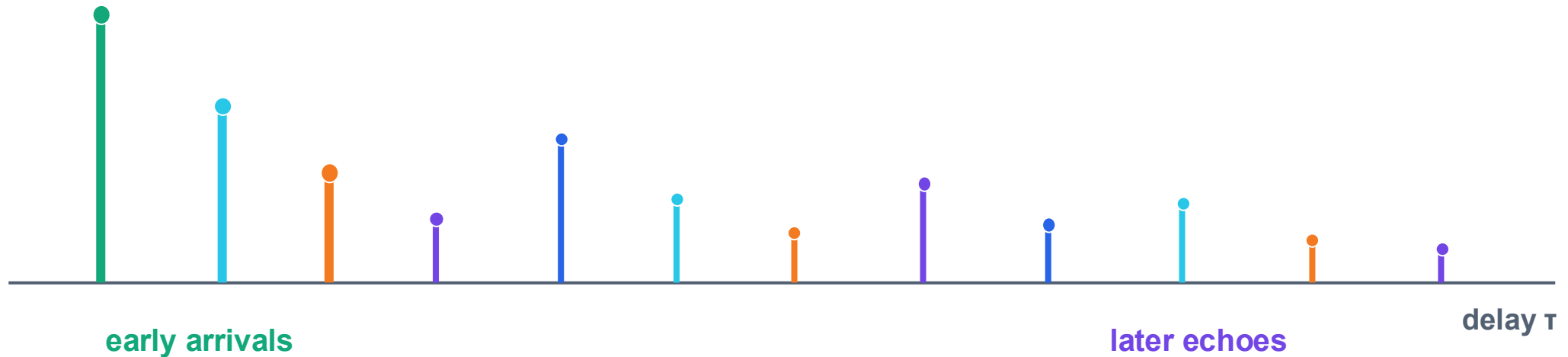


Begin with the arrivals we can resolve: every tap says how much signal arrives and when.

The impulse response compresses the whole environment into gain versus delay

$$h(\tau) = \sum_{l=0}^{L-1} \alpha_l \delta(\tau - \tau_l)$$

many physical rays can collapse into one resolvable delay tap



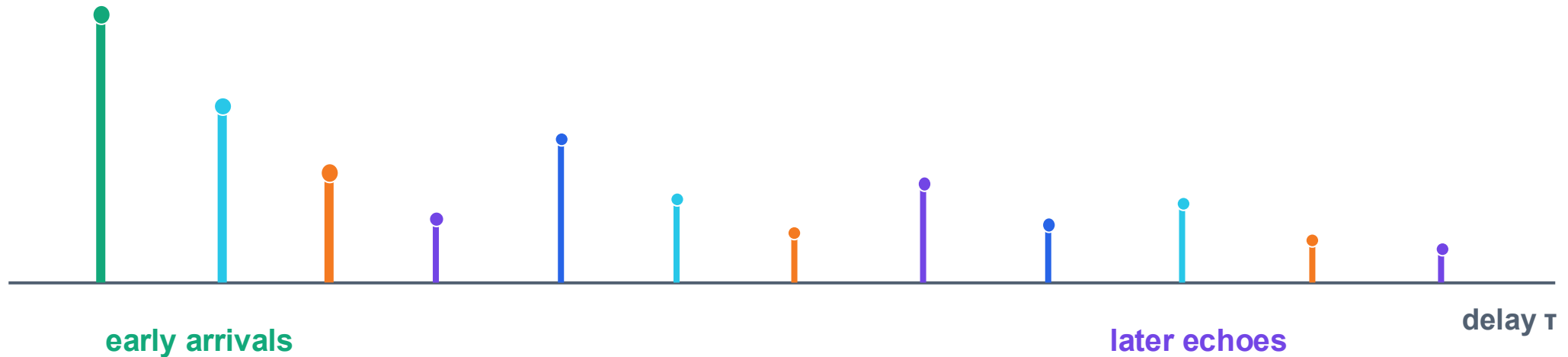
The detailed scene becomes a finite fingerprint $h(\tau)$: a set of effective gains at different delays.

The impulse response compresses the whole environment into gain versus delay

$$h(\tau) = \sum_{l=0}^{L-1} \alpha_l \delta(\tau - \tau_l)$$

$$r(t) = (x * h)(t) + n(t) = \sum_{l=0}^{L-1} \alpha_l x(t - \tau_l) + n(t)$$

many physical rays can collapse into one resolvable delay tap



Once $h(\tau)$ is known, convolution predicts how this environment transforms any transmitted waveform $x(t)$.

The receiver estimates the channel, then approximately reverses its effect



The receiver sees the transmitted waveform only after the unknown channel has reshaped it.

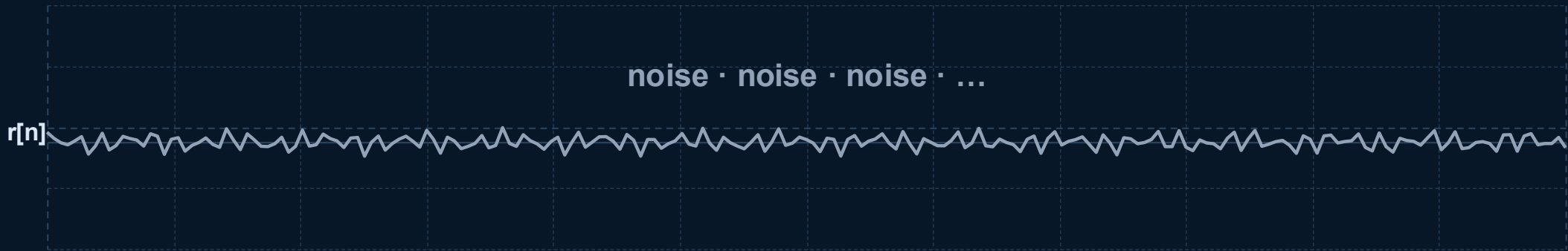
03

Recover the transmitted bits

For each stage: see the failure, isolate why it happens, then add the reference that fixes it.

The receiver listens continuously; no sample announces a packet

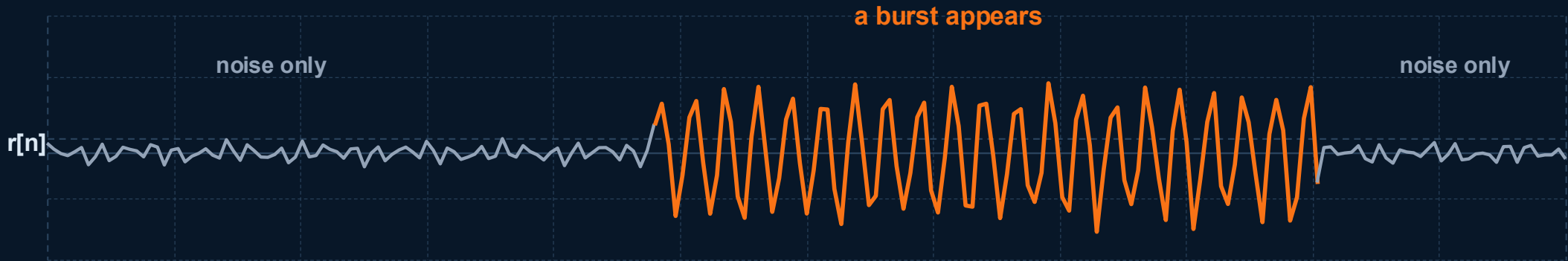
CONTINUOUS RECEIVE STREAM · THE RECEIVER HAS NO OUT-OF-BAND START MARKER



The receiver is always listening; the sample stream does not announce packet boundaries.

Then a finite signal burst appears inside the noise

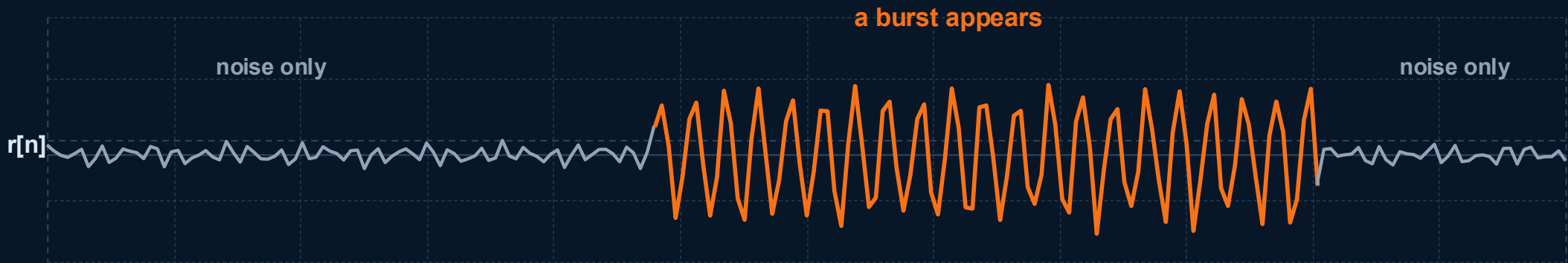
CONTINUOUS RECEIVE STREAM · THE RECEIVER HAS NO OUT-OF-BAND START MARKER



A finite burst is visible to us, but the receiver still needs a repeatable detection rule.

The first question is whether anything arrived

CONTINUOUS RECEIVE STREAM · THE RECEIVER HAS NO OUT-OF-BAND START MARKER



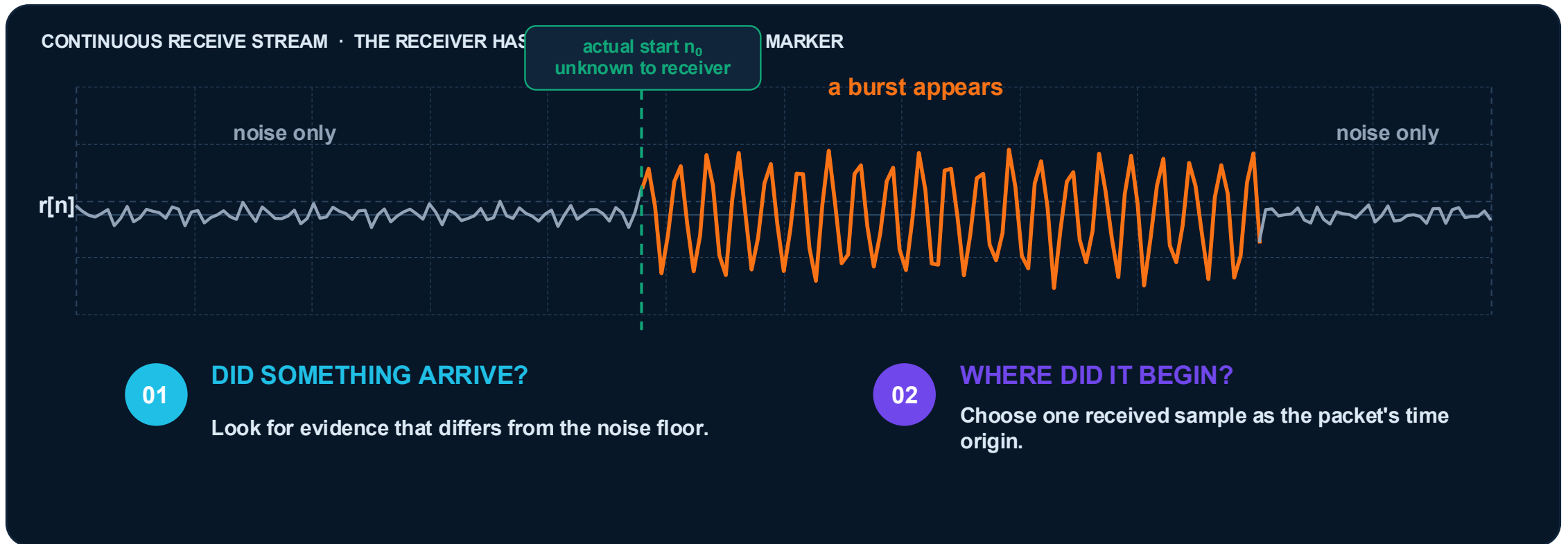
01

DID SOMETHING ARRIVE?

Look for evidence that differs from the noise floor.

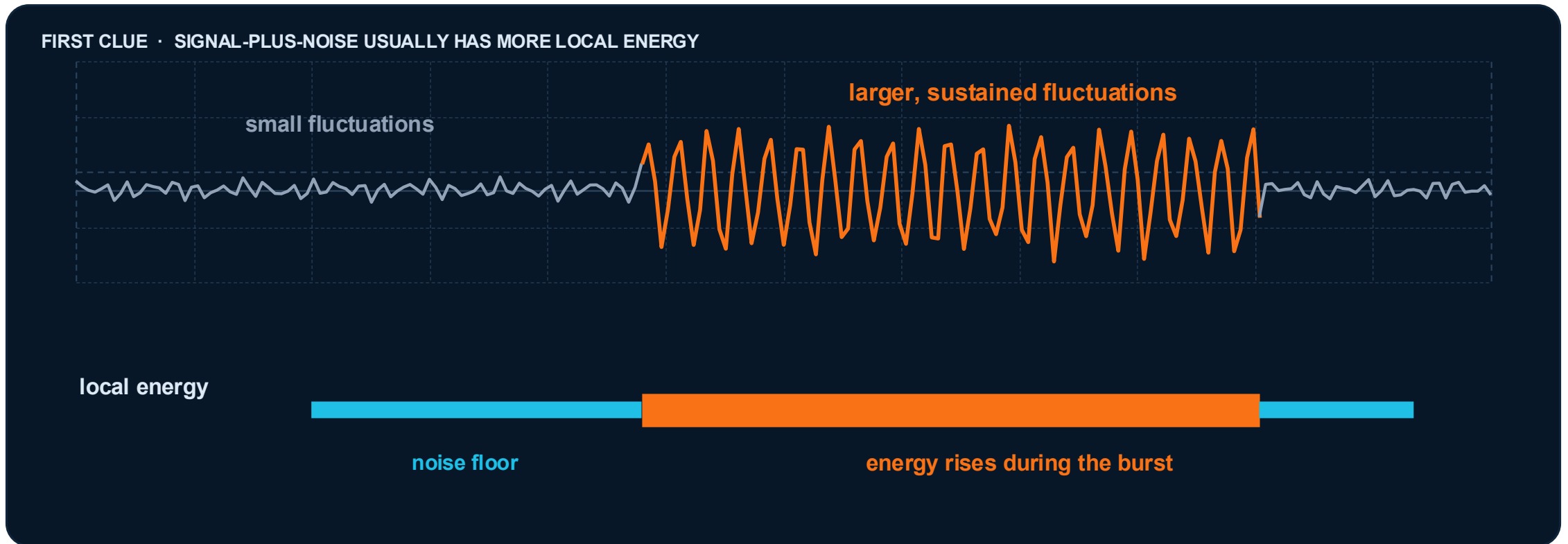
First ask whether the stream changed enough to indicate that something arrived.

The second question is which sample begins the packet



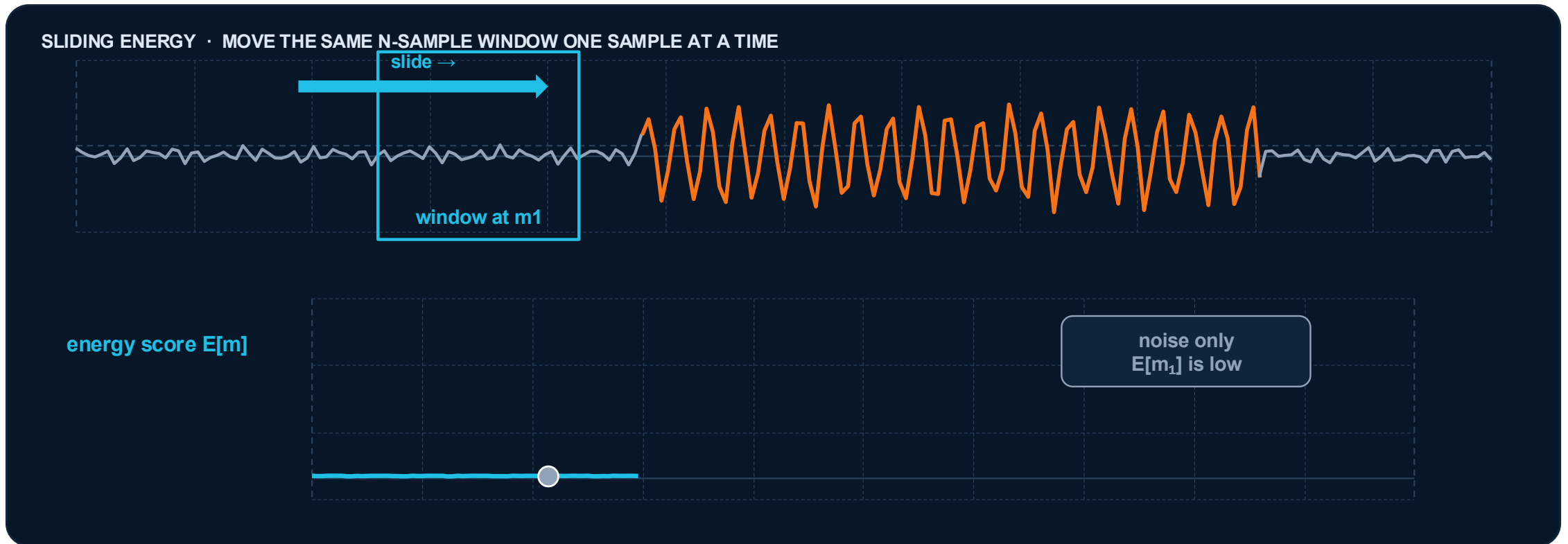
Then identify one sample n_0 as the start; all later timing is measured from that point.

A burst first reveals itself through a rise in received energy



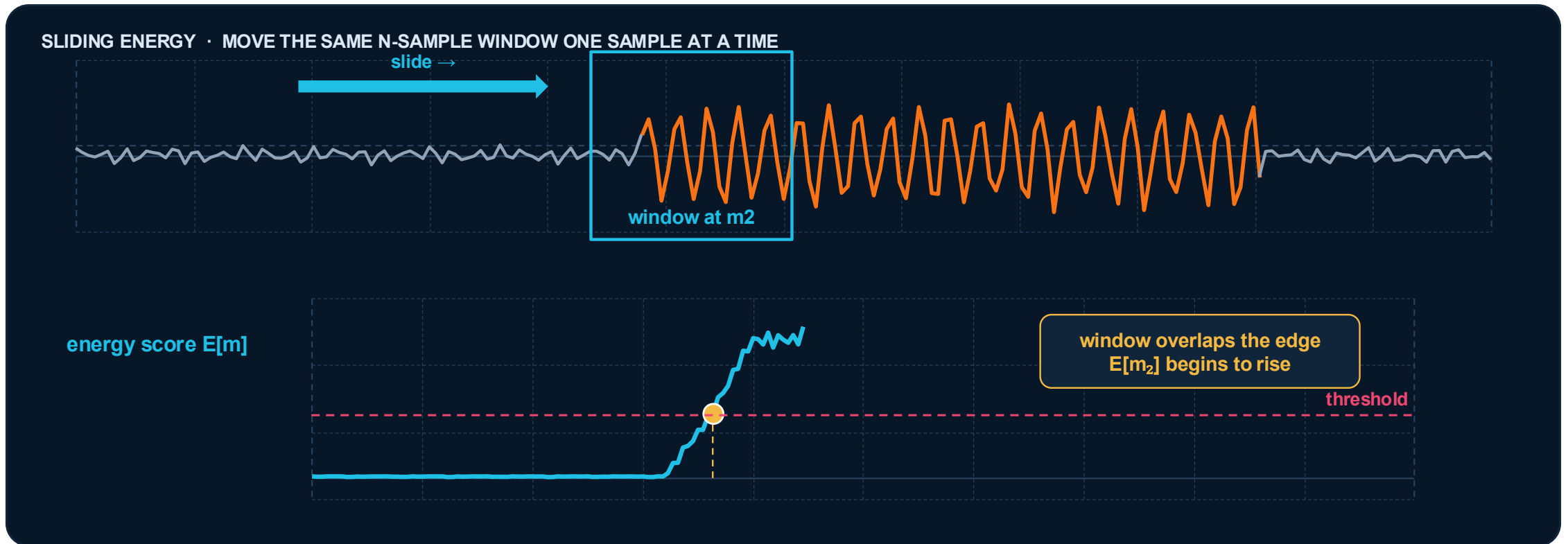
A sustained rise in local energy is a useful first clue that a signal has arrived.

Slide an energy window across time to build a detection score



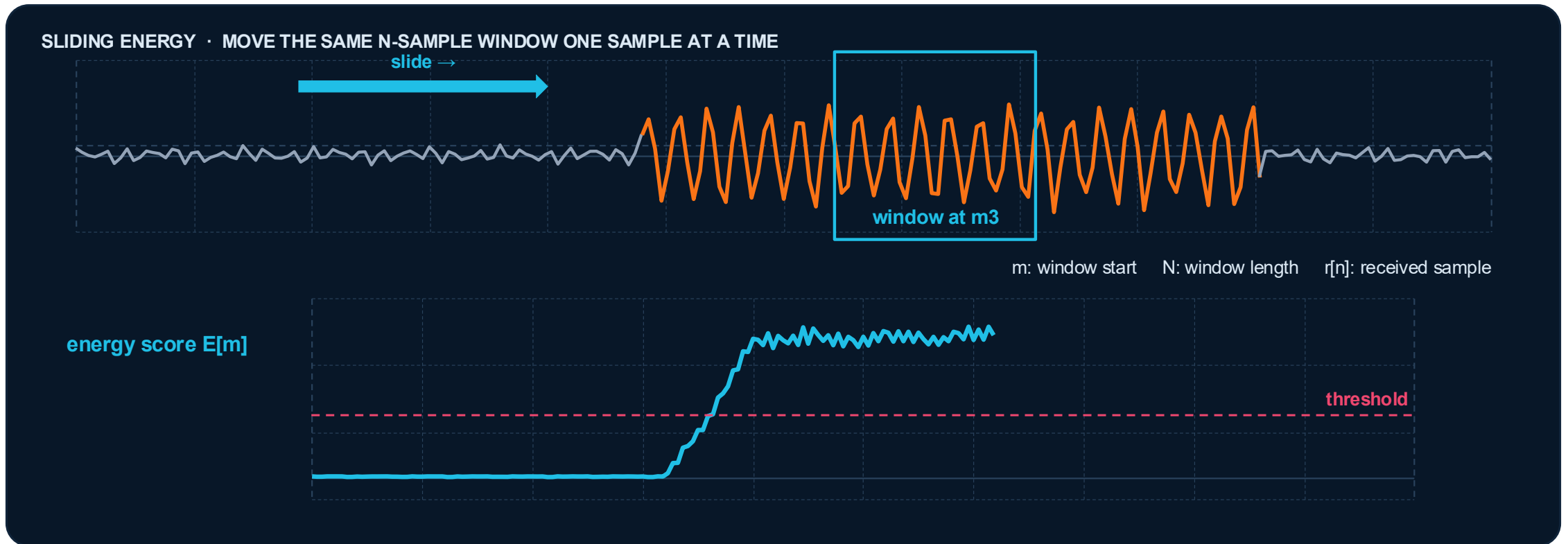
When the window contains only noise, the energy score remains near the noise floor.

Slide an energy window across time to build a detection score



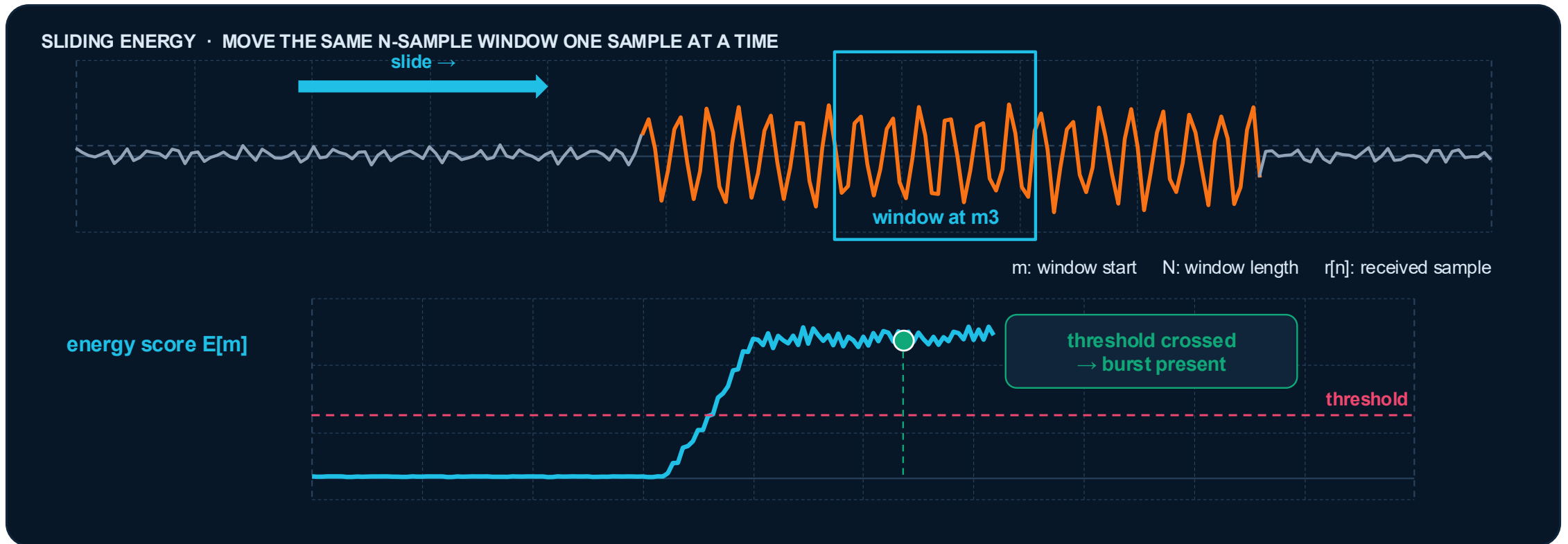
As the window overlaps the burst, the score rises gradually rather than jumping at one exact sample.

Slide an energy window across time to build a detection score



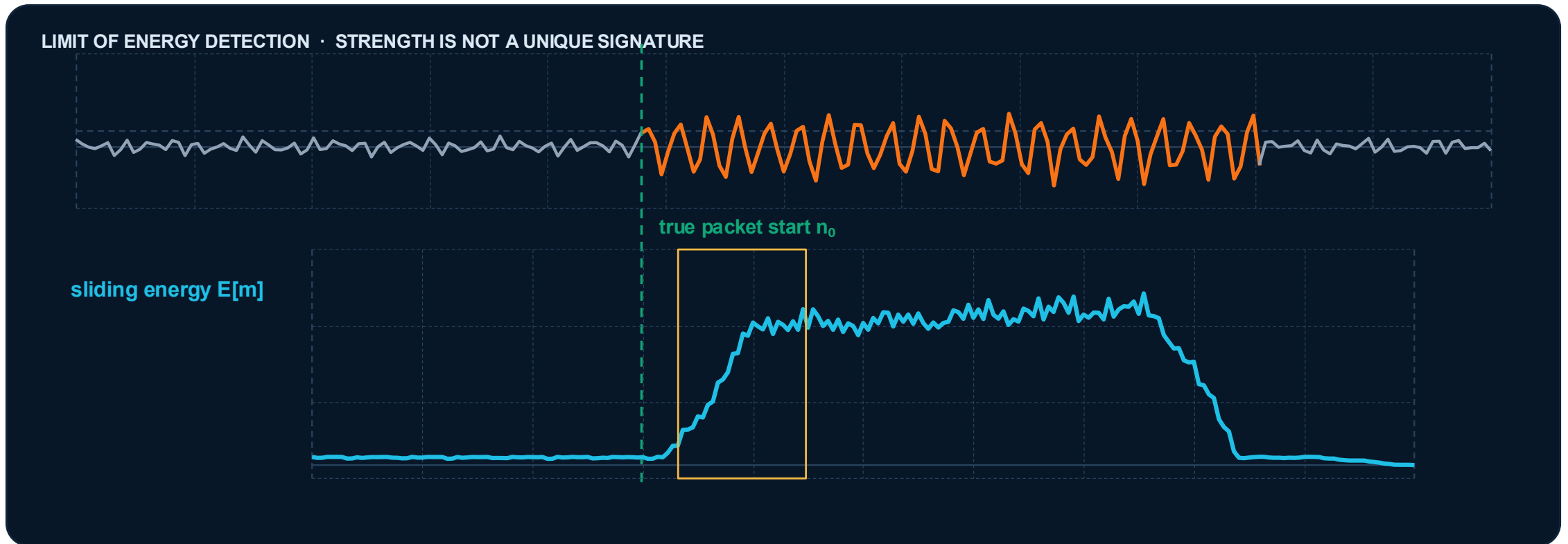
Once the score crosses a threshold, the receiver declares that a burst is probably present.

Slide an energy window across time to build a detection score



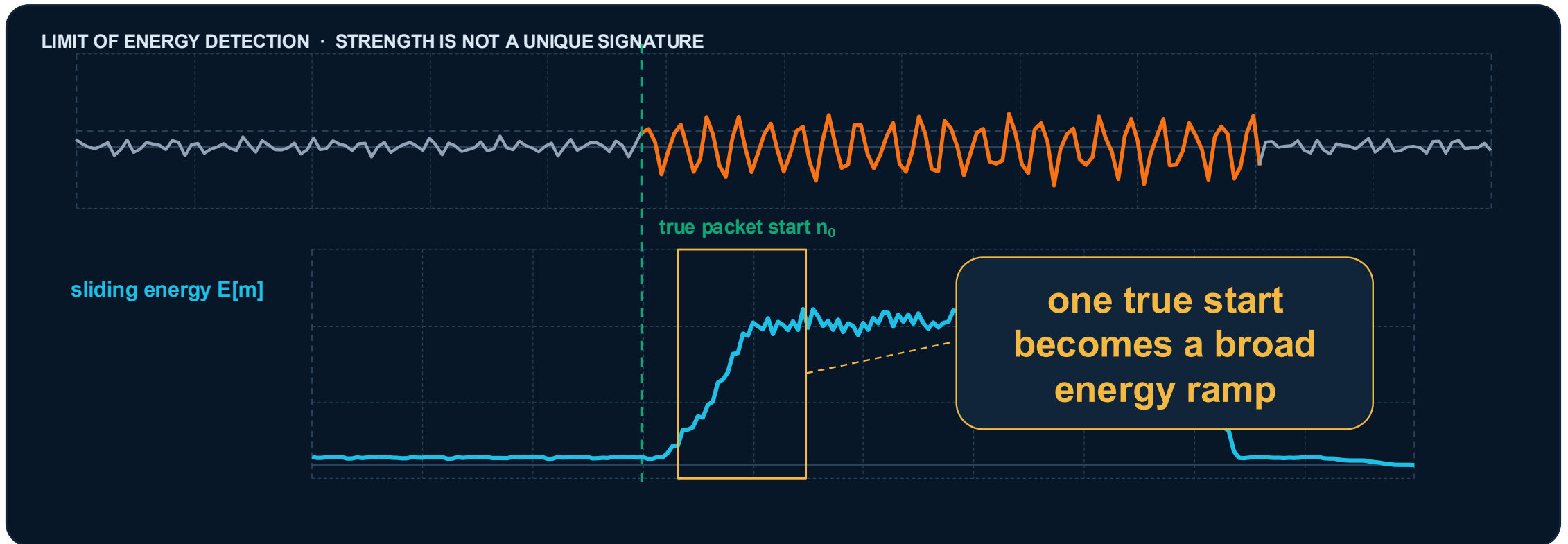
Once the score crosses a threshold, the receiver declares that a burst is probably present.

Energy detection blurs one packet start into a broad transition



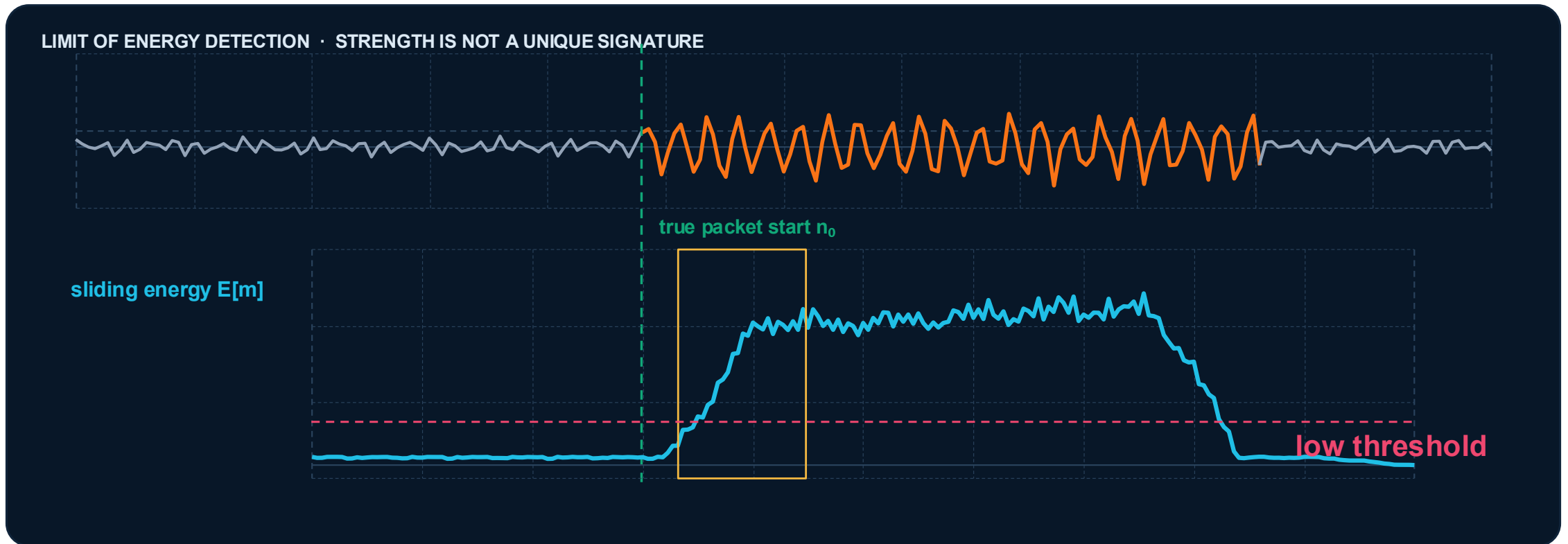
A finite energy window blurs the leading edge, so activity detection gives only a coarse time interval.

Energy detection blurs one packet start into a broad transition



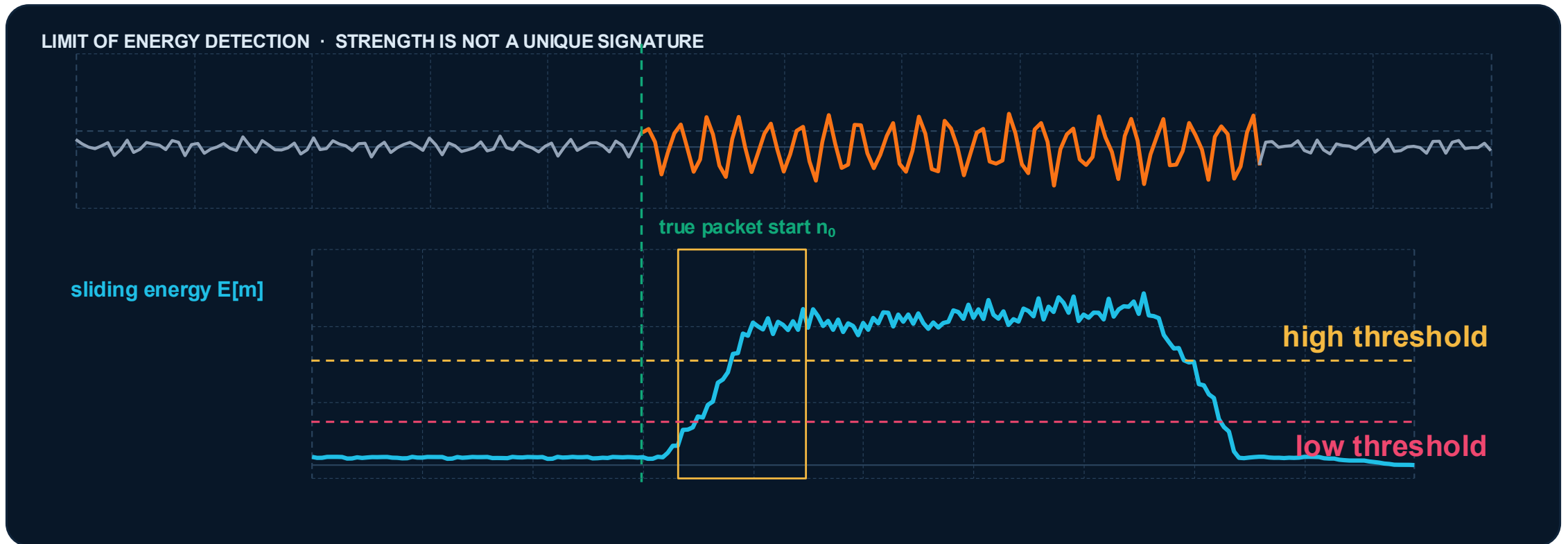
A finite energy window blurs the leading edge, so activity detection gives only a coarse time interval.

Changing the threshold changes the estimated start



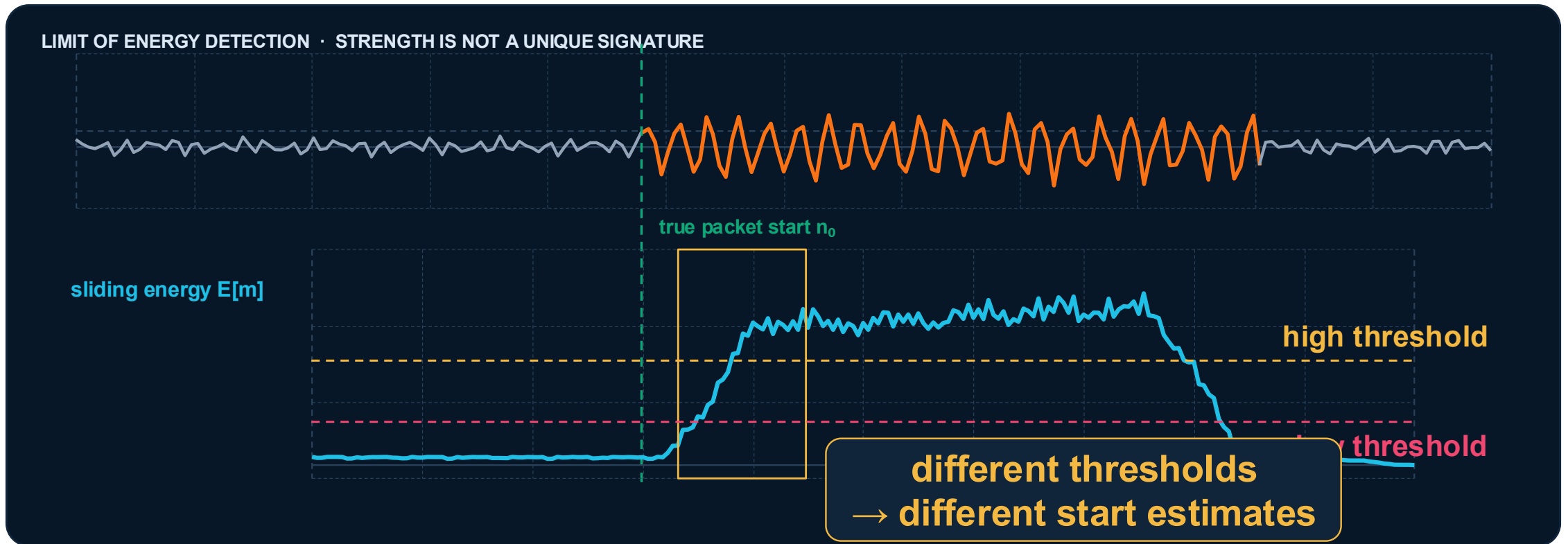
The estimated start changes with threshold, noise floor, fading, and the chosen window length.

Changing the threshold changes the estimated start



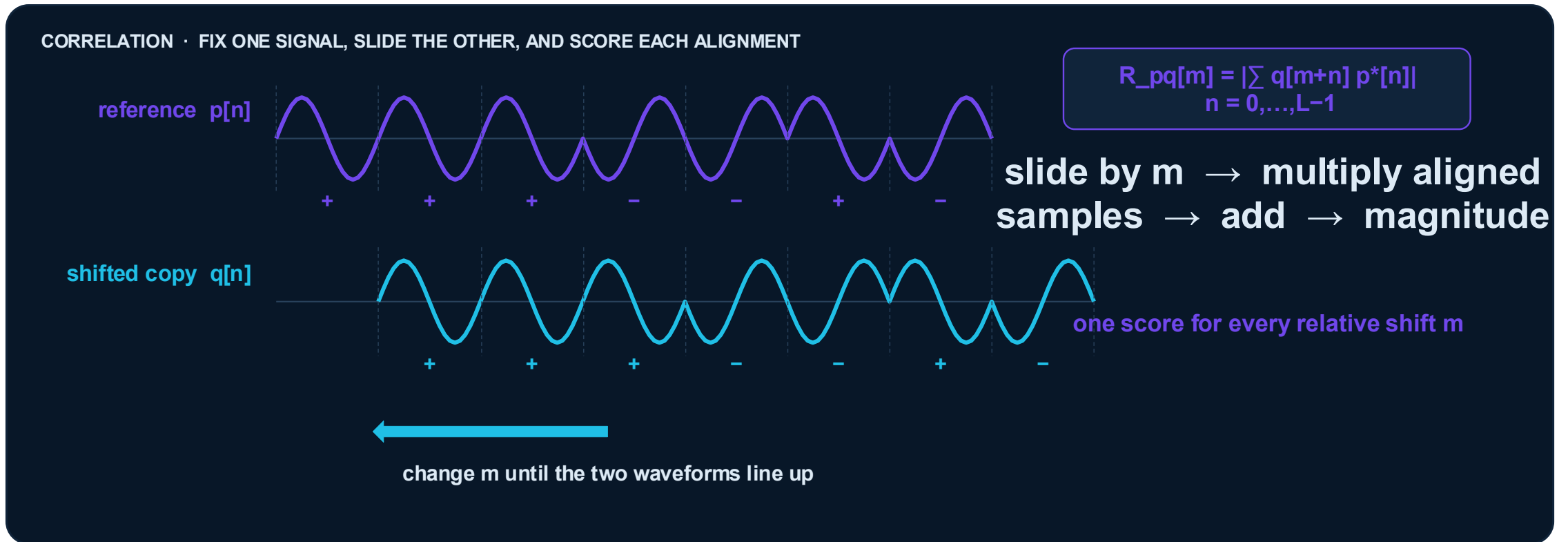
The estimated start changes with threshold, noise floor, fading, and the chosen window length.

Changing the threshold changes the estimated start



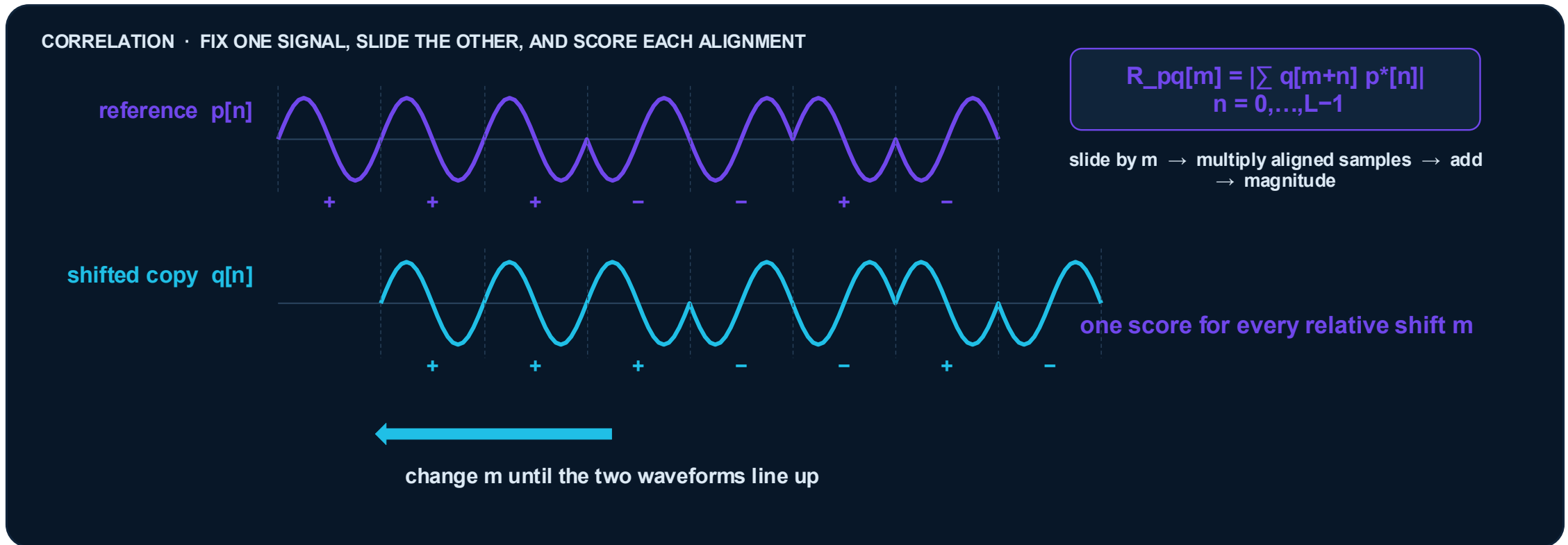
The estimated start changes with threshold, noise floor, fading, and the chosen window length.

Correlation slides one signal against another and scores every alignment



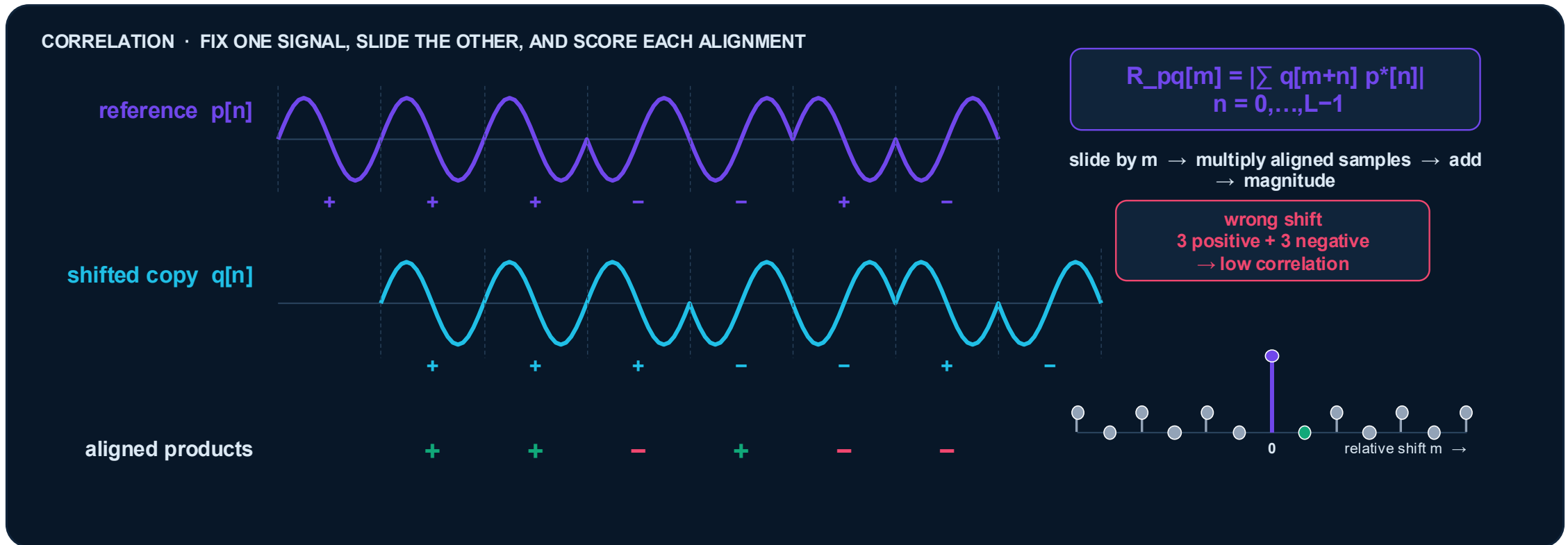
Correlation directly compares two signals: hold $p[n]$ fixed and slide $q[n]$ across it.

Correlation slides one signal against another and scores every alignment



Correlation directly compares two signals: hold $p[n]$ fixed and slide $q[n]$ across it.

When the signals are shifted, positive and negative products cancel



At the wrong shift, positive and negative contributions cancel, so the correlation remains low.

When they align sample-for-sample, correlation produces a peak

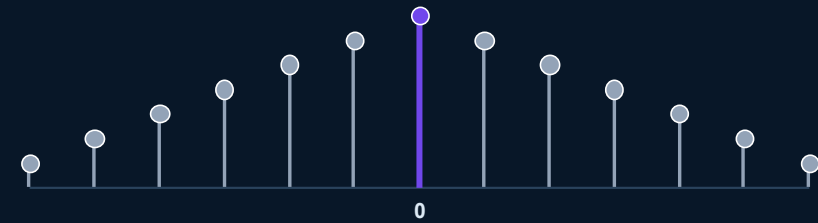
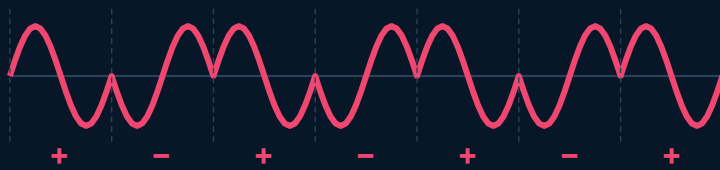


At exact sample-by-sample alignment, every contribution reinforces the sum and creates a peak.

A good sequence makes one alignment peak stand out

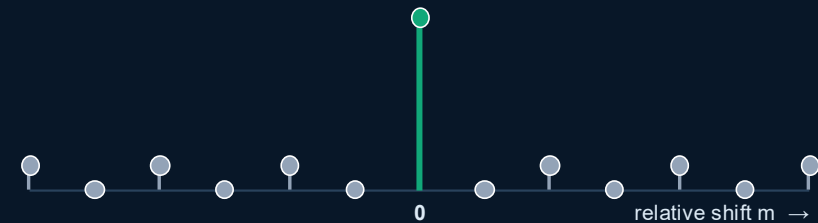
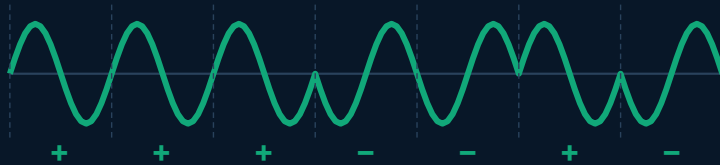
SEQUENCE DESIGN · THE MAIN PEAK MUST STAND ABOVE EVERY OFF-PEAK RESPONSE

repetitive
sequence



large off-peak responses → several plausible alignments

good sync
sequence



low sidelobes + one sharp main peak → unambiguous alignment

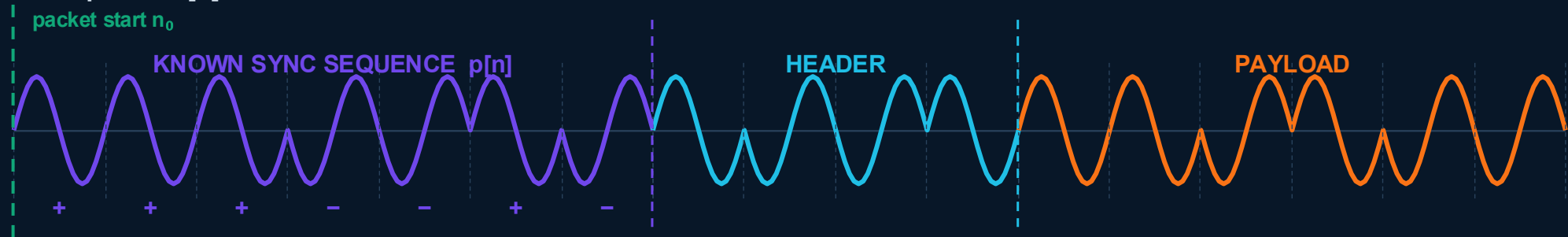
peak-to-sidelobe ratio ↑

With the same length and energy, a good sequence suppresses off-peak sidelobes—so the correct peak stands out.

Place the good sync sequence at the very front of every packet

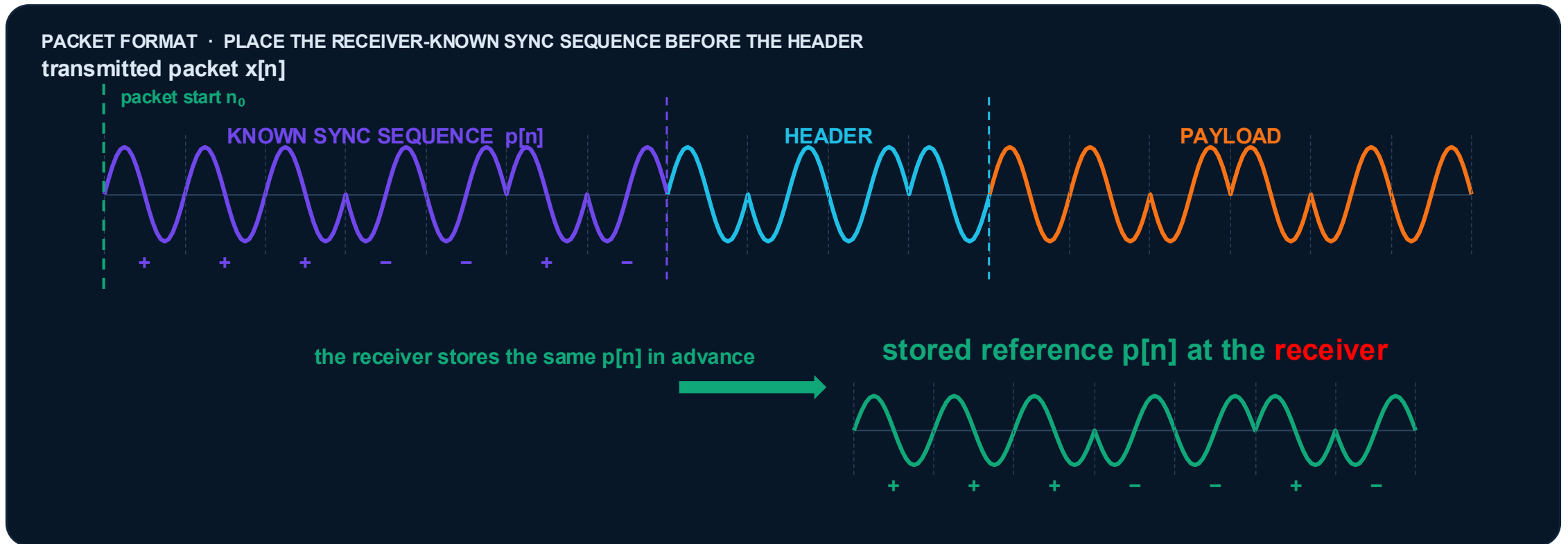
PACKET FORMAT · PLACE THE RECEIVER-KNOWN SYNC SEQUENCE BEFORE THE HEADER

transmitted packet $x[n]$



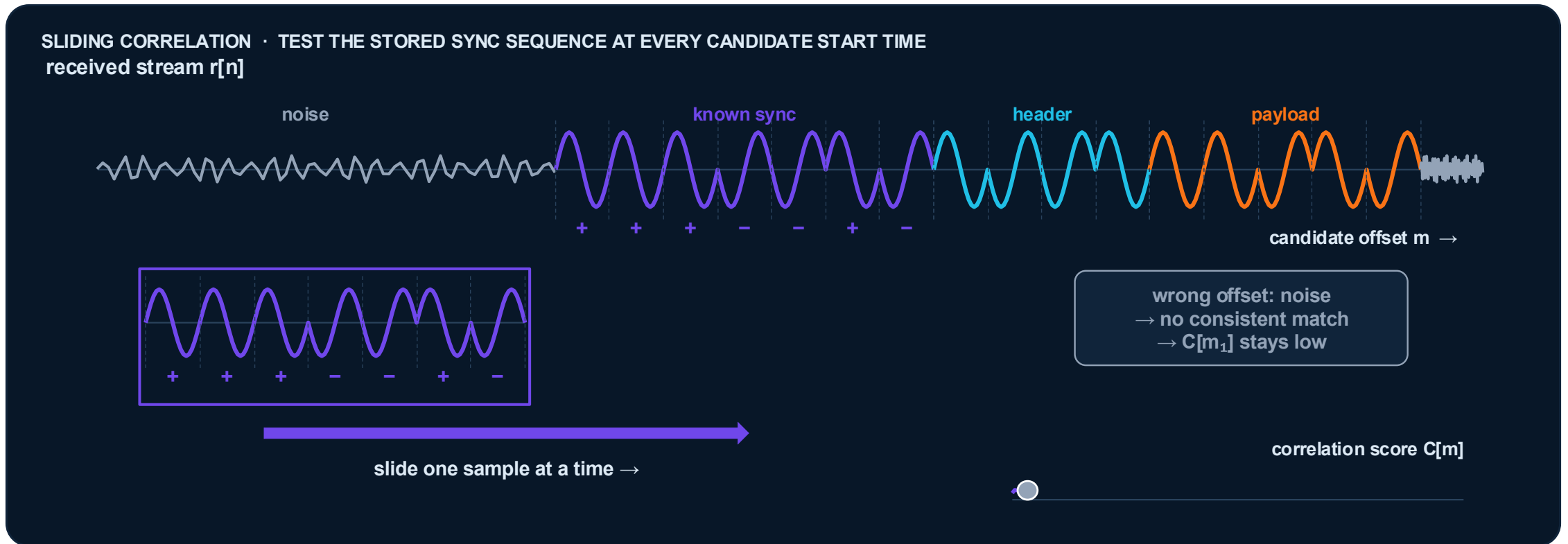
The known sync sequence comes before the header; finding it establishes the packet's time origin.

Place the good sync sequence at the very front of every packet



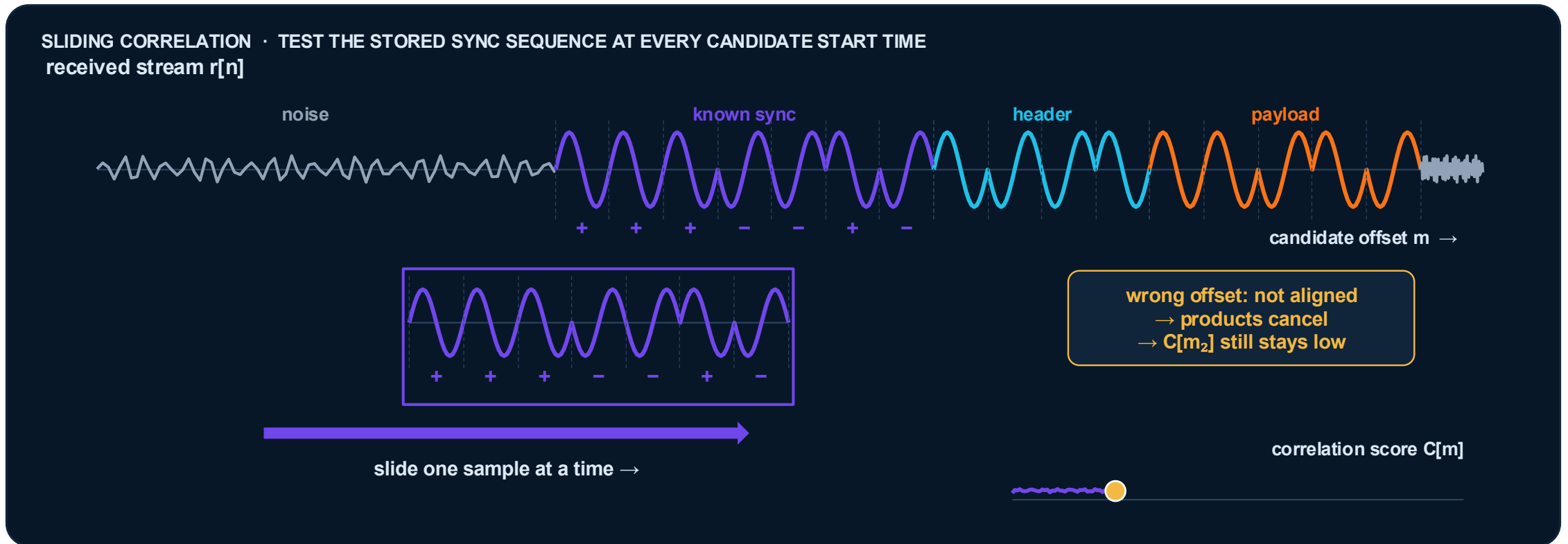
The known sync sequence comes before the header; finding it establishes the packet's time origin.

At a wrong offset, the correlation score stays low



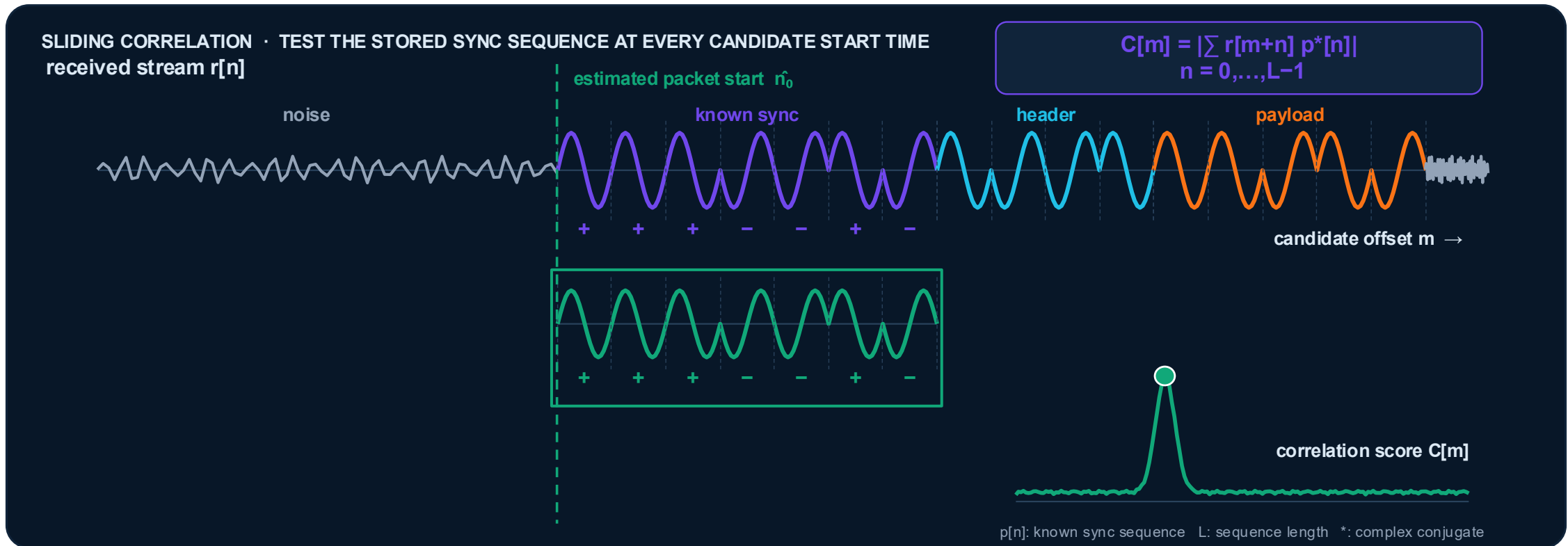
Away from the packet, the stored sync sequence does not match the received samples, so the score stays low.

Near the packet—but still misaligned—the score remains low



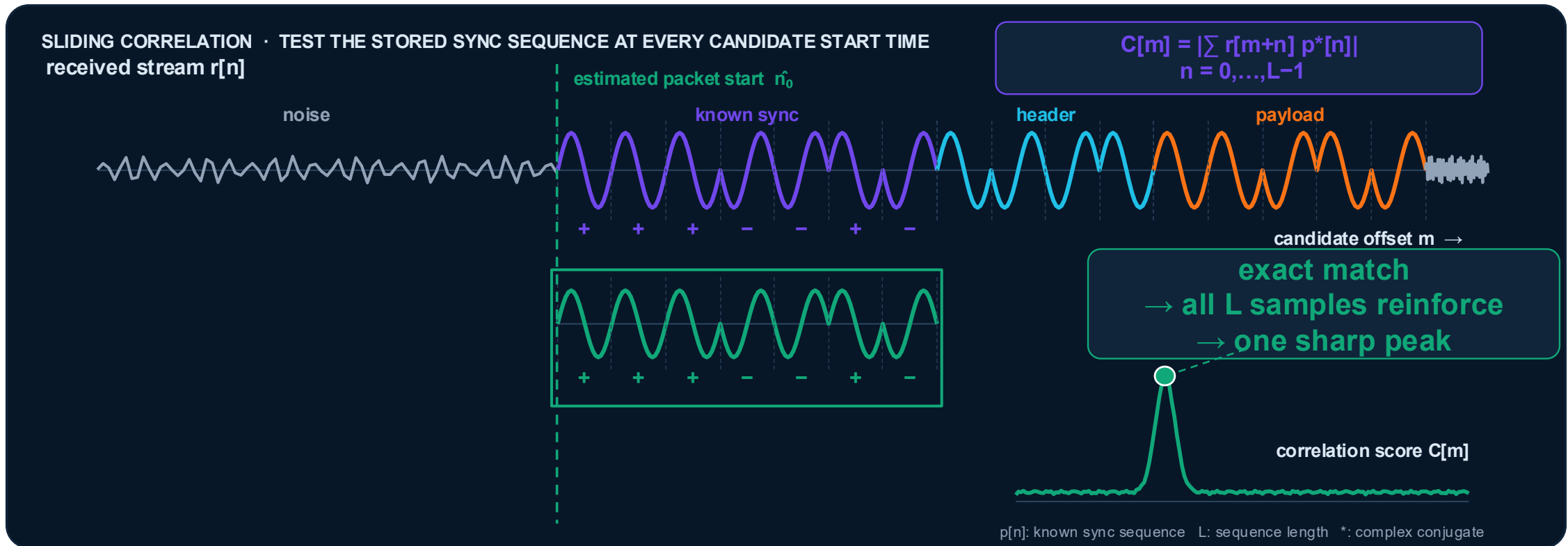
Even near the packet, a good sequence keeps the off-peak response low until the samples align exactly.

Only the exact match creates the peak that marks the packet start



Only the exact match produces the sharp peak; its location is the estimated packet start $\hat{n}_0$.

Only the exact match creates the peak that marks the packet start

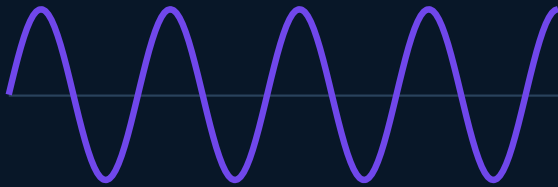


Only the exact match produces the sharp peak; its location is the estimated packet start $\hat{n}_0$.

After detecting the packet, ask how the channel changed the signal

CHANNEL ESTIMATION · INFER AN UNKNOWN CHANNEL FROM A RECEIVER-KNOWN REFERENCE

known reference (pilot) $x_p[n]$

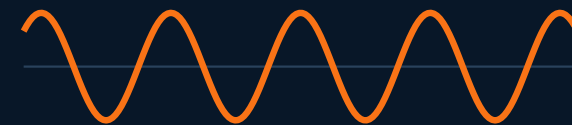


UNKNOWN CHANNEL

$h = ?$



received reference $y_p[n]$



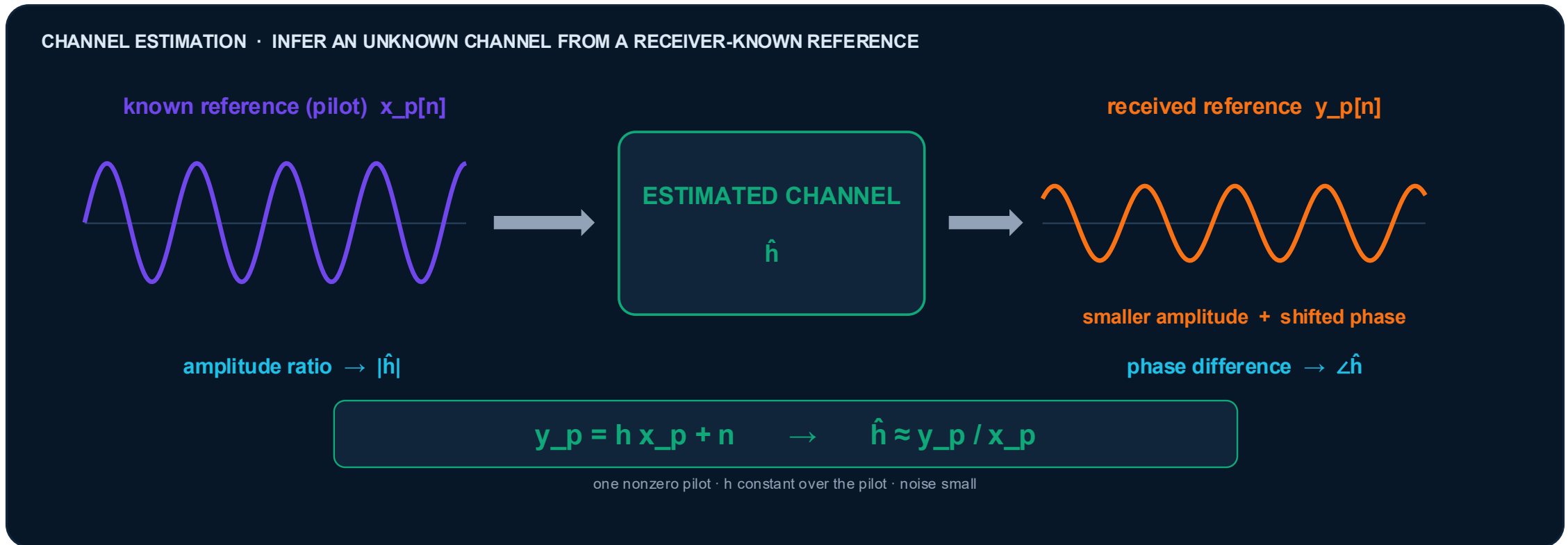
smaller amplitude + shifted phase

What transformation produced this change?

compare what was sent with what arrived

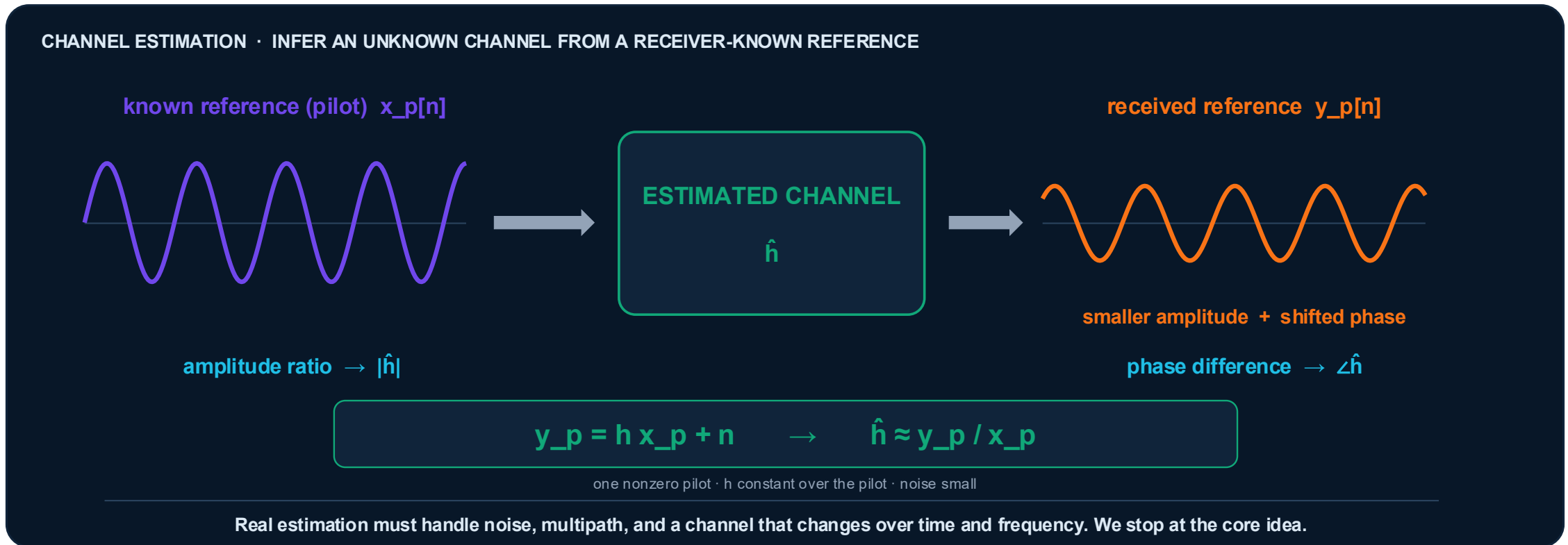
Send a reference the receiver already knows; the difference between x_p and y_p reveals the channel's effect.

Compare a known transmitted reference with what the receiver observes



Compare the received pilot with the known pilot: amplitude ratio and phase difference give a simple estimate of h .

The core idea is simple; practical channel estimation is not



Core idea: infer h from a known reference. Next, align timing and carrier so that comparison is reliable.

Wireless communication connects mobile intelligence to the physical world

- 01 Modulation maps bits to signal states.
- 02 The channel reshapes them in time, frequency, and space.
- 03 The receiver synchronizes and estimates before it decides.

Next: diversity, beamforming, MIMO, and link adaptation.

INFORMATION

COMPUTE

DECIDE

EXECUTE

FEEDBACK